

CHAPTER 2

Computer Instruction Set

Computer architecture is defined as the study of the components and their interconnections that form a computer system. The computer supports the instruction types and data, which become primary architectural considerations. In this chapter some important characteristics and properties of computer instruction sets are discussed. Topics will include: op-code encoding, addressing modes, and instruction types.

2.1 INTRODUCTION

An instruction manipulates the stored data, and a sequence of instructions constitutes a program. In general, an instruction has two components:

- Op-code field
- Address field(s)

The op-code field specifies how data is to be manipulated. The data items may reside within a CPU register or in the main memory. The purpose of the address field is to indicate the data address. When operations require data to be read from or stored

into two or more addresses, the address field may contain more than one address. For example, consider the following instruction:

ADD	R1, R0
op-code field	address field

Assume that this computer uses R1 as the source register and R0 as the destination register. The preceding instruction then adds the contents of CPU registers R0 and R1 and saves the sum in register R0. The number and types of instructions supported by a computer vary from one computer to another and depend primarily on the architecture of a particular machine.

Depending on the number of addresses specified, one can have the following instruction formats:

- Three-address
- Two-address
- One-address
- Zero-address

Since all instructions are stored in the main memory, instruction formats are designed so that instruction sizes are optimized and have powerful processing capabilities. The CPU architecture has considerable influence on a specific instruction format. For example, zero-address instructions are very predominant in stack machines.

The following are some important technical points that have to be considered when designing an instruction format:

- The size of an instruction word is chosen by the designer to specify several operations. For example, with 4- and 8-bit op-code fields, 16 and 256 distinct operations, respectively, can be specified.
- Instructions are used to manipulate various data elements, such as integers, floating-point numbers, and character strings. In particular, all programs written in a symbolic language such as FORTRAN or Pascal are internally stored as characters. Therefore, memory space will not be wasted if the word length of the machine is some integral multiple of the number of bits needed to represent a character. Since all characters are represented using typical 8-bit character codes such as ASCII or EBCDIC, it is desirable to have an 8-, 16-, 32-, or 64-bit word length.
- The size of the address field is chosen to guarantee high resolution. In any computer, the ultimate resolution is a bit. Memory resolution is a function of the instructional length and, in particular, of short instructions that provide less resolution. For example, in a computer with 32K 16-bit memory words, at least 19 bits are required to access each bit of the word. However, the resolution achieved by some processors lies at the extremes. For example, the Burrough's B1700 processor achieves a 1-bit resolution (each bit is addressable). CDC's CYBER 70 series computer, however, has a minimum addressable unit of one 60-bit memory word.

Instruction	Meaning
LDA addr	$\text{Acc} \leftarrow (\text{addr})$
STA addr	$M(\text{addr}) \leftarrow (\text{Acc})$
ADD addr	$\text{Acc} \leftarrow (\text{Acc}) + (\text{addr})$
AND addr	$\text{Acc} \leftarrow (\text{Acc}) \wedge (\text{addr})$
CMA	$\text{Acc} \leftarrow (\text{Acc})'$
INCA	$\text{Acc} \leftarrow (\text{Acc}) + 1$
JMP addr	Unconditionally branch to addr
HLT	Halt CPU

Figure 2.1 A Hypothetical Instruction Set

2.2 OP-CODE ENCODING

A processor can execute an instruction only if it is represented as a binary sequence. A unique binary pattern must be assigned to each op-code. This process is known as *op-code encoding*. Some popular op-code encoding techniques are addressed in this section.

The simplest way to carry out op-code encoding is to assign a fixed length of binary pattern to each op-code. For example, a k -bit binary pattern can represent 2^k distinct op-codes. This method is known as the *block-code technique*. To illustrate this concept, consider the hypothetical instruction set shown in Figure 2.1. In this figure, there are 8 different instructions that can be encoded using a 3-bit binary pattern $p_2p_1p_0$ (Figure 2.2).

Op-Code	Binary Pattern
	$p_2p_1p_0$
LDA	000
STA	001
ADD	010
AND	011
CMA	100
INCA	101
JMP	110
HLT	111

Figure 2.2 Op-code Encoding Using a 3-bit Block Code

The op-codes of the hypothetical instruction set can be decoded using a 3-to-8 decoder such as the 74LS138 shown in Figure 2.3. An n -to- 2^n decoder is required for an n -bit op-code. As the value of n increases, the cost of the decoder and decoding time also increases.

Some op-code encoding techniques are considered in which the length of the op-code is a function of parameters such as the number of addresses or the relative frequency of its usage. The following approaches are discussed:

- Expanding op-code technique
- Huffman encoding

The rationale behind expanding op-code technique is to find a compromise between the instruction length and memory resolution. Consider an instruction in which the lengths of the op-code and address fields are 4 and 12 bits, respectively. Using such a format, 16 operations can be specified that will allow access to 4096 memory locations. If the size of the address field is increased to 13 bits and the instruction length is kept at 16 bits, the op-code length is three bits. However, this change will reduce the number of possible operations by 50%. At the same time, it will increase memory resolution by 100%. Conversely, the original instruction format can be changed so that the sizes of the op-code and address field are 5 and 11 bits, respectively.

With this new format, 16 more operations can be specified, which will be a 100% increase. However, this gain results in a 50% reduction in memory resolution (because the 11-bit address field allows access to only 2048 different memory locations). This is the basic concept behind the expanding op-code technique. The following example is provided to explain the usefulness of this approach.

Consider an instruction format with an instruction length and address field size of 8 and 3 bits, respectively. Only 4 distinct, two-address instructions can be formed because the op-code field has only 2 bits. This result is illustrated in Figure 2.4.

If there are 3 two-address instructions rather than 4, 8 one-address instructions can also be specified. This happens because each one-address instruction requires only one address field, and the other 3-bit address-field can be used for specifying the 8 op-codes. This idea is illustrated in Figure 2.5.

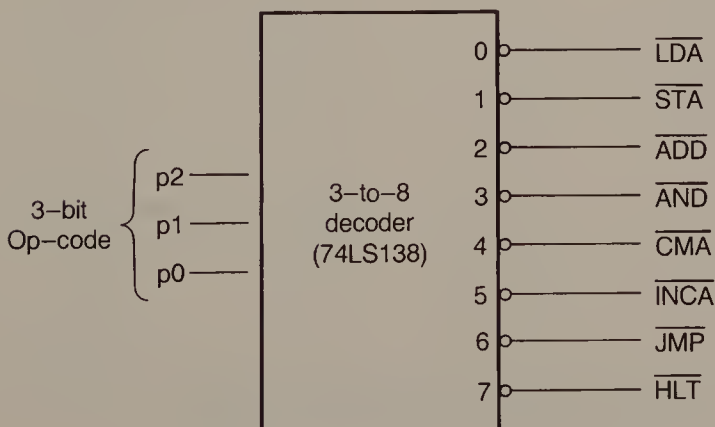


Figure 2.3 Instruction Decoder

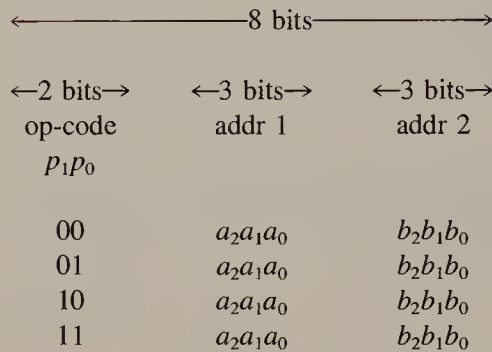


Figure 2.4 Four, two-address Instructions Are Derived Using a 2-bit Op-code Field

The length of the op-code for each one-address instruction is 5 bits. This means the length of the op-code field increases as the number of address fields is decreased. For this reason, this technique is referred to as the expanding op-code technique.

In a typical instruction set, it is necessary to include some zero-address instructions. Suppose the number of one-address instructions is reduced from 8 to 7. Then up to 8 zero-address instructions can be accommodated in the same instruction format. This is illustrated in Figure 2.6.

For zero-address instructions, 8 bits are used for op-code specification. The expanding op-code technique is employed to encode the instruction set of the PDP-11 computer.

Huffman's encoding scheme is discussed next. The block-code technique assumes all instructions are used with equal probability. In practice, not all instructions are used with the same relative frequency count. On the average, 40% of the instructions used in a program are load and store instructions. This pattern is similar to the occurrence of

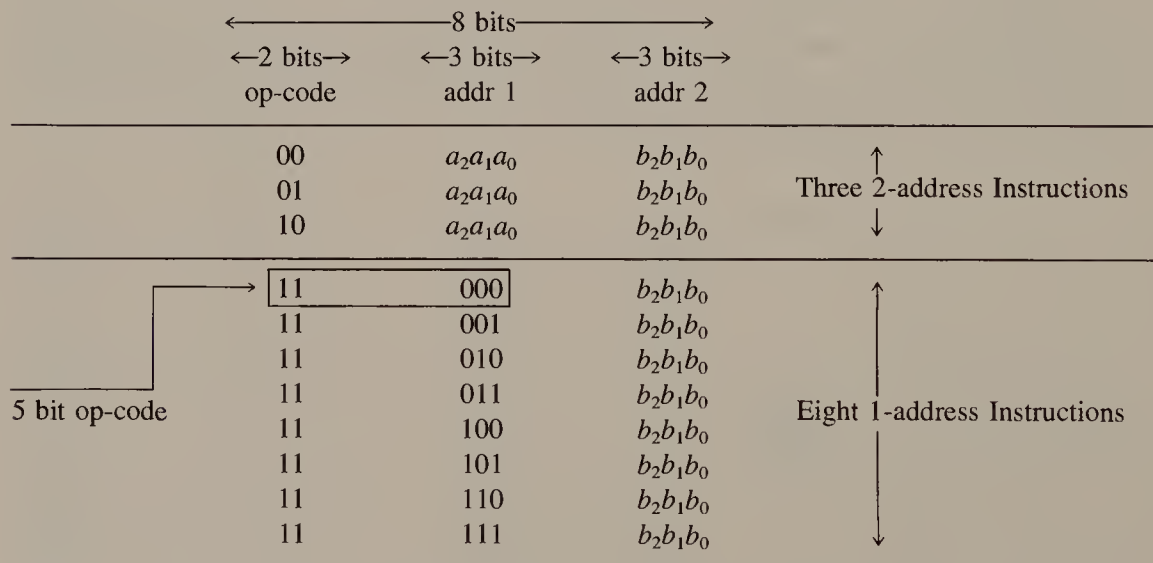


Figure 2.5 Three 2-address and Eight 1-address Instructions Using 8-bit Instruction Format

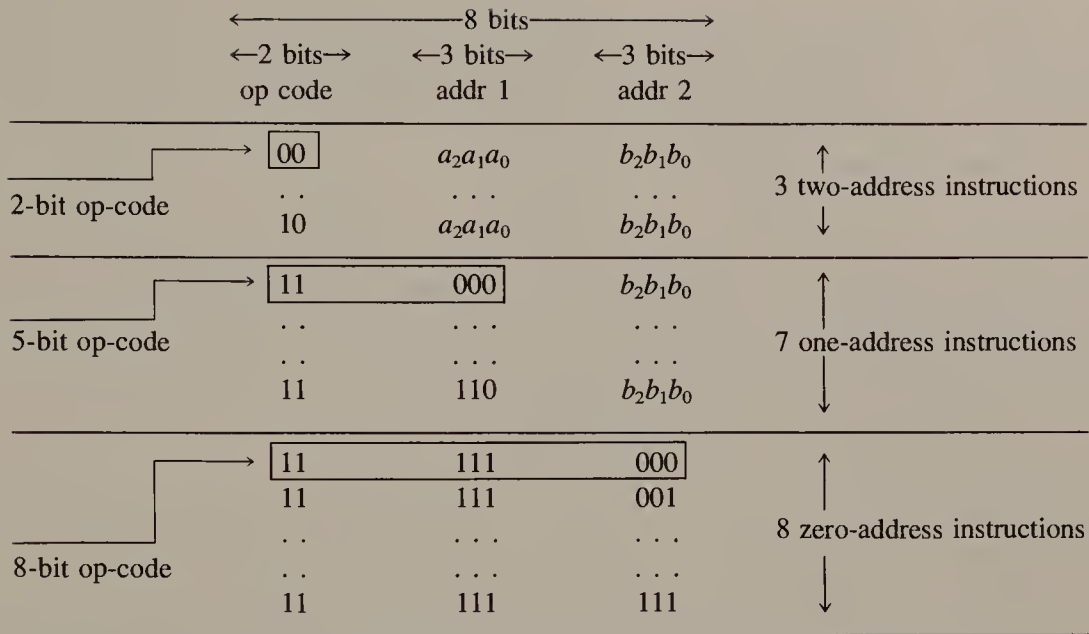


Figure 2.6 Illustration of 3 two-address, 7 one-address, and 8 zero-address instructions in an 8-bit instruction format

vowels in an English text. Therefore, this study warrants an encoding scheme that will encode the op-code of the most frequently used instructions with fewer bits and the least frequently used instructions with more bits. This allows the average number of bits required to encode a typical program to be optimum.

Huffman's procedure carries out this idea, as explained by the following example. Suppose it is desired to encode the hypothetical instruction set shown in Figure 2.7. A

Instruction Mnemonic	Relative Frequency Count
LOAD	$\frac{1}{4}$
STO	$\frac{1}{4}$
ADD	$\frac{1}{8}$
AND	$\frac{1}{8}$
NOT	$\frac{1}{16}$
RSHIFT	$\frac{1}{16}$
JUMP	$\frac{1}{16}$
HALT	$\frac{1}{16}$

Figure 2.7 An Instruction Set with Relative Frequency Count

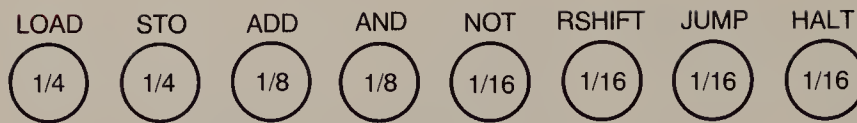


Figure 2.8 Initial Arrangement of Instruction Mnemonics

relative frequency count of each instruction is shown in this figure. These values are obtained by inspecting the occurrence of each instruction in a set of representative programs. In this procedure, first arrange the instructions to obtain a graph, as shown in Figure 2.8. In this graph, there is one node for each instruction mnemonic, and these nodes are labeled with the corresponding relative frequency count. The nodes are arranged in the ascending order of their relative frequency counts.

Next, scan all the nodes of the graph and select two nodes with minimum values. Create a new node with a value equal to the sum of these minimum values. Exclude the two nodes that were picked up from the subsequent scanning process. This result is shown in Figure 2.9. This figure is developed by scanning the nodes of Figure 2.8. Here, the nodes corresponding to the instruction mnemonics JUMP and HALT are shown to have the least values. So these two nodes are selected, and new node with a value $\frac{1}{8}$ ($\frac{1}{16} + \frac{1}{16} = \frac{1}{8}$) is created. After this, these nodes are excluded from the subsequent process by crossing them out. The nodes corresponding to the mnemonics NOT and RSHIFT or RSHIFT and JUMP could have been chosen. When there are several possibilities, the choice is arbitrary. The important aspect of this procedure is to select two nodes with minimum values and form a new node with a value equal to the sum of the two chosen nodes. If the scanning process is continued, a new graph develops, as shown in Figure 2.10.

This scanning process is continued until a single node with a value equal to 1 is formed. At this point, a tree is formed, as shown in Figure 2.11.

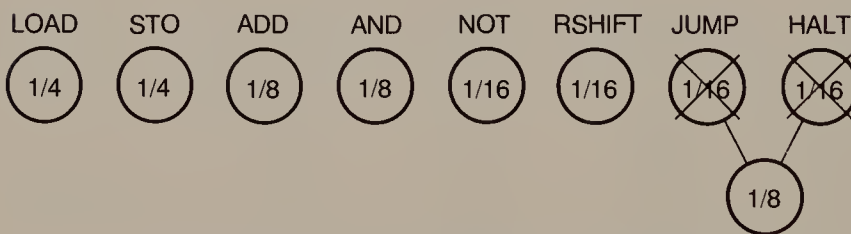


Figure 2.9 The Result of the Initial Scanning of the Nodes in Figure 2.8

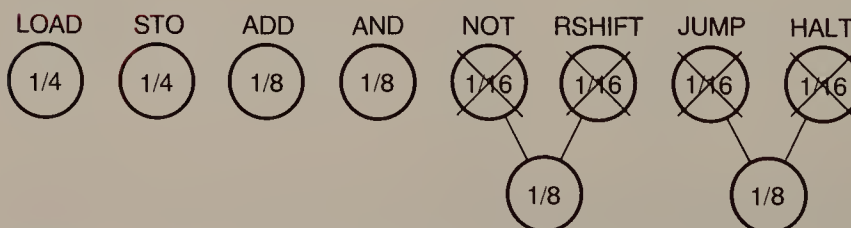


Figure 2.10 The Result of Scanning of Nodes of the Figure 2.9

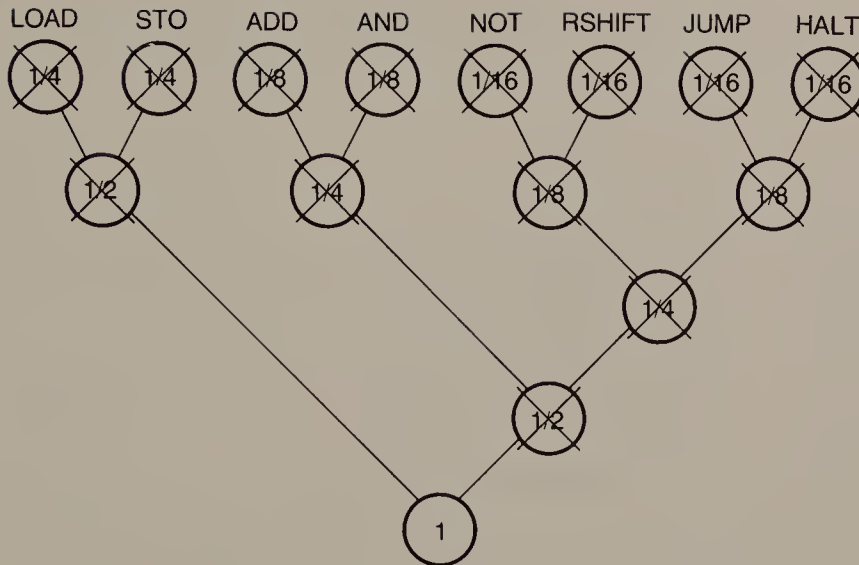


Figure 2.11 The Tree Obtained by Continuously Scanning the Graph of Figure 2.10

Now the right and left branches of this tree are labeled with 0 and 1, respectively, to obtain the Huffman tree shown in Figure 2.12.

To find the op-code for a mnemonic, a path from the root to the leaf node corresponding to this mnemonic must be found; the 0s and 1s are picked up from the path. For example, the op-code corresponding to the mnemonic LOAD in Figure 2.12 is 11. Starting from the root, first move to the left; and move again to the left to reach the node corresponding to LOAD. The values on the left branches are 1. The op-codes for other mnemonics can be found in a similar manner; they are tabulated as follows:

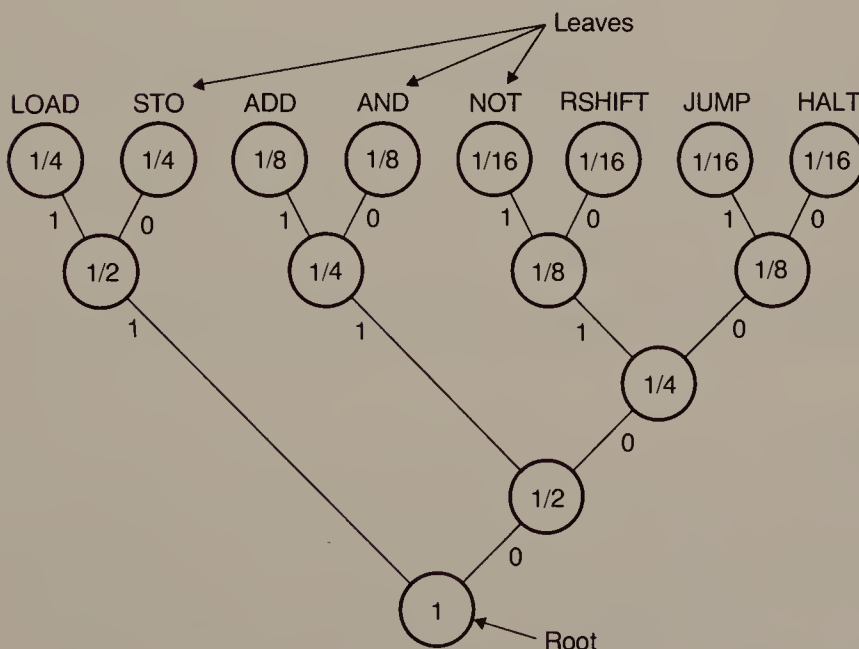


Figure 2.12 Huffman Tree

MNEMONIC	OP-CODE	PATH FROM THE ROOT
LOAD	11	Left→left
STO	10	Left→right
ADD	011	Right→left→left
AND	010	Right→left→right
NOT	0011	Right→right→left→left
RSHIFT	0010	Right→right→left→right
JUMP	0001	Right→right→right→left
HALT	0000	Right→right→right→right

From the preceding result, it is easy to see that Huffman's procedure encodes the most-frequently used instructions with short op-codes and the least-frequently used with long op-codes.

The average number of bits needed per instruction can be calculated using the formula

$$\sum_{i=1}^n l_i f_i \quad 2.1$$

Where l_i and f_i are op-code length and the relative frequency count of the i th instruction, respectively. For this example, the average number of bits is:

$$\begin{aligned}
 & 2(1/4) + 2(1/4) + 3(1/8) + 3(1/8) + 4(1/16) + 4(1/16) + 4(1/16) + 4(1/16) \\
 &= 1/4(2 + 2) + 1/8(3 + 3) + 1/16(4 + 4 + 4 + 4) \\
 &= 1 + 3/4 + 1 = 2.75 \text{ bits}
 \end{aligned}$$

Using the block-code scheme, each instruction will be encoded with a 3-bit op-code ($2^3 = 8$), giving an average value of:

$$\begin{aligned}
 & 3(1/4 + 1/4) + 3(1/8 + 1/8) + 3(1/16 + 1/16 + 1/16 + 1/16) \\
 &= 1.5 + 0.75 + 0.75 = 3.0 \text{ bits}
 \end{aligned}$$

From the information theory, the optimum number of bits needed to encode a set of messages is

$$- \sum_{i=1}^n f_i \log_2(f_i) \quad 2.2$$

The difference between the actual average length and the optimum length is called *redundancy* (R), and it can be written as

$$R = \frac{\text{actual length} - \text{optimum length}}{\text{actual length}} \quad 2.3$$

If equation (2.2) is applied to the example, the following results:

$$\begin{aligned}
 & -[2^{(1/4)}\log_2(1/4) + 2^{(1/8)}\log_2(1/8) + 4^{(1/16)}\log_2(1/16)] \\
 & = -[-1 - 0.75 - 1] \\
 & = 2.75 \text{ bits}
 \end{aligned}$$

If equation (2.3) is applied to the result of Huffman's scheme, R is found to be zero. However, the block-code scheme introduces a redundancy of $1/12$, or 8.33%.

Therefore, Huffman's scheme achieves an optimal result by keeping the redundancy to a minimum value. However, when op-codes are encoded using Huffman's scheme, the decoding process takes more time because a search must be conducted on the Huffman tree. This idea is used in Burrough's B1700 computer. The op-codes of the Zilog's Z 80 microprocessor's instruction set are encoded using a scheme that is very close to the Huffman's scheme. Even though block-code encoding takes extra storage space, it is widely used because of the simplicity of the decoding procedure.

2.3 ADDRESSING MODES

A processor executes a sequence of instructions in the following manner:

```

begin
    read next instruction from the memory
    decode the instruction and recognize the type
      of operation
    while the required operation is not a halt op-
      eration do

begin
    determine operand addresses
    retrieve the operands
    perform the desired operation
    determine the destination address
    save the result of the operation in the desti-
      nation
    read the next instruction from memory
    decode the instruction and recognize the type
      of operation
end
    while there is a hardware reset do

    skip    (/ this is a dummy statement that allows /)
           (/ the processor to execute an infinite loop/)
end

```

The sequence of operations that a processor has to carry out while executing an instruction is called its *instruction cycle*. The most important activity in an instruction cycle is the determination of the addresses of the operands involved in that instruction.

The manner in which a processor accomplishes this task is called the *addressing mode*. The typical addressing modes supported by the instruction sets of popular contemporary processors such as the 8085, Z 80, MC6809, MC68000, PDP-11, and VAX-11 will be examined.

An instruction is said to have an *inherent* addressing mode if its op-code indicates the address of the operand, which is usually the contents of a CPU register. For example, consider the following instruction:

STC. Set the carry flag in the status register. Since the op-code implies the address of the operand, the processor does not have to compute the operand address. This mode is very common with 8-bit microprocessors such as the 8085, Z 80, and MC6809.

Whenever an instruction contains the operand value, it is called an *immediate-mode* instruction. For example, consider the following instruction:

Add #25,R1; $R1 \leftarrow R1 + 25$

In this instruction, the symbol # indicates that it is an immediate-mode instruction. This convention is adopted in the assemblers for processors such as the MC6809, MC68000, PDP-11, and VAX-11. In these systems, the machine representation of this instruction occupies two consecutive memory words:

The first word holds the op-code, whereas the next word holds the data value. (For the preceding case, it is 25).

To execute this instruction, the processor has to access memory twice.

An instruction is said to have an *absolute addressing mode* if it contains the address of the operand. For example, consider the following move instruction:

MOV @#5000, R2; $R2 \leftarrow (5000)$

This instruction copies the contents of memory location 5000 in the CPU-register R2. As in the previous case, the object code corresponding to this instruction occupies two consecutive memory words. However, in this case, the contents of the memory word that follows the op-code is interpreted as the address of the operand. To execute this instruction, the processor has to make three memory accesses: one for the op-code, one for the address, and one for the data. The execution time of an absolute-mode instruction is more than the corresponding immediate-mode instruction.

An instruction is said to have a *register mode* if it contains a register address as opposed to a memory address. In this mode, the operand values are held in the CPU registers. For example, consider the following register mode and instruction:

ADD R2, R3; $R3 \leftarrow R2 + R3$

In the register-addressing mode, the effective address (EA) of an operand is a CPU register. Since many contemporary CPUs have a small number of registers, the machine representation of a register mode instruction requires only a few bits. Memory space can be conserved by using register-mode instructions. In this mode, the processor does not require any memory reference for data retrieval. Hence, the instruction exe-

cution rate can be considerably increased. Since there is always a limit on the number of CPU registers, it is not possible to handle a large number of operands by the exclusive use of the register-addressing mode. However, having a CPU with large number of registers is a key characteristic of the reduced-instruction-set computers (RISC). The RISC architecture is discussed in a later section of this chapter.

Whenever an instruction specifies the address of a CPU register that holds the address of an operand, the resulting addressing mode is known as the *register indirect* mode. From this definition, it follows that the EA of an operand in the register-indirect mode is the contents of the CPU register R. More formally, this result is written as follows:

$$EA = (R)$$

To illustrate this idea clearly, consider the following instruction:

MOV (R2), (R3); (R3) $\leftarrow$ (R2)

Assume that the following configuration exists:

(R2)	=	5000 ₁₆
(R3)	=	4000 ₁₆
(5000)	=	1256 ₁₆
(4000)	=	4629 ₁₆

This instruction copies the contents of the memory location, whose address is specified by the CPU register R2, into the location whose address is specified by the CPU-register R3. Thus, after the execution of this instruction, the memory location 4000 will contain the value 1256. Whenever a CPU register is used as a data pointer, the assembler convention is to enclose that register using a set of parentheses. Alternatively, an indirect register may be specified by using the prefix @:

MOV, @R2, @R3

This addressing mode is very useful whenever there is a need to manipulate two different memory areas.

Two variations of the register indirect mode are the *auto-increment* and *auto-decrement* modes. In auto-increment mode, first the contents of the specified CPU register are used as the address of the operand, and then data transfer takes place. After this, the register contents are automatically incremented by some constant k . To indicate this mode, the register involved will be enclosed by parentheses and immediately followed by the plus sign. For example, consider the following instruction:

MOV (R2) +, R3

In this instruction, the source operand is in the auto-increment mode, and the action taken by this instruction can be described as follows:

$$R3 \leftarrow (R2)$$

$$R2 \leftarrow R2 + k$$

The auto-decrement mode is similar to the auto-increment mode, except the contents of the specified CPU register are first decremented by k , and the resulting value is used as the address of the operand. This action is symbolically represented by surrounding the CPU register with parentheses together with a minus sign just before the left parentheses. For example, consider the auto-decrement mode clear instruction:

CLR – (R5)

This instruction can be precisely described as follows:

$R5 \leftarrow R5 - k$

$(R5) \leftarrow 0$

The constant value k used in the auto-increment and auto-decrement modes is actually a function of the number of bits involved in the data transfer. Typically, this value is 1 for 8-bit, 2 for 16-bit, and 4 for 32-bit operands.

These modes are useful in array manipulations. For example, assume the CPU registers R2 and R3 are initialized with the starting addresses of two arrays X and Y, respectively. Then, the following instruction transfers the first element of the array X into the first element of the array Y:

MOV (R2) +, (R3) +

After this data transfer, the CPU registers are automatically pointed to the next corresponding elements of these arrays. If the system does not support an auto-increment mode, then the same result can be obtained by using a sequence of three instructions:

MOV (R2), (R3)

ADD #k, R2

ADD #k, R3

The auto-increment mode allows one to write programs that are more efficient both in terms of space and time.

By using the stack pointer in auto-decrement and auto-increment modes, PUSH and POP operations can be obtained. For example, consider the following instructions:

MOV R2, – (SP)

MOV R3, – (SP)

MOV R4, – (SP)

This sequence pushes the contents of the CPU registers R1, R2, and R3 into the stack. To restore the register contents, three successive POP operations are performed, as follows:

MOV (SP) +, R4

MOV (SP) +, R3

MOV (SP) +, R2

Another important concept used in the context of addressing modes is address modification. In this approach the EA of an operand is expressed as the sum of two parameters: reference address (RA) and modifier (M), formally written as:

$$EA = RA + M$$

The modifier M is also called the *offset*, or displacement. Such an address-modification principle is the basic concept associated with the following addressing modes:

- Indexed mode
- Base-register mode
- Relative mode

In the indexed mode, the value of RA is included in the instruction, and a CPU register contains the value M. The CPU register X is called the *index register*. This mode is useful for accessing arrays. For example, consider the following Pascal integer array y:

```
var
    y: array [0..9] of integer;
```

Assume that each element of this array requires 1 byte and that the entire array is configured in the memory as shown in Figure 2.13. From this figure, notice the array starts at the memory address 0100. Now, assume the index register X contains the value 0002, and execute the following indexed-mode load instruction:

```
LDA 0100 (X)
```

This instruction indicates that the register X is the index register. Its machine representation includes the reference address, 0100, which is the starting address of array Y (see Figure 2.13). For this situation, the EA of the operand is:

$$\begin{aligned} EA &= RA + X \\ &= 0100 + 2 \\ &= 0102 \end{aligned}$$

Therefore, when this instruction is executed, the contents of memory location 0102 are transferred to the A register. This memory address actually holds the array element y [2]. Since the register X contains the required index value of the array element, it is referred to as the index register. To access the third element of array y, register X needs to be incremented by 1. This operation can be performed quickly. Therefore, the indexed mode allows a programmer to carry out array manipulations in an efficient manner.

In the base-register addressing mode, the parameter RA is held in a separate register called the base register, and the modifier M is included in the instruction. This mode is very significant in a system that provides a virtual memory support. In particular, the base-register mode has application in segmented memory systems. In these systems, the base register holds the base address (or the starting address) of a segment.

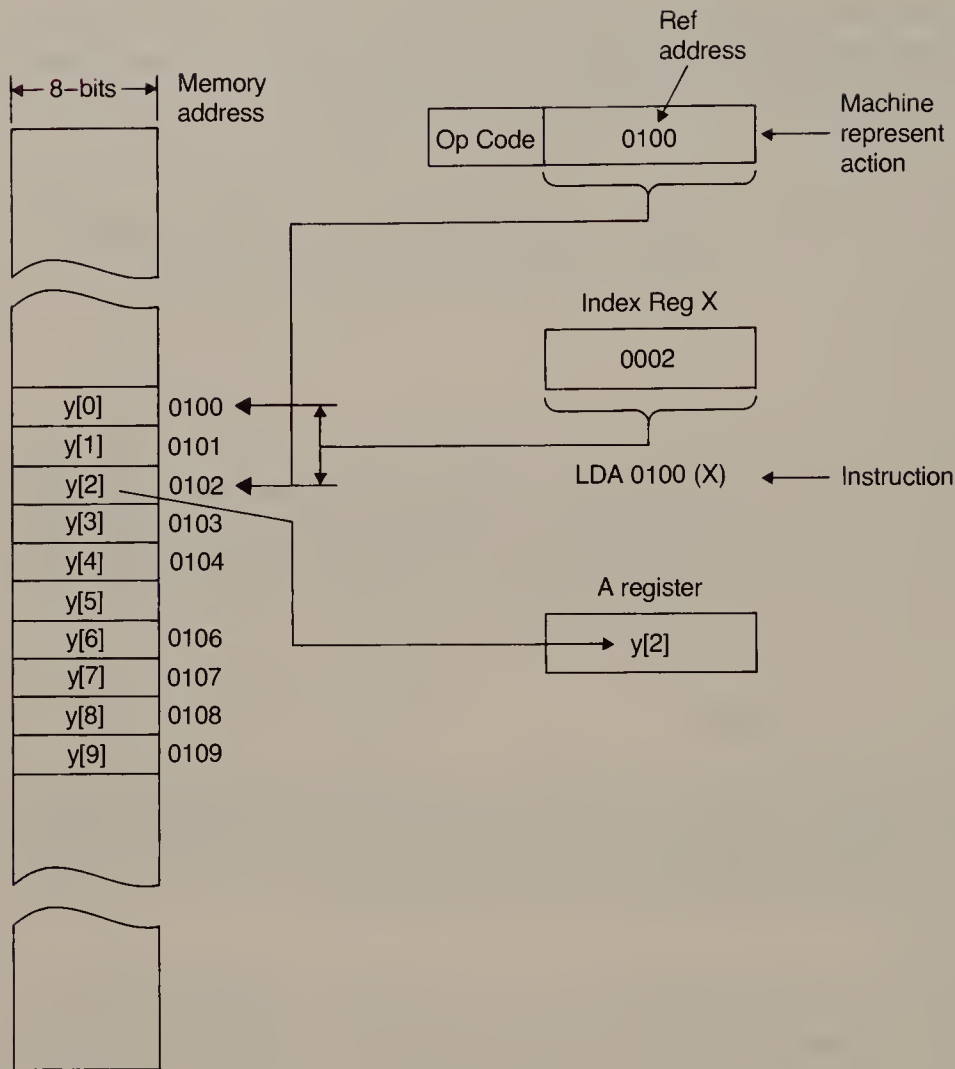
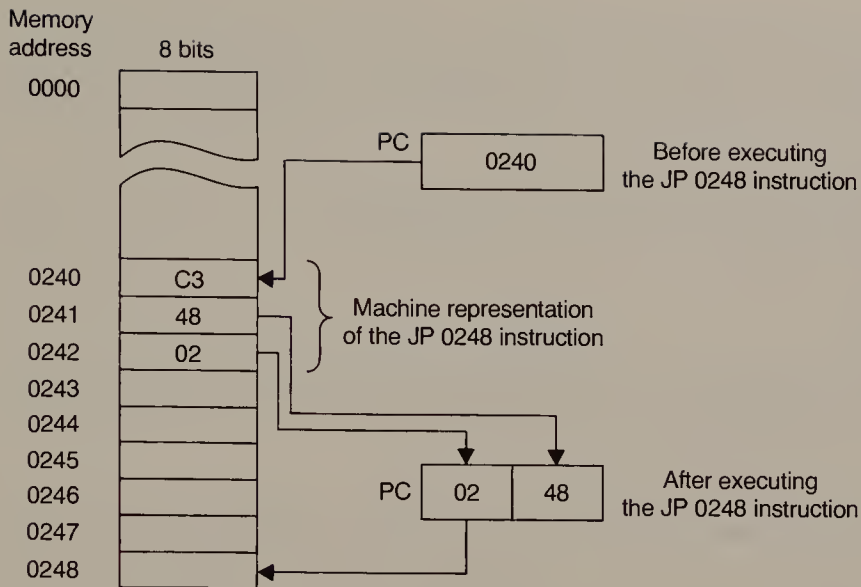


Figure 2.13 Use of the Indexed Addressing Mode in Accessing Arrays

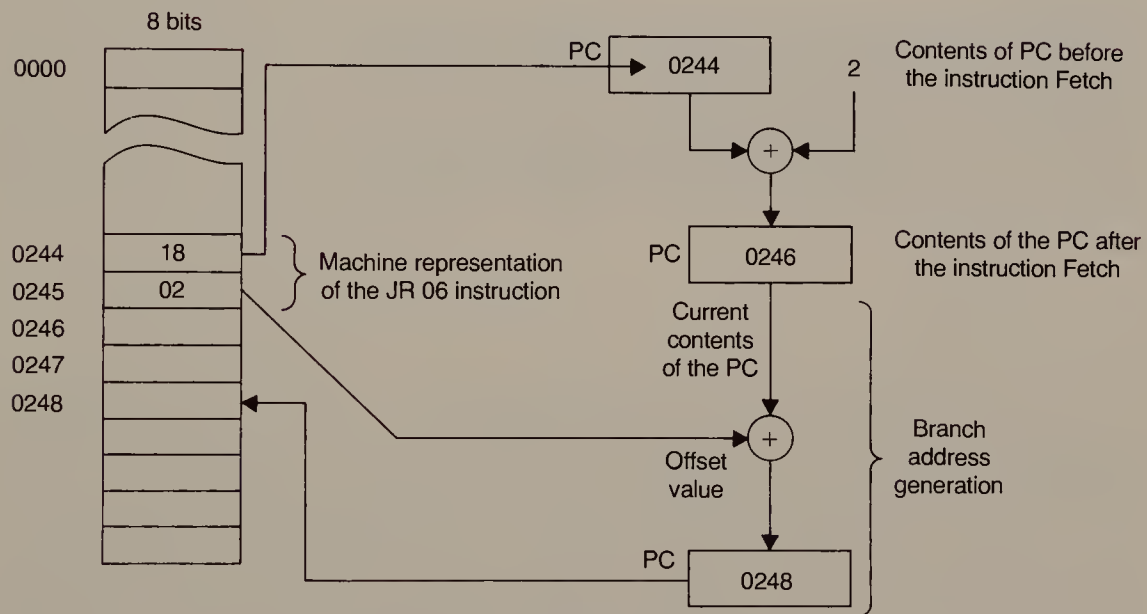
In general, the size of the indexed mode modifier (M) is the same as the number of bits required to specify a memory address. In the base-register mode, the number of bits in the modifier field (M) may be less than the number of bits required for a direct memory reference.

The contents of the M field are often interpreted as a 2's complement number. Whenever the sizes of the modifier and the memory address fields are unequal, the sign-extended form of M is used in the effective address calculation.

If the PC is configured as the base register, the *relative addressing mode* results. This mode is particularly useful in designing short branch instructions. For example, consider the Z 80 branch instruction JP 0248. This instruction is equivalent to the high-level language statement GOTO 0248. The machine representation of this instruction requires 3 bytes: 1 byte for the op-code (C3) and 2 bytes for the branch address (0248). Assume this instruction is stored as shown in Figure 2.14(a). Since PC always holds the address of the next instruction to be executed, implementation of a branch instruction implies loading the PC with the branch address.



a. Operation of an Absolute Jump Instruction



b. Operation of a Relative Branch Instruction

Figure 2.14 Mechanics of an absolute and a relative branch instructions (All Numbers Are in Hex Decimal Form)

Alternatively, consider the Z 80 relative branch instruction JR 02. The numerical value 02 represents the value of the modifier or offset. The machine representation of this instruction required only 2 bytes: 1 byte for the op-code (18) and 1 more to hold the offset value 02. If we assume that this instruction is stored as shown in Figure 2.14(b), the execution of this instruction can be explained as follows:

- First, the entire instruction is fetched.
- Since this is a 2-byte instruction, after instruction Fetch, the PC will contain 0246 (0244 + 2).
- The branch address is computed as follows:

$$\begin{aligned}
 \text{branch} &= \text{current contents of the PC} + \text{offset value PC} \\
 &= 0246 + 02 \\
 &= 0248
 \end{aligned}$$

- Finally, the PC is loaded with this branch address, and the next instruction to be executed is located in the memory location 0248.

When the offset is a negative number, reverse branching takes place. For example, consider JR 06 with all other data (in hex) as in Figure 2.14(b). In this case, 06 will be subtracted from 0246. Since this is a subtraction of an 8-bit signed number from a 16-bit signed number, the correct result is obtained if the 8-bit number (−06) is sign-extended and then subtracted from 0242 using 2's complement (ignoring the final carry) as follows:

$$\begin{array}{rcl}
 0246 & = & 0000 \quad 0010 \quad 0100 \quad 0110 \\
 \text{2's complement of 6} & = & \boxed{1111 \quad 1111} \quad 1111 \quad 1010 \\
 & & \hline
 & 1 \quad \underbrace{0000} \quad \underbrace{0010} \quad \underbrace{0100} \quad \underbrace{0000} \\
 & \nearrow & 0 \quad 2 \quad 4 \quad 0 \\
 & \text{Ignore carry} & \\
 & \text{Sign extension} &
 \end{array}$$

Since the offset value is an 8-bit quantity, only −128 to 127 bytes can be branched relative to the current contents of the PC. In most computers, conditional branching instructions use the relative mode. If one needs to go beyond the −128 to +127 range with conditional branching, then the unconditional branching instruction can be used anywhere in this range to branch to any location within the computer's directly addressable memory. Another important aspect of the relative addressing mode is its ability to produce a relocatable program. For example, consider the situation in Figure 2.14(b). In this configuration, assume that the branch address is 2 bytes away from the current contents of the PC. This means when this program is relocated, the JR 02 instruction will be placed in a different location, and different PC contents will result. Nevertheless, the program execution will produce the same result because the system identifies the branch address as the one located 2 bytes away from the current contents of the PC.

The usefulness of various addressing modes will now be discussed. The implementation of a particular addressing mode is largely dependent on the processor organization. In a processor with many general registers, indexed and indirect addressing modes are easy to obtain. If a computer supports powerful addressing modes, the task

of designing language translators, operating systems, and efficient application programs can be greatly simplified. For example, the auto-increment and auto-decrement modes allow one to implement stack operations and program loops efficiently. For a compiler designer, efficient stack operations are very important for implementing procedure calls.

The indexed addressing mode allows an applications programmer to manipulate arrays, whereas absolute and relative modes allow the programmers to write position-independent programs. A program is said to be *position-independent* if it can be placed anywhere in memory. This is a desirable feature for operating system designers because it allows the operating system to relocate programs in a dynamic manner. In a multiuser system, many different users may need the same library program provided by the operating system. If this library routine is position-independent, the operating system can load the machine code of this routine into any available portion of the main memory.

2.4 INSTRUCTION TYPES

The purpose of a computer instruction set is to provide a virtual machine with all features so that compilers, operating systems, and library subroutines can be designed quickly (see Figure 2.15).

From Figure 2.15, it can be seen that a program using the instruction set is translated into the machine code by the translator program *assembler*. Finally, the machine code is interpreted by the microcode so that the hardware produces the desired result. In other words, the microcode directs the hardware by generating necessary commands (control signals). In general, instructions available in a processor may be broadly classified into six groups. These are:

Data transfer instructions

Arithmetic instructions

Logical instructions

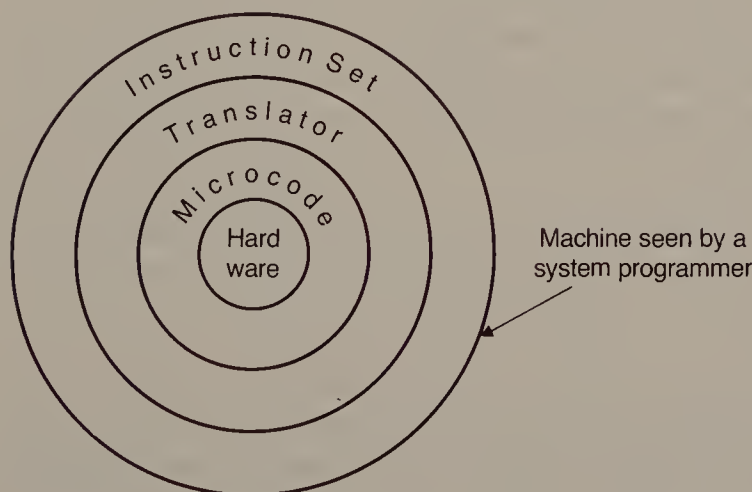


Figure 2.15 Creation of a Virtual Machine via Instruction Set

Program-control instructions

System-control instructions

I/O instructions

These instruction types are discussed in the following sections.

2.4.1 Data-transfer Instructions

Data-transfer instructions are concerned primarily with data transfers between the processor and main memory. Typically, an ideal instruction set must be able to handle the following transfers:

- Register to register
- Register to memory
- Memory to register
- Memory to memory

In contemporary computers, the system architecture is configured so the preceding possibilities can be implemented in an efficient manner. For example, in a three-bus architecture, simultaneous data transfers can occur. Various bus architectures are included in Chapter 4. This group is given primary consideration while designing the control unit because results reported by D. E. Knuth [5] and S. H. Fuller [2] have shown that, on the average, 40% of the instructions in many user programs are data-transfer instructions.

2.4.2 Arithmetic Instructions

All instruction sets typically include ADD and SUBTRACT instructions. The instruction sets of the VLSI microprocessors such as the Intel 8086 and Motorola's MC68000/68020 include multiplication and division instructions. The CPUs of these processors must include provisions for performing multiplication and division. The design of the hardware required for these operations is covered in Chapter 3. Many data-processing applications require the system to manipulate decimal data quickly. For this purpose, the instruction set of modern processors include instructions that can exclusively process BCD numbers. For example, the ABCD instruction of the MC68000 processor is capable of adding two BCD numbers stored in the main memory. Similarly, floating-point arithmetic operations are vital with respect to scientific programming. To facilitate this manipulation, some processors include floating-point instructions. For example, the extended instruction set of the PDP-11/70 computer includes instructions such as MULF and DIVF to multiply and divide floating-point numbers, respectively. To speed up the floating-point arithmetic operations, the PDP-11/70 CPU includes 6 additional 32-bit general-purpose floating-point registers (FO through F5) and the floating-point coprocessor, FP-11. With the advent of VLSI technology, it is possible to obtain numerous coprocessors compatible with microprocessors. For example, the I8087, AM9511, and

MC68881 coprocessor chips are compatible with I8086, I8085, and MC68000 microprocessors, respectively. Likewise, Motorola's floating-point ROM chip, the MC6839, can be interfaced with the MC6809 microprocessor to handle floating-point computations.

In general, the following assignment statements are common with respect to high-level language:

X: = X + 1;

Y: = Y - 2;

Z: = 2;

As a consequence, the instruction repertoire of the state-of-the-art processors includes special instructions to increment or decrement by small quantities and to assign a small value to a specified memory address. According to the Motorola designers, these instructions are called *quick mode* instructions. With VAX-11 computers, these instructions are referred to as *short immediate* or *literal* mode instructions. For example, the assignment statement Z: = Z - 4 can be implemented in the MC68000 and VAX-11 assembly languages as follows:

MC68000

LEA Z, A2	;	Load the address of Z into the address register A2
SUBQ.B #4, (A2)	;	Subtract 4 from the memory byte whose address is; specified in the register A2.

↑

This suffix indicates
that this is a quick
mode instruction.

VAX-11

MOVEAB Z, R6	;	Move the address of Z into the CPU
	;	register R6
SUBB S^#4 (R6)	;	Subtract 4 from the memory
	;	byte whose address is specified in the register R6.

↙ ↘

This prefix indicates
that the operand
is in the literal
mode.

2.4.3 Logical Instructions

Invariably, the instruction sets of all contemporary processors include instructions to perform Boolean AND, OR, NOT, and EXCLUSIVE-OR operations on a bit-by-bit basis. These instruction sets also include the following shift instructions:

- Arithmetic shift (left or right)
- Logical shift (left or right)
- Rotate shift (left or right) through or without the carry flag

In fact, the MC68000 and the VAX-11 instruction sets include even more elegant shift instructions. For example, consider the following shift instructions:

MC68000

LSR.W #3,D5 This instruction performs a left logical shift of the low-order 16-bits of the data register D5 by three places.

VAX-11

ROTL #16,R7

These instructions rotate the contents of the CPU register R7 to the left by 16 places. After this instruction is executed, the low-order 16-bits and the higher-order 16-bits of the 32-bits register R7 are exchanged. The VAX-11 instruction set also allow a programmer to perform memory-to-memory compare operations. The test instruction tests the specified operand with zero and sets the zero and sign flags, depending on the status of the tested operand. These flags are held in a dedicated register called condition code register. Its design is covered in the next chapter.

2.4.4 Program-control Instructions

In a conventional computer, instructions are always executed in the same order they are presented. In reality, the flow of control depends on the result of a computation. In this situation, a program can select a particular sequence of instructions to execute based on the results of a computation. Instructions that perform this are called *program-control instructions*. In general, these instructions may be classified into four groups:

- Unconditional branch instructions
- Conditional branch instructions
- Subroutine call instructions
- Interrupt-handling instructions

An unconditional branch instruction transfers the control to the specified address regardless of the status of the computation. Processors such as the Z 80, MC68000, PDP-11, and VAX-11 include both the absolute and relative branch instructions.

A conditional branch instruction works as follows:

If ⟨condition⟩ *then* branch to execute a new instruction *else* execute the following instruction.

In this situation, assume the condition flags are already set by some instruction that immediately precedes the conditional branch instruction. Typically, the instruction

may be an arithmetic instruction (such as ADD, SUBTRACT, INCREMENT, DECREMENT, or COMPARE) or a logical instruction (such as TEST or COMPARE). Using the condition flag settings, traditional relational operators (such as equal to, not equal to, greater than, greater than or equal to, less than, or less than or equal to) can be implemented. For example, consider the following VAX-11 instruction sequence:

```

MOVAB    X,R7;          Move the address of X register into R7
MOVAB    Y,R8;          Move the address of Y into register R8
TSTB, (R7)  ;           Check whether X = 0
BEQL UPDTY  ;           If it is then update Y
BRB NEXT    ;           otherwise go to next instruction
UPDTY: ADDB S^ #2, (R8): perform Y: = Y + 2
NEXT: .....

```

This sequence is equivalent to the Pascal *if* statement:

```
If X = 0 then Y: = Y + 2
```

In this example, the variables X and Y are assumed to be 8-bit 2's complement numbers. The test instruction sets the Z-flag (zero-status flag) only if X is zero, and the branch instruction BEQL causes a branch only if the Z-flag is set to 1. PDP-11, MC68000, and VAX-11 conditional branch instructions can handle both signed and unsigned operand values.

The preceding numerical example suggests that by utilizing the logical and conditional branching instructions, the following control structures, which are regarded as the primitive components of a structured programming language such as Pascal or C; can be implemented:

- If <cond> then <statement>
 else <statement>
- While <cond> do
 <statement>
- Repeat
 <statement>
 Until <cond>
- Case <label> of
 id1 : <statement>
 id2 : <statement>
end

The VAX-11 instruction set includes useful instructions such as SOBGTR (subtract 1 from the loop index and branch if the index is still greater than zero) and AOBLEQ (add 1 to the loop index and branch if index is less than or equal to limit) to implement Pascal *repeat* and *for* loops, respectively.

For example, the VAX-11 code:

```

MOV B    #10, R5    ; Initialize R5 with 10
CVTBL    R5, R5     ; Sign extend R5 to 32 bits
CLRB     R6         ; Clear R6 to 0
MOVAB    SUM, R7    ; Point R7 to SUM

LOOP:    ADDB2      R5, R6
         SOBGTR     R5, LOOP ; Repeat until (R5) = 0
         MOV B      R6, (R7) ; Transfer the result to SUM

```

is equivalent to the Pascal program:

```

i := 10;
SUM := 0;
repeat
    SUM := SUM + i;
    i := i - 1;
until i = 0

```

Similarly, the VAX-11 program fragment

```

MOV B    #1, R5    ; Initialize R5 with the initial
CVTBL    R5, R5     ; Value of the loop index
MOV B    #10, R6    ; Initialize R6 with
CVTBL    R6, R6     ; Limiting value of the loop index
MOVAB    SUM, R7    ; Point R7 to SUM
CLRB     R8         ; Clear R8

LOOP:    ADDB2      R5, R8 ; Increment index by 1
         ADBLEQ     R6, R5, LOOP; and repeat until Loop index >
                                   limit
         MOV B      R8, (R7) ; save result

```

is equivalent to this C program segment:

```

{
    sum = 0;
    for (i = 1;    i  <= 10;    i + +)
        sum + = i;
}

```

We leave the task of verifying the correctness of the preceding results as an exercise. With respect to VAX-11 programming, all loop parameters must be specified as 32-bit quantities. This is why we use the CVTBL instruction to sign-extend a byte into 32-bit operand (long-word operand).

A subroutine is a program segment for carrying out repeatedly needed tasks such as converting code from binary to ASCII, searching, and sorting. A subroutine may be written and tested separately. A subroutine can be linked with a user program so that the latter can call the former as many times as needed. Thus the use of subroutines can save programmer's time as well as the memory space needed by an application program.

A large program can be thought of as a collection of independent program modules, where each module may be a subroutine or a set of subroutines. This is the key feature of the modern software approach called *modular programming*.

In this method, a programmer has a global view of all components of a large program, and efficient programs can be developed within a short period of time. Since each subroutine can be independently tested, it follows that the modular programming approach considerably improves the overall software reliability. The subroutine concept strongly encourages the idea of program sharing. For example, in a multiuser system, several user programs may share the same I/O subroutine provided by the operating system. Therefore, a user does not have to spend time developing I/O routines.

Subroutine calls and returns from subroutines are usually handled by two special instructions, CALL and RET, respectively. The CALL instruction is of the form CALL <addr>, where the parameter <addr> refers to the address of the first instruction of the subroutine. When this instruction is executed, the current contents of the PC are saved in the stack, and the PC is loaded with <addr>. The current contents of the PC provide the address of the instruction that immediately follows the CALL instruction. This address is also called the *return address* because this is the point where execution of the calling program will take place after exiting from the subroutine. The CALL instruction is functionally equivalent to the following instruction sequence:

```

PUSH PC      ; save the return address in
              ; the stack (SP) + ← PC
JP addr      ; branch to the subroutine

```

The RET instruction is usually the last instruction of the subroutine. When this instruction is executed, the return address previously saved in the stack is retrieved and loaded into the PC. The control is then transferred to the calling program. A RET instruction is functionally equivalent to POP PC ; PC ← — (SP)

At this point, one may be wondering why the return address is not saved in a

CPU register rather than the stack. This arrangement fails to work if nested subroutine calls are to be implemented. Subroutine nesting refers to one subroutine calling another.

For example, consider the main program M and two subroutines P and Q shown in Figure 2.16(a): The main program calls subroutine P, and this subroutine in turn calls subroutine Q. The expected control flow sequence is shown in Figure 2.16(b). Parameters MR and PR refer to the return address of the main program (M) and the subroutine (P) respectively.

When the main program calls subroutine P, the return address (MR) is pushed into the stack (see Figure 2.16(c)) and the control is transferred to subroutine P. Similarly, when subroutine P calls subroutine Q, the return address (PR) is pushed into the stack (see Figure 2.16(d)), and the control is transferred to subroutine Q. When subroutine Q completes its execution, the return address is retrieved from the stack and loaded into the PC. Since the return address is PR, the execution of subroutine P is resumed. Similarly, when subroutine P terminates, the return address (MR) (see Figure 2.16(e)) is retrieved from the stack and loaded into the PC. The execution of the main program is then resumed.

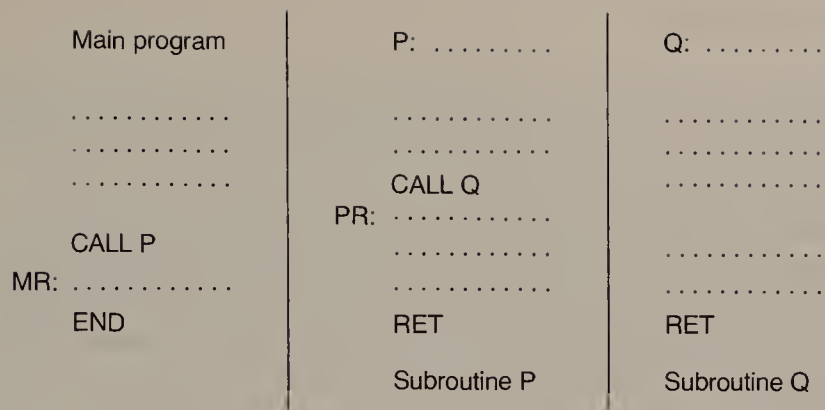
To implement subroutine nesting, the return addresses must be retrieved exactly in the reverse order in which they are saved. Since a stack is a LIFO data structure, its use is a natural solution to this problem. Suppose a CPU register is used to save the return address. The return address (PR) will write over the return address (MR), and control will not be transferred back to the main program at all.

Solutions to some problems such as traversing a binary tree are naturally recursive and are precisely solved by writing recursive subroutines. A subroutine is said to be recursive if it calls itself. A recursive evaluation involves a descent process all the way to the basis part of the recursive definition. Then it involves an ascent process all the way up in exactly the reverse order. Therefore, the use of a stack offers a solution for implementing recursive subroutines.

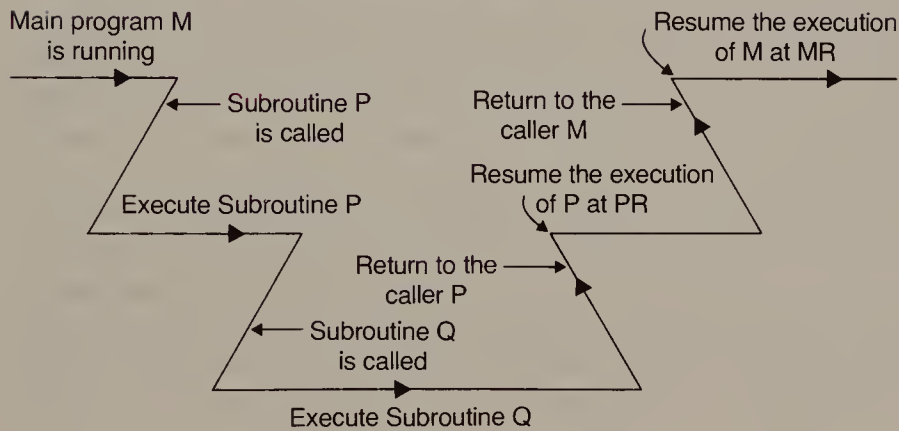
The registers involved in implementing a subroutine call are called *linkage registers*, and the process is known as the *subroutine linkage convention*. In all microprocessors, the PC is used as the linkage register. In the PDP-11 computer, any CPU register can be configured as a linkage register under program control. In the majority of the processors, the system hardware automatically uses the SP in response to CALL and RET instructions. An exception to this is Motorola's 8-bit microprocessor, the MC6809. The CPU of this processor entails two dedicated 16-bit registers: U (user stack pointer) and S (system stack pointer). In this microprocessor, the system hardware automatically uses the stack pointer S for handling subroutine calls and returns. A user can use the stack pointer U to configure a temporary data storage area in a different portion of the stack, preventing the user stack area from causing any interference to the subroutine linkage.

The effective use of subroutines calls for a method for transferring data from the caller to the subroutine, and vice versa. This aspect is often referred to as *parameter- or argument-passing* conventions. High-level languages such as Pascal and Ada adopt typical parameter-passing conventions such as call-by-value and call-by-reference.

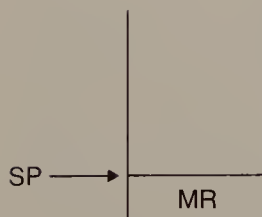
In the *call-by-value* approach, the main program transfers a parameter to a subroutine by copying the parameter value in the local variable that is created when the subroutine becomes active. When the subroutine changes the value of this local variable,



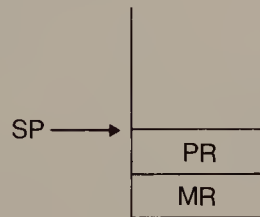
a. A Typical Two-Level Subroutine Nesting



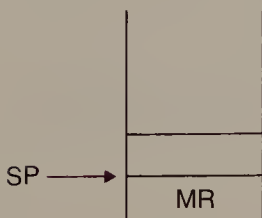
b. Expected Control Flow



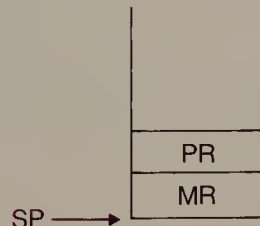
c. Contents of the stack after Executing CALL P



d. Contents of the stack after Executing CALL Q



e. Contents of the stack Executing the RET Instruction in the Subroutine Q



f. Contents of the stack after Executing the RET Instruction in the Subroutine P

Figure 2.16 Implementation of a Two-level Subroutine Nesting

the actual parameter of the main program does not change. This is a desirable property for function subprograms because it keeps a function from altering the value of the argument. Hence, the function returns a unique value to the caller.

In the *call-by-reference* approach, the main program transfers the address of a parameter to a subroutine. A subroutine can alter the value of the parameter that belongs to a caller. This approach allows a subroutine to transmit the results to the caller. The ability of a subroutine to alter the variables of the calling program is known to have a side effect. There is no side effect with the call-by-value approach. However, the call-by-reference approach uses the side effect for passing the results to the caller.

To implement these features, contemporary processors include either special instructions or additional hardware. For example, the LINK and UNLK (unlink) instructions of the MC68000 processors allow a compiler designer to implement Pascal functions and procedures with minimum effort. The VAX-11 CPU includes two dedicated registers: AP (argument pointer) and FP (frame pointer). These hardware elements greatly simplify the task of argument passing. A thorough discussion of this topic is beyond the scope of this book.

An interrupt may be defined as a hardware-initiated subroutine call. For example, in a microprocessor-based system, an I/O device such as a keyboard may generate an interrupt to inform the processor that valid data is available. When this interrupt is recognized, the processor suspends the execution of the currently running program, saves the contents of the PC in the stack, and transfers the control to a service routine dedicated to serve the keyboard. The service routine in this case reads the keyboard data, saves it in the main memory for further processing, and returns the control to the suspended program.

If the service routine needs the CPU registers, their previous contents must be saved in the stack before the service routine actually uses them. This is a time-consuming operation, and to speed it up, different microprocessors use different approaches. For example, in the Z 80 microprocessor, all CPU registers are duplicated. For each CPU register X, there is an alternate companion register X'. At any given time, either the original or alternate set of CPU registers can remain active. Normally, a user program is run using the original set of CPU registers. When an interrupt occurs, the system automatically switches to the alternate register set so the service routine can immediately use all the registers in the alternate set. However, after the interrupt is serviced, the system automatically switches back to the original set of registers so the execution of the suspended program can continue. This concept provides a fast response to external interrupts.

Modern processors such as the MC68000 and VAX-11 provide a number of software internal interrupts called TRAP instructions. These are covered in detail in Chapter 6.

2.4.5 System-control Instructions

With the advent of low-cost VLSI microprocessors, designing systems with several processors is feasible. Such a multiprocessor system has significant advantages, such as high processing speed and more reliability. In a typical multiprocessor system, several processors will be allowed to share a single memory unit. In this situation, at any given

time only one processor should be allowed to access the shared memory. This problem is known as the *mutual exclusion problem*.

Some processors such as the Motorola MC68000 have a special instruction called test and set (TAS) to provide a hardware-based solution to the above problem. The TAS instruction is used to prevent access to a shared resource by other programs when one program has control of the resource. This is sometimes called *lockout*. The TAS instruction is used to test and modify a byte length held either in a data register or in memory. For example, consider TAS (memory). If (memory) = 0, then set the zero flag Z = 1, else set Z = 0, N = 1, and then set bit 7 of the memory location to 1. Now, consider TAS (A1). Suppose (A1) = 00₁₆; then after execution of TAS, Z = 1 and (A1) is changed to 80₁₆. If the initial zero value indicated the memory area was free for use, a subsequent test of the memory area with the value 80₁₆ would indicate the memory was in use. To prevent accessing a shared memory in a multiprocessor system simultaneously by more than one processor, the TAS instruction has an indivisible read-modify-write cycle. Once the operand is addressed by the 68000 executing the TAS instruction, the bus is not available to another processor until the TAS instruction is completed.

As an illustration of synchronization by the TAS instruction, consider two 68000 processors that are interfaced through a shared RAM. It is desired to transfer (D₀) from Processor 1 to (D₂) by Processor 2 using the RAM byte TRDATA, as shown in Figure 2.17. To accomplish this transfer and provide proper synchronization, Processor 1 can execute the following routine for writing (D₀) into the RAM location TRDATA:

```

LOOP1    TAS      TEST
          BNE      LOOP1
          MOVE     D0, (TRDATA)
          CLR.B    TEST
          -
          -
          -

```

Processor 2 can then transfer (TRDATA) to D₂ by executing the following routine:

```

LOOP2    TAS      TEST
          BNE      LOOP2
          MOVE     (TRDATA), D2
          CLR.B    TEST
          -
          -
          -

```



Figure 2.17 Interfacing Two Processors Via Shared RAM

Initially the contents of the location TEST is zero. When Processor 1 executes the TAS instruction, it finds that (TEST) = 0; it sets (TEST) to 80₁₆ and falls through the wait loop “LOOP1”. Then it executes the MOVE D0 (TRDATA) instruction. At this point, assume that Processor 2 executes the TAS instruction. Processor 2 finds that (TEST) = 80, and it will not pass LOOP2 (because Z = 0). When Processor 1 is done with data transfer, it will execute the CLRB TEST instruction and so (TEST) are cleared to zero. This causes Processor 2 to fall through LOOP2, and hence it can access the shared memory. Observe that at any given time, the value of the memory byte TEST is 00 or 80H. For this reason, this byte often is referred to as *binary semaphore*.

2.4.6 I/O Instructions

I/O instructions allow a processor to perform input and output operations. An input instruction allows a peripheral to transfer a word to either a CPU register or memory. Similarly, an output instruction enables a processor to transfer a word into the buffer register of a peripheral device. In processors such as PDP-11 and MC68000, a peripheral device is mapped to a main memory address. In this situation, data moves from CPU to a memory location and vice versa constitute output and input operations, respectively. This method is known as *memory-mapped I/O*. The instruction set of any processor employing this approach does not include any special I/O instructions. Also, in this environment, a programmer can exploit all the available addressing modes for performing I/O operations in an efficient manner. In processors such as 8085, Z80, and 8086, a peripheral device can be mapped to an address in a separate address space called *I/O space*. The instruction set of each one of these processors include special input (IN) and output (OUT) instructions. These instructions are shorter than the regular move instructions, and so program execution time is expedited. Also, since the memory address space does not conflict with the I/O space, the entire memory can be completely utilized exclusively for storing code and data.

From the preceding discussion, we notice that the processor is completely tied up with I/O operations. This limits the system's *throughput* (number of tasks processed per unit time) because the operational speed of a peripheral device is typically 20 to 50 times lower than the processor. In order to overcome this difficulty, a hardware approach called *direct memory access* (DMA) is employed. In this method, the processor initiates an I/O operation by transferring typical parameters such as starting address in the memory, block size, and the type of operation to a hardware device called a DMA controller. After this, the DMA controller takes control of the I/O transfer. The processor is free to do any computation after initializing the DMA controller. Since I/O and CPU activities overlap, the DMA approach significantly increases a system's throughput.

2.5 REDUCED INSTRUCTION SET COMPUTER (RISC)

RISC, which stands for reduced instruction set computer, promises a new generation of faster and cheaper machines. The initial application of RISC principles has been in desktop workstations and minicomputers aimed at the fast-growing scientific and tech-

nical computing market. This market is currently worth \$8 billion, and sales could leap to \$28 billion by 1989. The major RISC vendors are Pyramid Technology, which makes UNIX-based supermini computers, and RIDGE computers, which produces computers for engineering workstations.

The basic idea behind RISC is for machines to cost less yet run faster, by using a small set of simple instructions for their operations. Also, RISC allows a balance between hardware and software based on functions to be achieved to make a program run faster and more efficiently. The philosophy of RISC embraces six principles: reliance on optimizing compilers, few instruction and addressing modes, fixed instruction format, instructions executed in one machine cycle, only call/return instructions accessing memory, and hardwired control.

The trend has always been to build CISCs (complex instruction set computers), which use many detailed instructions. However, because of their complexity, more hardware would have to be used, which actually would slow down the computer. The more instructions, the more hardware logic is needed to implement and support them. For example, in a RISC machine, an ADD instruction takes its data from registers. On a VAX, each operand can be stored in any of 14 different forms, so the compiler must check 14 possibilities.

The principles of understanding optimizing compilers and what actually happens when a program is executed lead to RISC. It turns out that RISC is really as much a philosophical approach as an architecture. To many implementers, RISC is just common sense.

However, not everyone in the industry favors using RISC as opposed to CISC. Some computer architects see RISC as a fad, or misleading claim. Their claim is that the advantages of RISC have nothing to do with reduced instruction sets. A study done by D. Patterson [7] at the University of California, Berkeley, showed that much of the performance of RISC and CISC machines has come from having lots of registers rather than from having few instructions. Critics also noted that RISC designs need to keep juggling program requirements with the number of available registers. If those factors got out of balance, there would be a lot of time-consuming memory swaps made by the processor, hence negating much of the performance advantage. According to Hewlett-Packard's Spectrum machine, which uses only 32 registers, this is not always the case.

The Spectrum line was the result of the reduction and simplification of the HP 300 instruction set. To exploit the increased power of the RISC-type architecture, users would employ an optimizing compiler to recompile automatically their existing application software. The software can run efficiently on everything from personal computers to mainframes. Hewlett-Packard was able to only use 32 registers based on a tremendous amount of analysis and simulation. Therefore, it achieved optimal performance with a very few registers.

Another argument critics have against RISC is the claim that the RISC technology is not well suited for modern general-purpose computing jobs. One example is floating-point arithmetic capability for high-precision numerical calculations. These operations require more than one machine cycle to execute; thus they do not fit into the single-cycle-per-instruction RISC philosophy. It is also very difficult to perform memory management or swapping chunks of data between devices such as disk drives and CPU memory with simple instructions. Therefore, none of the general-purpose computers

built with RISC principles are completely RISC machines. They are called “RISC-like.” This means most include at least a few complex instructions as well as instructions taking more than one machine cycle. However, there are some “pure” RISC machines that are built by universities, that have only 30 or 40 instructions.

Critics of RISC also claim RISC relies too heavily on compiler design. Without a good optimizing compiler, RISC is not better than CISC and may be worse. Opposing arguments claim writing the optimizing compiler for a RISC machine is easier than for a CISC machine because of the simple RISC design.

2.5.1 Case Study: RISC I (University of California, Berkeley)

The RISC machine presented in this section is the one investigated by D. Patterson and C. Sequin [8]. The authors proposed the computer RISC I with the following design constraints:

1. Only one instruction is executed per cycle.
2. All instructions have the same size.
3. Only load and store instructions can access memory.
4. High-level languages (HLL) are supported.

Owing to the larger user community of C and Pascal, these are the two languages that are considered for RISC I. A simple architecture implies a lesser number of transistors, and this leads to the fact that most pieces of a RISC HLL system are in software. Hardware is utilized for time-consuming operations. Using C and Pascal, a comparison study was made to determine the frequency of occurrence of particular variable and statement types. Studies revealed that integer constants appeared most frequently, and a study of the code produced revealed that the procedure calls are the most time-consuming operations.

Basic RISC Architecture

The RISC I instruction set contains a few simple operations (arithmetic, logical, and shift operations). These instructions operate on registers. Instruction, data, addresses and registers are all 32 bits long. RISC instructions fall in four categories: ALU, memory access, branch, and miscellaneous. The execution time is given by the time taken to read a register, perform an ALU operation, and store the result in a register. Register 0 always contains a 0. Load and store instructions move data between registers and memory. These instructions use two CPU cycles. Variations of memory-access instructions exist in order to accommodate sign-extended or zero-extended 8-bit, 16-bit and 32-bit data. Though absolute and register indirect addressing are not directly available, they may be synthesized using register 0. Branch instructions include CALL, RETURN, and conditional and unconditional jumps. The conditional instructions available are the standard ones used in PDP-11.

Instruction Format

opcode(7)	scc(1)	dest(5)	source1(5)	imm(1)	source2(13)
-----------	--------	---------	------------	--------	-------------

For register-to-register instructions, *dest* selects one of the 32 registers as destination of the result of the operation that is itself performed on registers *source1* and *source2*. If *imm* equals 0, the low-order 5 bits of *source2* specify another register. If *imm* equals 1, then *source2* is regarded as a sign-extended 13-bit constant. Since the frequency of integer constants is high, the immediate field has been made an option in every instruction. Also, *scc* determines whether the condition codes are set. Memory-access instructions use *source1* to specify the index register and *source2* to specify offset.

Register Windows

As mentioned earlier, the procedure-call statements take the maximum execution time. A RISC program has more call statements, since the complex instructions available in CISC are subroutines in RISC. The RISC register window scheme strives to make the call operation as fast as possible and also to reduce the number of accesses to data memory. The scheme works as follows.

Using procedures involve two groups of time-consuming operations, namely, saving or restoring registers on each call/return and passing parameters and results to and from the procedure. Statistics indicate that local scalars are the most frequent operands.

This creates a need to support the allocation of locals in the registers. One available scheme is to provide multiple banks of registers on the chip to avoid saving and restoring of registers. Thus each procedure call results in a new set of registers being allocated for use by that procedure. The return alters a pointer that restores the old set. A similar scheme is adopted by RISC. However, there are some registers that are not saved or restored; these are called *global registers*. In addition, the sets of registers used by different processes are overlapped in order to allow parameters to be passed. In other machines, parameters are usually passed on the stack with the calling procedure using a register to point to the beginning of the parameters (and also to the end of the locals). Thus all references to parameters are indexed references to memory. In RISC I the set of window registers (r10 to r31) is divided into three parts. Registers r26 to r31 (HIGH) contain parameters passed from the calling procedure. Registers r16 to r25 (LOCAL) are for local scalar storage. Registers r10 to r15 (LOW) are for local storage and for parameters to be passed to the called procedure. On each call, a new set of r10 to r31 registers is allocated. The LOW registers of the caller are required to become the HIGH registers of the called procedure. This is accomplished by having the hardware overlap the LOW registers of the calling frame with the HIGH registers of the called frame. Thus without actually moving the information, parameters are transferred (Refer to Figure 2.18 for an illustration.)

Multiple register banks require a mechanism to handle the case in which there are no free register banks available. RISC handles this problem with a separate register-overflow stack in memory and a stack pointer to it. Overflow and underflow are handled with a trap to a software routine that adjusts the stack. The final step in allocating variables in registers is handling the problem of pointers. RISC resolves this by giving addresses to the window registers. If a portion of the address space is reserved, we can determine with one comparison whether an address points to a register or to memory. Load and store are the only instructions that access memory and they take an extra cycle already. Hence this feature may be added without reducing the performance of the load and store instructions. This permits the use of straightforward computer technology and still leaves a large fraction of the variables in registers.

	<u>proc A</u>	<u>proc B</u>	<u>proc C</u>
HIGH _A	r31 _A		
	r26 _A		
LOCAL _A	r25 _A		
	r16 _A	r31 _B	
LOWA/HIGHB	r15 _A	r26 _B	
	r10 _A	r25 _B	
LOCAL _B		r16 _B	
		r15 _B	r31 _C
LOWB/HIGHC		r10 _B	r26 _C
			r25 _C
LOCAL _C			r16 _C
			r15 _C
LOW _C			r10 _C
GLOBAL			

Figure 2.18 Usage of Overlapped Register Windows (From Patterson, D., and C. Sequin *A VLSI RISC, Computer*, September 1982. Reprinted with permission.)

Delayed Jump

A normal RISC I instruction cycle is long enough to execute the following sequence of operations:

1. Read a register.
2. Perform an ALU operation.
3. Store the result back into a register.

Performance is increased by *prefetching* the *next instruction* during the current instruction. To facilitate this, jumps are redefined such that they do not occur until after the following instruction. This is called *delayed jump*, and its significance is explained in Chapter 7.

Evaluation

A study was made to compare RISC I and VAX and PDP-11 and MC68000. The results of the test are illustrated in Figure 2.19. This data is collected by studying the code generated by a C compiler for these four machines for the call return routine.

In the preceding test, two parameters are assumed to have been passed and three registers are saved.

2.5.2 RISC Assessment

The following material strives to show that RISC can provide as good a high-level language (HLL) environment as CISC. Two metrics are used to compare the performance of HLL programs on RISC and CISC. The first is speed and the second is the

System	Instructions executed	Size (bytes)	Register Accesses	Data Memory accesses
VAX	5	16	59	19
MC68000	9	30	41	12
PDP-11	19	44	51	15
RISC I	6	24	12	0.2

Figure 2.19 Performance Assessment of RISC I (From Patterson D., and R. Piepho, *Assessing RISCs in High-level Language Support*, IEEE Micro, 1982. Reprinted with permission.)

penalty of using HLL on a given machine. The index of evaluation used in the latter is the ratio of speed of execution of a program written in assembly to the speed of the same when written in HLL. This ratio is known as the HLL execution support factor (HLESF). A system with HLESF close to 0 penalizes the use of HLL, whereas an HLESF close to 1 does not reward the use of assembly language.

The results of an experiment conducted to evaluate the relative performance of RISC, 68000, Z80, and VAX-11/780 with respect to the preceding two metrics are reproduced in Figure 2.20. Five variables, or benchmark programs, have been utilized

Benchmark	Number of times slower than RISC			
	RISC (ms)	6800	Z80	VAX 11/780
E—String search	0.46	2.8	1.6	1.3
F—Bit test	0.06	4.8	7.2	4.8
H—Linked list	0.10	1.6	2.4	1.2
K—Bit matrix	0.43	4.0	5.2	3.0
I—Quicksort	50.40	4.1	5.2	3.0

HLESF - HLL Execution Support Factor				
Benchmark	RISC	6800	Z80	VAX11/780
E	0.62	0.17	0.32	0.23
F	1.00	0.23	0.27	0.34
H	1.00	0.92	0.96	0.88
K	0.94	0.21	0.29	0.34
I	0.92	0.16	0.44	0.47

Figure 2.20 Two Metrics to Assess the RISC's Effectiveness in HLL Support (From Patterson, D., and R. Piepho, *Assessing RISCs in High-level Language Support*, IEEE Micro, 1982. Reprinted with permission.)

for this experiment. These are selected as being representatives of frequent “real-world” problems. They allow manipulation of characters, integers, and floating-point data besides testing interrupt handling and addressing modes. Actually there are 12 such programs, but 7 were omitted either due to a lack of virtual memory or the difficulty involved in writing them in HLL. A brief description of the remaining 5 follows:

1. *String search* Examines a long character string for the first occurrence of a substring.
2. *Bit test, set and reset* Tests, sets, or resets a bit within a tightly packed bit string.
3. *Linked-list insertion*: Inserts a new entry into a doubly linked list.
4. *Quicksort*: Performs a nonrecursive quicksort algorithm on large vectors of fixed length records.
5. *Bit-matrix transportation*: Takes a tightly packed square bit matrix and transposes it.

RISC is a combination of scientific and philosophic rule called *Occam’s razor*, which states that among competing theories, the simplest should be preferred to the complex.

RISC, which is a trend toward simplicity, will eventually have a definite influence on the computers of the future.

QUESTIONS AND PROBLEMS

- 2.1** What are the characteristics of a good instruction format?
- 2.2** What are the merits and demerits of the block-code encoding technique?
- 2.3** Explain the key idea behind the expanding op-code and Huffman encoding techniques.
- 2.4** In a computer instruction format, the instruction length and the size of an address field are 11 and 4 bits, respectively. Is it possible to have
 - 5 two-address instructions
 - 45 one-address instructions
 - 32 zero-address instructionsusing this format? Justify your answer.
- 2.5** Using the instruction format of Problem 2.4, determine whether or not is it possible to have

- 7 two-address instructions
- 15 one-address instructions
- 18 zero-address instructions

Justify your answer.

- 2.6** Assume that a computer architect has already designed 6 two-address and 24 zero-address instructions using the instruction format of the problem 2.4. What is the maximum number of one-address instructions that can be added to the instruction set?
- 2.7** The instruction repertoire of a computer consists of 8 instructions, $I_0, I_1, I_2, \dots, I_7$. The relative frequency of these instructions are as follows:

INSTRUCTION	RELATIVE FREQUENCY
I_0	$\frac{1}{4}$
I_1	$\frac{1}{8}$
I_2	$\frac{1}{8}$
I_3	$\frac{1}{8}$
I_4	$\frac{1}{8}$
I_5	$\frac{1}{8}$
I_6	$\frac{1}{16}$
I_7	$\frac{1}{16}$

Encode these instructions using Huffman's method.

Calculate the redundancy introduced by Huffman's scheme.

- 2.8** What is the essential difference between base and index register addressing modes?
- 2.9** Mention some important uses of each one of the following addressing modes.
- a)** Immediate
 - b)** Postincrement
 - c)** Predecrement
 - d)** Indexed
 - e)** Base register
 - f)** Relative
- 2.10** In a hypothetical computer, the system can directly address 16M bytes. The CPU of this computer has 16 base registers. Determine the size of the displacement

field so that we can address 6.25% of the total memory addresses (without changing the contents of the base registers).

- *2.11** In a computer, the main memory is divided into 256 pages and each page contains 256 bytes. The instruction format of this computer is as follows:

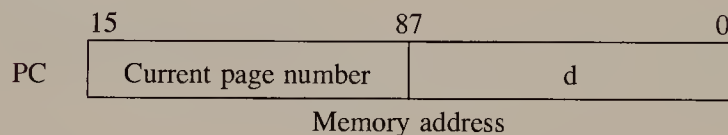
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

Bits 0–7 Address of a byte within a page (d)

Bit 8 0—direct addressing
 1—Indirect addressing

Bit 9 0—a reference to page 0
 1—a reference to the current page which is determined by the high-order 8-bits of the PC as shown below

Bits 10–15 Op-code



- a. How much of main memory can be directly addressed?
- b. How much of main memory can be indirectly addressed?

- 2.12** A computer instruction set supports the following Boolean instructions:

XOR X, Y ; $Y \leftarrow X \text{ Exclusive or } Y$

AND X, Y ; $Y \leftarrow X \text{ AND } Y$

Explain how will you perform the Boolean-OR operation $Y \leftarrow X \text{ OR } Y$ using the above two instructions.

- 2.13** Assume that a CPU has only two registers, R_1 and R_2 , and that only the following instruction is available.

XOR R_i, R_j ; $R_j \leftarrow R_i + R_j$
 ; $i, j = 1, 2$

Using this XOR instruction, find an instruction sequence in order to exchange the contents of the registers R_1 and R_2 .

*From Danhof, K. J. and C. L. Smith, *Computing Systems Fundamentals: An Approach Based on Microcomputers*, Addison-Wesley, 1981, p. 24. Reprinted with permission.

- 2.14** Explain why position independent codes are very important with operating system-supported routines.
- 2.15** What are the salient features of a RISC machine?

REFERENCES

1. Danhof, K. J., and C. L. Smith. *Computing System Fundamentals: An Approach Based on Microcomputers*. Reading, Mass.: Addison-Wesley, 1981.
2. Fuller, S. H., "Performance Evaluation," in *Introduction to Computer Architecture* edited by H. S. Stone, New York: SRA, 1980.
3. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
4. Hayes, J. P. *Digital System Design and Microprocessors*. New York: McGraw-Hill, 1984.
5. Knuth, D. E. *The Art of Computer Programming*. Vol. 1. New York: Addison-Wesley, 1969.
6. Mano, M. M. *Computer Systems Architecture*, Englewood Cliffs, N.J.: Prentice-Hall, 1983.
7. Patterson, D. "Reduced Instruction Set Computers." *CACM* (January 1985).
8. Patterson, D., and C. Sequin. "A VLSI RISC." *Computer* (September 1982).
9. Patterson, D., and R. Piepho. "Assessing RISCs in High-Level Language Support." *IEEE Micro*, (November 1982).
10. Rafiquzzaman, M. *Microcomputer Theory and Applications with the Intel SDK-85.* 2d. ed. New York: John Wiley, 1987.
11. Tanenbaum, A. S. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1984.

CHAPTER 3

Execution Unit

In this chapter, the typical components of a computer execution unit are discussed:

1. General Purpose registers
2. ALU
3. Dedicated hardware or firmware to perform special operations such as multiplication and division

In addition, the advanced topics of coprocessors, which are used to enhance the performance of the main execution unit, and LSI bit-slice processors are discussed.

3.1 INTRODUCTION

As mentioned previously, in a conventional stored-program computer, instruction codes are brought into the CPU from the main memory, where they are executed by the execution unit. Unless otherwise specified, instructions are always fetched in a strictly sequential fashion. The control unit, as the word implies, controls the sequencing of data and instructions coming into the ALU to be executed.

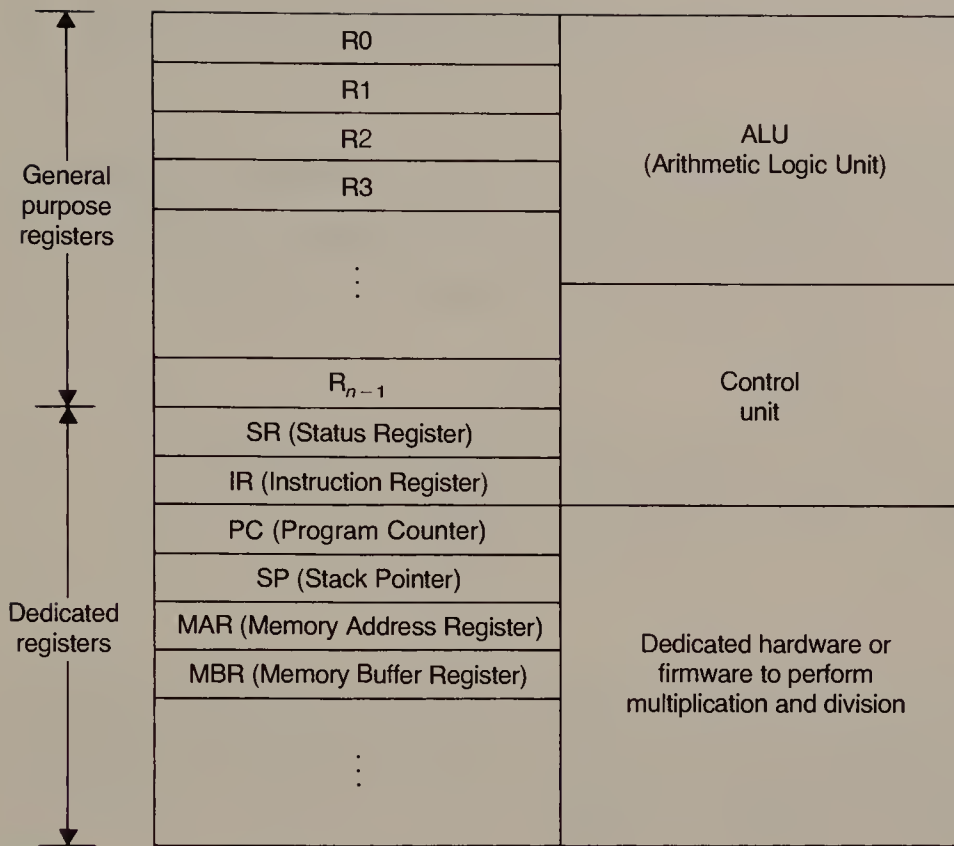


Figure 3.1 A Typical CPU Model

For the purpose of discussion, a typical CPU is modeled as shown in Figure 3.1. Figure 3.1 shows that a conventional CPU consists of the following:

1. General-purpose registers
2. Dedicated registers
3. An ALU
4. Dedicated hardware or firmware elements that perform a special operation such as multiplication or division.
5. A control unit

3.2 REGISTER SECTION

The register section is an important part of any CPU. A CPU with many registers reduces the number of visits to the main memory, therefore simplifying the programming task and shortening execution time. All CPU registers can be classified into two groups: general-purpose and dedicated registers. A general-purpose register may be configured as an accumulator, address pointer, or a data pointer. The contents of dedicated registers such as a program counter and stack pointer, are reserved for program execu-

tion. A summary of some commonly known special-purpose registers and their tasks follows:

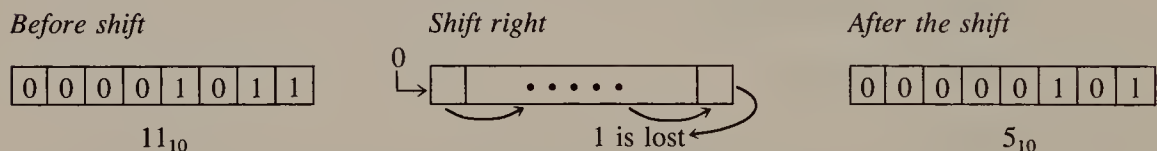
REGISTER	TASK
PC	Usually holds the address of the next instruction to be executed.
SP	Usually holds the address of the top element of the stack.
IR	Holds the instruction code currently being executed.
MAR	Holds the address of the data item to be retrieved from the main memory.
MBR	Holds the data item retrieved from the main memory.
SR or PSW	Holds the condition code flags and other information that describe the status of the running program.

3.2.1 General Register Design

The primary task of a general-purpose register (GPR) is to store address or data for an indefinite amount of time, then to be able to retrieve the data when needed. A GPR is also capable of manipulating the stored data by shift left or right operations. Figure 3.2 contains a summary of typical shift operations.

In a logical shift operation, a bit that is shifted out will be lost, and the vacant position will be filled with a 0. For example, if we have the number $(11)_{10}$, after right shift, the following occurs:

8-bit word

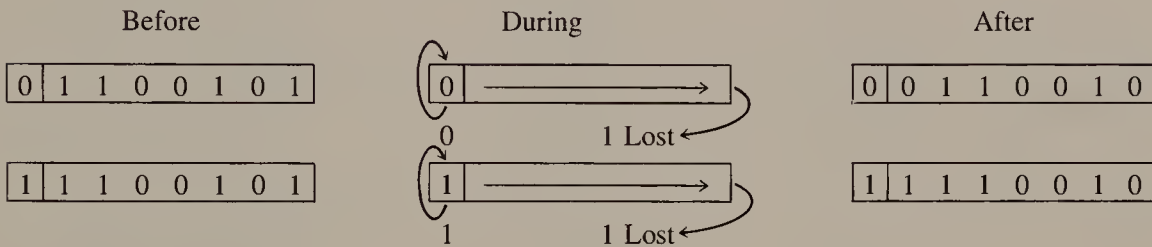


Shift type	Logical	Arithmetic	Rotate
Right			
Left			

Figure 3.2 Summary of Typical Shift Operations

It must be emphasized that a logical left or right shift of an unsigned number by n positions implies multiplication or division of the number by 2^n , respectively, provided that a 1 is not shifted out during the operation.

In case of true arithmetic left or right shift operations, the sign bit of the number to be shifted must be retained. However, in most typical computers, this is true for right shift and not for left shift operation. For example, if a register is shifted right arithmetically, the most-significant bit (MSB) of the register is preserved, thus ensuring that the sign of the number will remain constant. This is illustrated next:



There is no difference between arithmetic and logical left shift operations. If the most-significant bit changes from 0 to 1, or vice versa, in an arithmetic left shift, the result is incorrect. For example, if the original value of the register is $(3)_{10}$, the results of two successive arithmetic left shift operations are interpreted as follows:

ORIGINAL	AFTER FIRST SHIFT	AFTER SECOND SHIFT
0011 ₂ = $(3)_{10}$	0110 ₂ = $(6)_{10}$	1100 ₂ = (-4)
	$3 \times 2 = 6$, correct	$6 \times 2 = 12$ not -4 , incorrect

For this reason, in processors such as the MC68000, the overflow flag bit (V-bit) of the status register(SR) is set when the MSB is changed at any time during an arithmetic shift left (ASL); otherwise the flag is cleared. The overflow flag is not affected during a logical left shift.

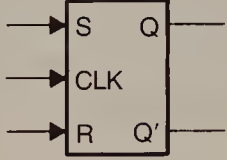
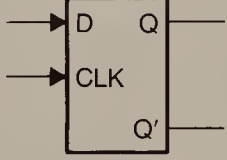
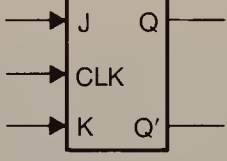
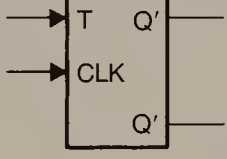
Rotation is useful for the serialization of parallel data. Peripheral devices such as a cassette recorders will expect serial 8-bit data. In this situation, the CPU transfers the contents of an 8-bit CPU register to the cassette recorder by rotating the register 8 times.

Let us now discuss the hardware aspects of a register. From a hardware perspective, a register can be viewed as an interconnection of a set of flip-flops. The characteristics of standard flip-flops are summarized in Table 3.1.

In order to design a general purpose register, first let us propose a basic cell S.

The internal organization of the S cell is shown in Figure 3.3. A 4-input multiplexer selects one of the external inputs as the D flip-flop input, and the selected input appears at the flip-flop output Q after the clock pulse. The CLR input is an asynchronous clear input, and whenever this input is asserted (held low), the flip-flop is cleared to zero. Using the basic cell S as the building block, a 4-bit GPR can be designed. Its schematic representation is shown in Figure 3.4.

Table 3.1 A SUMMARY OF STANDARD FLIP-FLOPS

Flip-flop Type	Block Diagram Representation	Function Table	Characteristic Equation	Excitation Table
S – R		S R Q ⁺	$Q^+ = S + R' \cdot Q$ with $S \cdot R = 0$	Q Q ⁺ S R
		0 0 Q		0 0 0 X
		0 1 0		0 1 1 0
		1 0 1		1 0 0 1
		1 1 ?		1 1 X 0
D Flip-flop		D Q ⁺	$Q^+ = D$	Q Q ⁺ D
		0 0		0 0 0
		1 1		0 1 1
				1 0 0
				1 1 1
JK Flip-flop		J K Q ⁺	$Q^+ = J \cdot Q' + K' \cdot Q$	Q Q ⁺ J K
		0 0 Q		0 0 0 X
		0 1 0		0 1 1 X
		1 0 1		1 0 X 1
		1 1 Q'		1 1 X 0
T Flip-flop		T Q ⁺	$Q^+ = T \oplus Q$	Q Q ⁺ T
		0 Q		0 0 0
		1 Q'		0 1 1
				1 0 1
				1 1 0

Note: Q⁺: Next state—that is, the value of Q after clock.

X: Don't care.

The truth table modeling the operation of this register is shown in Figure 3.5.

This table shows that manipulation of the selection inputs S_1 and S_0 can cause three distinct operations: parallel load, shift left, and shift right. If $S_1 S_0 = 11$, the external inputs X_3 through X_0 are selected as the D-inputs for the flip-flops, the output V_i will follow the input x_i after the clock. By choosing the correct values for serial shift inputs R and L, logical, arithmetic or rotate shifts can be achieved. Figure 3.6 shows how to realize three different right shifts by choosing the appropriate values for the right shift serial input R.

This register can be loaded with any desired data in a serial fashion. For example, after 4 successive right shift operations, data X will be loaded into the register if the register is set in the right shift mode and the required data X is applied serially to input R.

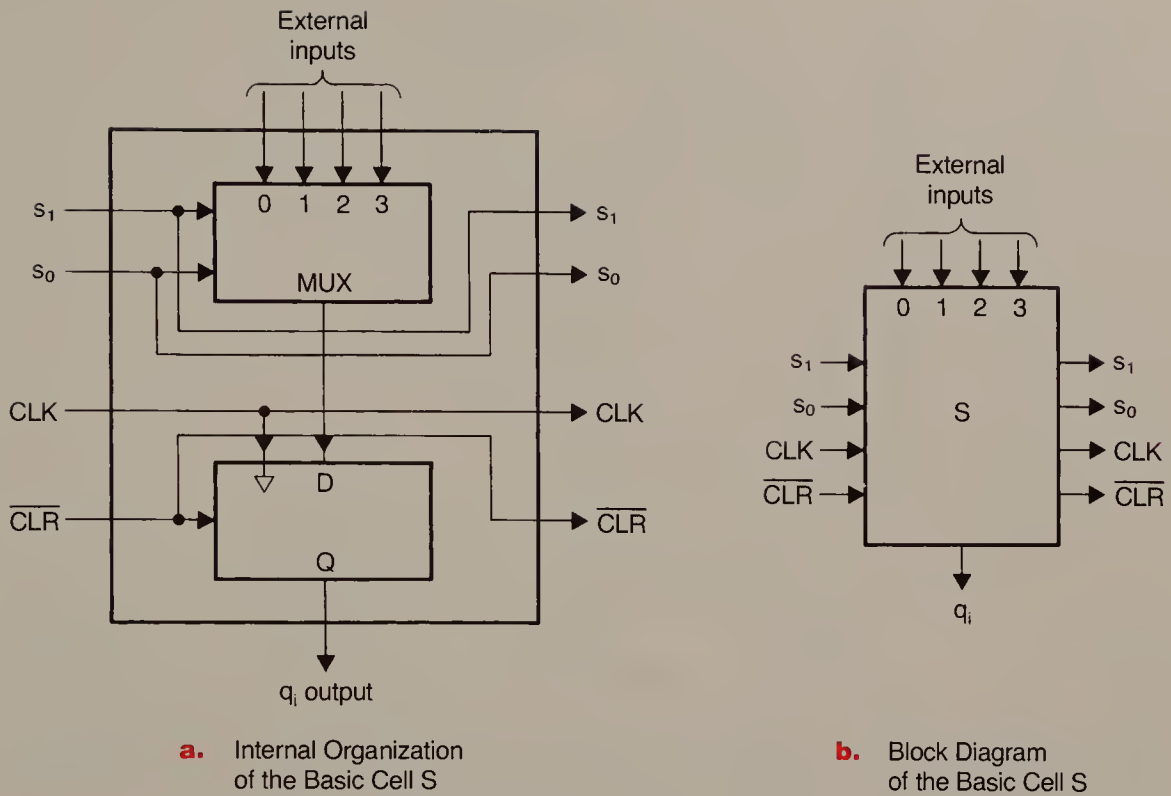


Figure 3.3 A Basic Cell for Designing a GPR

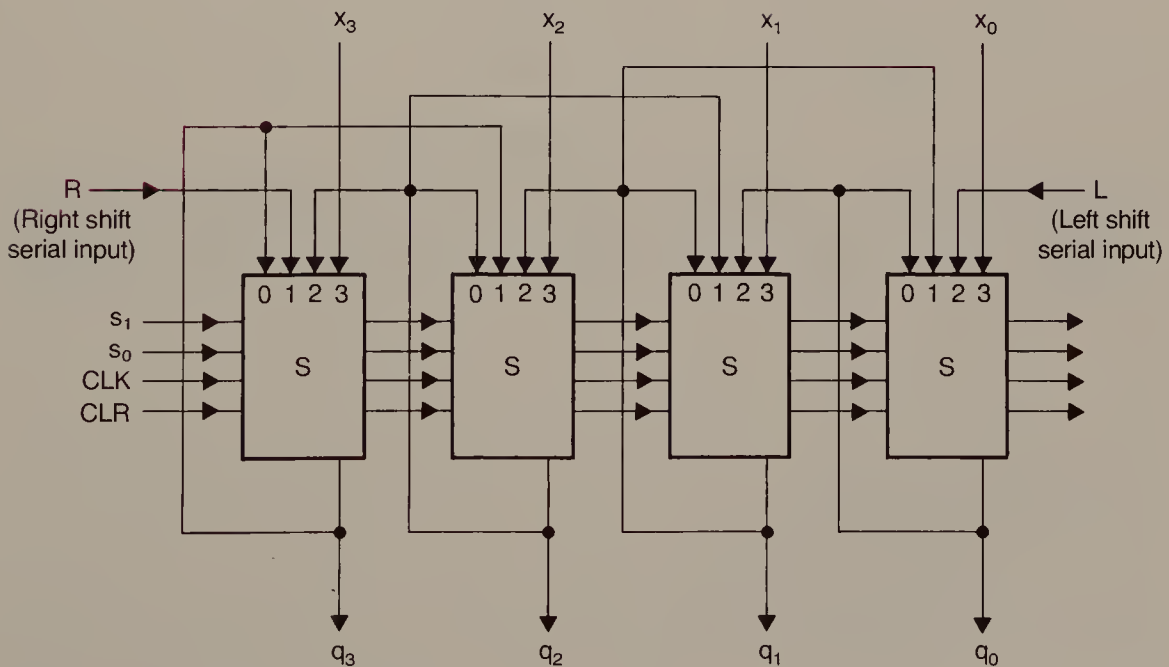


Figure 3.4 A 4-bit General Register

Selection Inputs		Clock Input	Clear Input	Operation
s_1	s_0	CLK	$\overline{\text{CLR}}$	
X	X	X	0	Clear
0	0		1	No operation
0	1		1	Shift right
1	0		1	Shift left
1	1		1	Parallel load

Note: X – don't care;  : rising edge of the clock

Figure 3.5 Truth Table Controlling the Operation of the General Register

3.2.2 Combinational Shifter Design

The optional speed of a flip-flop-based shifter is a function of the clock frequency. A high-speed shifter can be designed using combinational circuit components such as a multiplexer. The block diagram, internal organization, and the truth table of a typical combinational shifter are shown in Figure 3.7. From the truth table, the following equations can be obtained:

$$Y_3 = s_1's_0'i_3 + s_1's_0'i_2 + s_1s_0'i_1 + s_1s_0'i_0$$

$$Y_2 = s_1's_0'i_2 + s_1's_0'i_1 + s_1s_0'i_0 + s_1s_0'i_{-1}$$

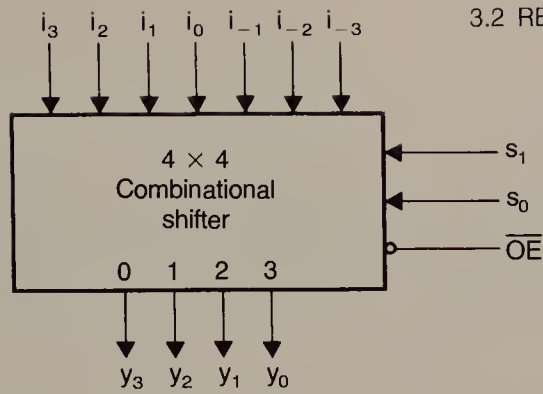
$$Y_1 = s_1's_0'i_1 + s_1's_0'i_0 + s_1s_0'i_{-1} + s_1s_0'i_{-2}$$

$$Y_0 = s_1's_0'i_0 + s_1's_0'i_{-1} + s_1s_0'i_{-2} + s_1s_0'i_{-3}$$

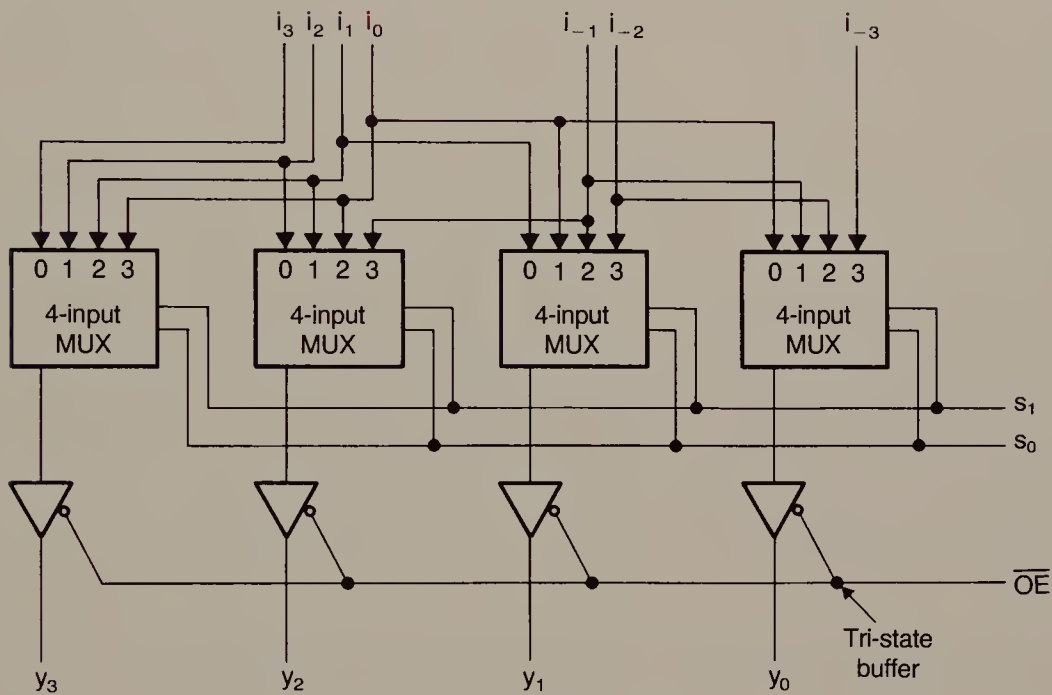
Combinational shifters of this type are produced by Texas Instruments (74350) and Advanced Micro-Devices (AM25S10). The 4×4 shifter of Figure 3.7 can be expanded to obtain a system capable of rotating 16-bit data to the left by 0, 1, 2, or 3 positions. (see Figure 3.8.)

$s_1s_0 = 01$	
Value of R	Shift realized
0	Logical right shift
q_3	Arithmetic right shift
q_0	Rotate right

Figure 3.6 Three Variations of the Right Shift Operations



a. Block Diagram



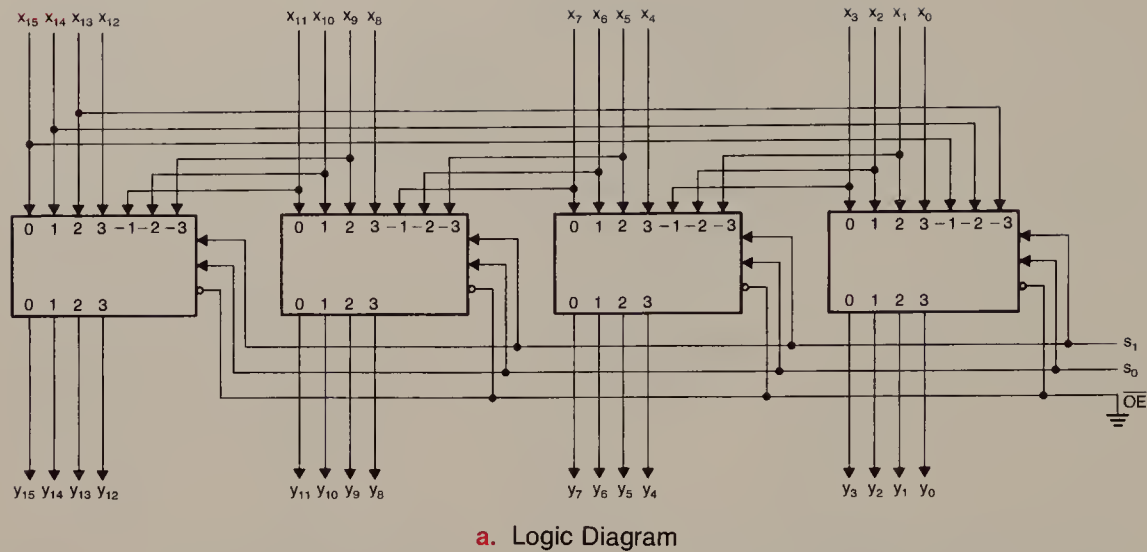
b. Internal Schematic

$\overline{\text{OE}}$	Shift Count		Output				Comment
	s_1	s_0	y_3	y_2	y_1	y_0	
1	X	X	Z	Z	Z	Z	Output lines float
0	0	0	i_3	i_2	i_1	i_0	Pass (no shift)
0	0	1	i_2	i_1	i_0	i_{-1}	Left shift once
0	1	0	i_1	i_0	i_{-1}	i_{-2}	Left shift twice
0	1	1	i_0	i_{-1}	i_{-2}	i_{-3}	Left shift three times

Note: Z—High-impedance state.
X—Don't care.

c. Truth Table

Figure 3.7 4×4 Combinational Shifter



Shift Count		Output															
s_1	s_0	y_{15}	y_{14}	y_{13}	y_{12}	y_{11}	y_{10}	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
0	1	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}
1	0	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}
1	1	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}

b. Truth Table**Figure 3.8** Combinational Shifter Capable of Rotating 16-bit Data to the Left by 0, 1, 2, or 3 Positions

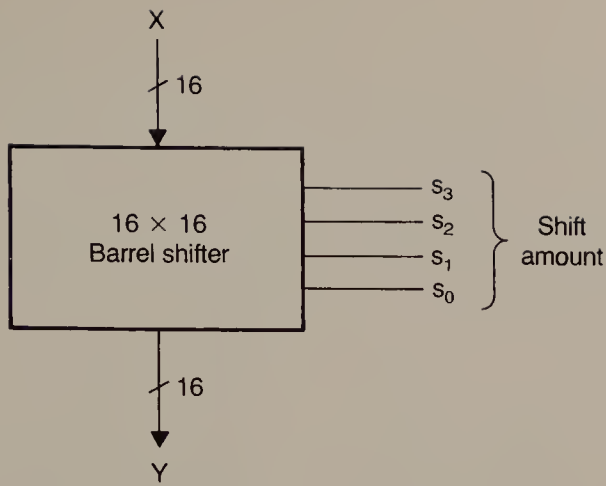
This design can be extended to obtain a more-powerful shifter called the *barrel shifter*.

The block-diagram representation of a 16×16 barrel shifter is shown in Figure 3.9(a). This shifter is capable of rotating the given 16-bit data to the left by n positions, where $0 \leq n \leq 15$. Figure 3.9(b) shows the truth table representing the operation of this shifter.

Functionally, the barrel shifter can be divided into two 16×16 combinational logic units (see Figure 3.9(c)). The level 1 shifter rotates the given 16-bit data (x) i -positions to the left, where $0 \leq i \leq 3$. The level 2 shifter can rotate the input data to the left by $4k$ positions where $0 \leq k \leq 3$. For example – if x is to be rotated by 11 positions, then set $i = 3$ ($S_1S_0 = 11$) and $k = 2$ ($S_3S_2 = 10$).

The complete logic diagram of the final hardware is shown in Figure 3-9(d). The level 1 shifter is identical to the hardware developed earlier (see Figure 3.8). The interconnections between the level 1 and level 2 shifters are marked to maintain the clarity of the diagram. The task of verifying the correctness of the design is left as an exercise.

The barrel shifter is an on-chip component for VLSI 32-bit microprocessors such

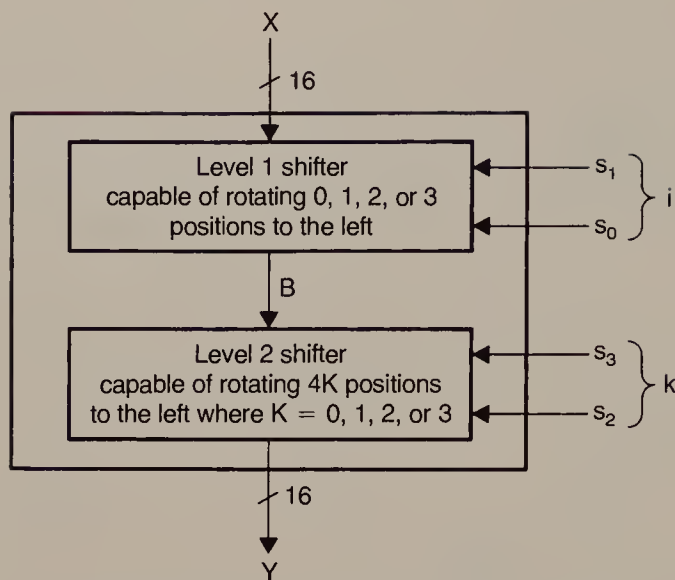


a. Block Diagram of a 16×16 Block Shifter

Shift Amount				Output															
s_3	s_2	s_1	s_0	y_{15}	y_{14}	y_{13}	y_{12}	y_{11}	y_{10}	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	0	0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
0	0	0	1	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}
0	0	1	0	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}
0	0	1	1	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}
0	1	0	0	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}
0	1	0	1	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}
0	1	1	0	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}
0	1	1	1	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9
1	0	0	0	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8
1	0	0	1	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7
1	0	1	0	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6
1	0	1	1	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5
1	1	0	0	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4
1	1	0	1	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3
1	1	1	0	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2
1	1	1	1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1

b. Truth Table of the 16×16 Barrel Shifter

Figure 3.9



c. Functional Block Diagram of a 16×16 Barrel Shifter

as the MC 68020 and the Intel 80386. The barrel shifter of the MC 68020 processor chip occupies 7.0% of the total chip area. The 80386 barrel shifter is capable of shifting a number from 0 through 64 positions in one clock period (clock rate is 16 MHz). Such high-speed shifting is desirable for manipulating floating-point numbers.

In scientific computation, numbers that are too large as well as too small need to be handled. In order to obtain such a wide range, numbers are represented by a special method called *floating-point notation*, similar to scientific notation. A complete discussion of this topic is beyond the scope of this chapter.

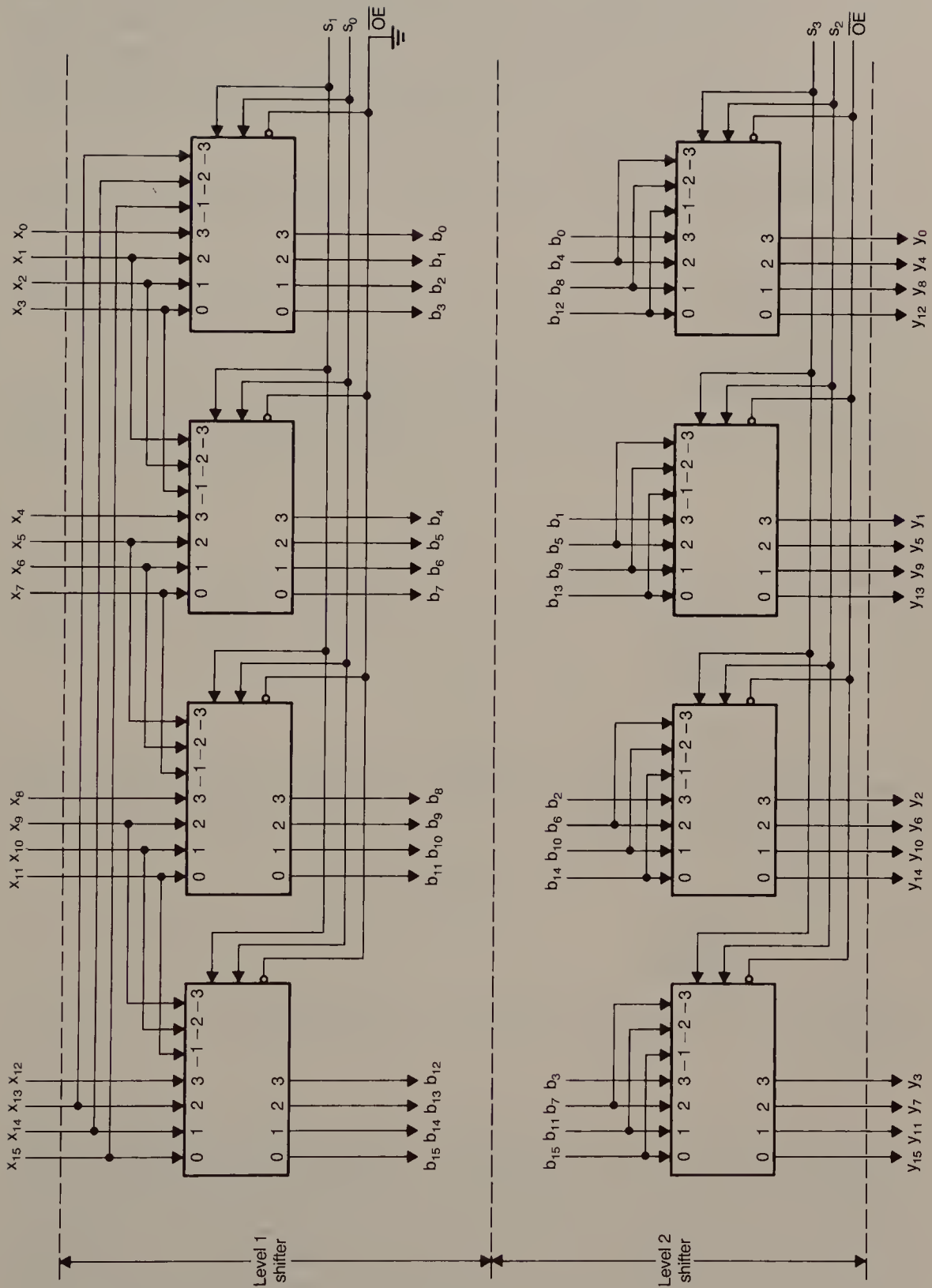
3.2.3 Flag Register

Previously, it was mentioned that the program status is held by a set of condition codes. In reality, condition codes are realized by designating one flip-flop for each condition. These flip-flops are typically set or reset depending on the result produced by the most recent arithmetic or a logical operation. In the following, it is assumed that the condition code register contains only the following flags:

Z (zero), N (negative), C (carry), and V (overflow)

The Z flag is set to 1 if the most-recent operation produced a zero result. The N flag is set to 1 if the most-recent arithmetic operation produced a negative result. In processors such as the PDP-11 and the MC68000, the C and V flags are manipulated in the following manner:

C Flag During the addition operation, the C flag is set to 1 if a carry is generated from the most-significant bit position; otherwise it is reset to 0. If the operation performed is a subtraction, the C flag is set to 1 when no carry is generated from the most-significant position.



d. Complete Logic Diagram Full 16-bit Barrel Shifter

V Flag During an addition operation, the V flag is set to 1 only when the operands added have the same sign and produce a sum of opposite sign. During a subtraction operation, the V-flag is set to 1 only when the operands A and B have different signs with $A - B$ and B having the same sign. It can be shown that for 8-bit signed operation $V = c_7 \oplus c_8$ where c_8 is final carry and c_7 is previous carry.

To illustrate how condition flags are affected, the following example is provided:



Example 3.1

Given that

$$A = 01100010$$

$$B = 01000101$$

determine the status of the condition flags if the operation performed were as follows.

a. $A + B$

b. $A \wedge B$

Solution

a.

A	01100010
B	01000101
$A + B$	10100111
	$\downarrow \quad \searrow$ $C_8 = 0 \quad C_7 = 1$

From this result, the sign of the result is negative: no overflow and no end-around carry; the result is nonzero. Therefore,

Therefore, $N = 1$, $V = 1$, $C = 0$, and $Z = 0$.

b.

A	10100010
B	01000101
$A \wedge B$	00000000

Following the same reasoning used in part (a), it is inferred that $N = 0$, $V = 0$, $C = 0$, and $Z = 1$.



Using condition codes, a programmer can detect whether or not an arithmetic operation produced a valid result. For instance, if the addition of two unsigned integers results in an overflow (carry), then C-flag is set to 1. Similarly, the V-flag serves as an overflow/underflow indicator when two signed integers represented in 2's complement form are added or subtracted. The condition codes are also useful in determining the relative magnitudes of two n -bit unsigned integers A and B. The previous chapter showed that by using compare or test, instructions decisions such as whether $A = B$ or $A \neq B$ can be made.

Note that the compare instruction of the form $\text{CMP } A, B$ subtracts B from A and

sets the flags based upon difference produced. After the compare operation, if the Z flag is set to 1, then $A = B$; otherwise $A \neq B$. From the fundamentals of digital arithmetic, it is known that

$$\begin{aligned} A - B &= A + 2\text{'s complement of } B \\ &= A + B' + 1 \\ &= A + 2^n - B \\ &= 2^n + A - B \end{aligned}$$

Observe that if B is an n -bit unsigned integer, then the following relationship holds: 2's complement of $B = 2^n - B$.

Suppose that $A > B$. This means that $A - B$ is positive, and thus $2^n + A - B > 2^n$. But, an unsigned n -bit integer can cover numbers in the range of 0 and $2^n - 1$. Therefore, when $A > B$, the subtraction operation produces a carry from the most-significant position, and thus the C-flag is reset to 0. $A > B$ also implies that $A \neq B$. Hence, we have the following interpretation:

$$C = 0 \quad \text{and} \quad Z = 0 \quad \text{implies that } A > B$$

$$C = 0 \quad \text{implies that } A \geq B$$

The negation of $A > B$ is true only when $A \leq B$. Similarly, the negation of $A \geq B$ is equivalent to asserting the condition $A < B$. This result suggests that the following interpretations in terms of C and Z flags are possible.

$$C = 1 \quad \text{or} \quad Z = 1 \quad \text{implies that } A \leq B$$

$$C = 1 \quad \text{implies that } A < B$$

Suppose that the operands A and B are represented in 2's complement form. Then the Z-, N-, and V-flags are used to determine their relative magnitudes. This interpretation is summarized in Figure 3.10.

The verification of this interpretation is left as an exercise.

FLAG STATUS	INTERPRETATION
$Z = 0$	$A \neq B$
$Z = 1$	$A = B$
$(Z = 0)$ and $(VN = 00 \text{ or } VN = 11)$	$A > B$
$(VN = 00 \text{ or } VN = 11)$	$A \geq B$
$(VN = 01 \text{ or } VN = 10)$	$A < B$
$Z = 1$ or $(VN = 01 \text{ or } VN = 10)$	$A \leq B$

Figure 3.10 Status Flag Interpretation
When A and B are in 2's Complement Form

3.3 ADDERS

Addition is the basic operation performed by an ALU. The operation is versatile, since $A - B$ is equivalent to $A + 2$'s complement of B , and $C * A$ may be obtained by adding A to itself for $C - 1$ times. Thus the speed of the adder unit is essential to the efficient operation of an execution unit.

A hardware unit that adds two single-bit numbers with a carry is called a full adder. For convenience, the block diagram, the truth table and the implementation of a full adder are shown in Figure 3.11.

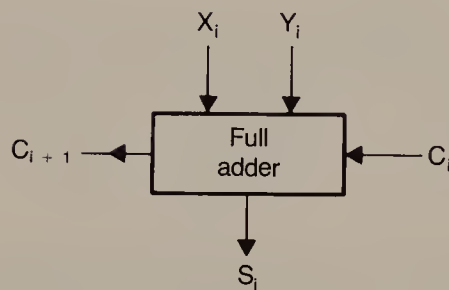
$$\text{From the truth table, } c_i = x_i' y_i' c_i + x_i' y_i c_i' + x_i y_i' c_i' + x_i y_i c_i$$

$$= c_i(x_i' y_i' + x_i y_i) + c_i'(x_i' y_i + x_i y_i')$$

$$\text{Let } z = x_i' y_i + x_i y_i' = x_i \oplus y_i$$

$$\text{then } c_i = c_i z' + c_i' z$$

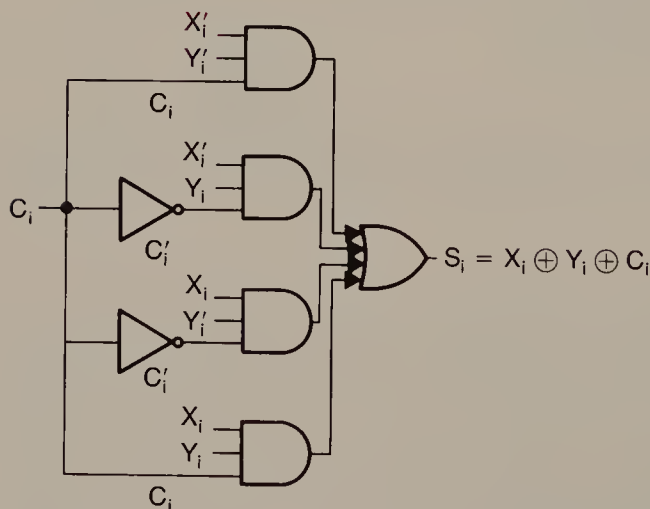
$$= z \oplus c_i = x_i \oplus y_i \oplus c_i$$



a. Block Diagram

X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

b. Truth Table



c. Implementation

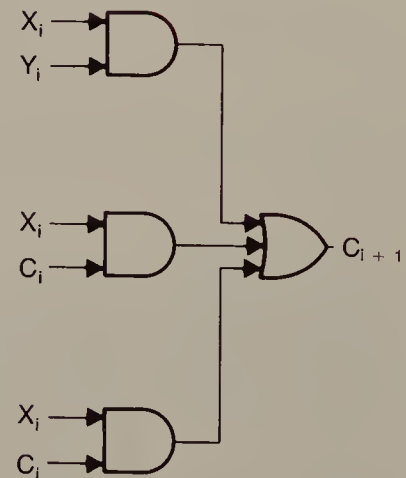


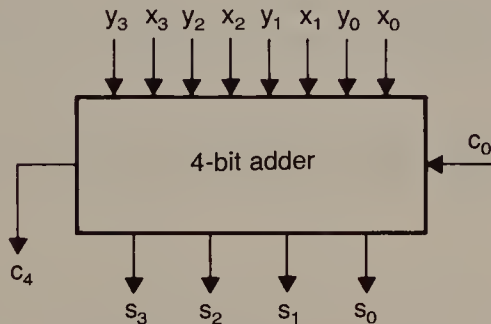
Figure 3.11 Full Adder

$$\begin{aligned}
 \text{Also, } c_{+1} &= x_i' y_i c_i + x_i y_i' c_i + x_i y_i c_i' + x_i y_i c_i \\
 &= x_i' y_i c_i + x_i y_i c_i + x_i y_i' c_i + x_i y_i c_i + x_i y_i c_i' + x_i y_i c_i \\
 &= y_i c_i + x_i c_i + x_i y_i
 \end{aligned}$$

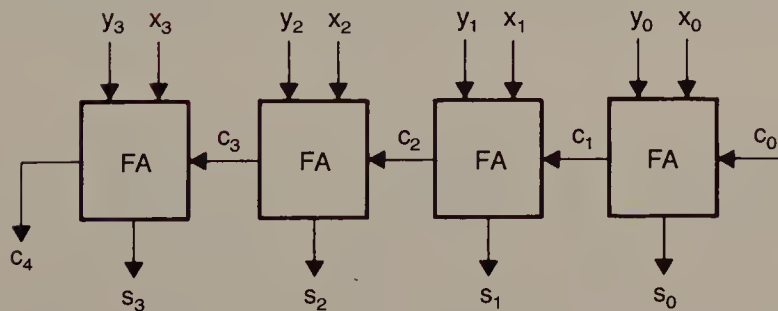
As has been made apparent by Figure 3.11, for generating C_{i+1} from C_i , two gate delays are required. In order to generate S_i from C_i , three gate delays are required, since C_i must be inverted to obtain C_i' . Note that no inverters are required to get X_i' or Y_i' from X_i or Y_i , respectively, since the numbers to be added are usually stored in a register that is a collection of flip-flops. The flip-flop often generates both normal and complemented outputs. For the purpose of discussion, assume that the gate delay is Δ time units, and the actual value of Δ is decided by the technology. For example, if Transistor Translator Logic (TTL) circuits are used, the value of Δ will be 10 ns.

By cascading n full adders, an n -bit parallel adder capable of handling two n -bit operands (X and Y) can be designed. The implementation of a 4-bit ripple-carry adder is shown in Figure 3.12.

When two unsigned integers are added, the input carry, C_0 , is always zero. Yet, when this system is to operate on 2's complement numbers, the value of C_0 may be zero or one. (This is explained later in the section). The 4-bit adder is also called a



a. Block Diagram of a 4-bit Ripple-Carry Adder



b. Four Full Adders are Cascaded to implement a 4-bit Ripple-Carry Adder

Figure 3.12 Implementation of a 4-bit Ripple-carry Adder

carry-propagate adder (CPA), since the carry is propagated serially through each full adder. This hardware can be cascaded to obtain a 16-bit CPA, as shown in Figure 3.13.

Although the design of an n -bit CPA is straight forward, the carry propagation time limits the speed of operation. For example, in the 16-bit CPA (see Figure 3.13), the addition operation is completed only when the sum bits S_0 through S_{15} are available.

To generate S_{15} , C_{15} must be available. The generation of C_{15} depends on the availability of C_{14} , which must wait for C_{13} to become available. In the worst case, the carry propagates through 15 full adders. Therefore, the worst-case add-time of the 16-bit CPA can be estimated as follows:

$$\begin{aligned} &\text{Time taken for carry to propagate} \\ &\quad \text{through 15 full adders (the delay} \\ &\quad \text{involved in the path from } C_0 \text{ to } C_{15}) = 15 * 2\Delta \\ &\text{Time taken to generate } S_{15} \text{ from } C_{15} = \frac{3\Delta}{33\Delta} \end{aligned}$$

If $\Delta = 10$ ns, then the worst-case add-time of a 16-bit CPA is 330 ns. This delay is prohibitive for high-speed systems in which the expected add-time is typically less than 100 ns. This creates the necessity to devise a new technique to increase the speed of operation by a factor of 3. One such technique is known as the carry look-ahead. In this approach the extra hardware is used to generate each carry (C_i , $i > 0$) directly from C_0 . To be more practical, consider the design of a 4-bit carry look-ahead adder (CLA). It will be shown how this may be used to obtain 16-bit adder that operates at a speed higher than the 16-bit CPA.

Recall that in a full adder, the output carry C_{i+1} is related to its carry input, C_i , as follows:

$$C_{i+1} = X_i Y_i + X_i C_i + Y_i C_i$$

This result can be rewritten:

$$C_{i+1} = G_i + P_i C_i \quad \text{where} \quad G_i = X_i Y_i \quad \text{and} \quad P_i = X_i + Y_i$$

The function G_i is called the *carry-generate function*, since a carry is generated when $X_i = Y_i = 1$. If X_i or Y_i is a 1, then the input carry C_i is propagated to the next

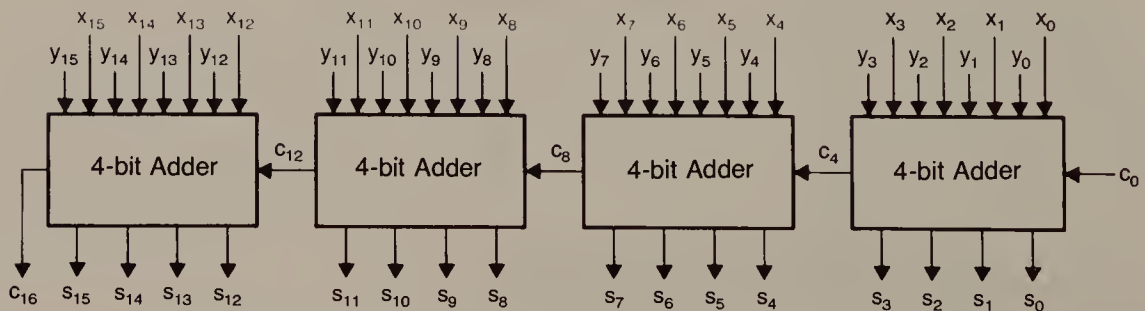


Figure 3.13 Implementation of a 16-bit Adder Using 4-bit Adders as Building Blocks

stage. For this reason, the function P_i is often referred to as *carry-propagate function*. Using G_i and P_i , C_1 , C_2 , C_3 , and C_4 can be expressed as follows:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1$$

$$C_3 = G_2 + P_2C_2$$

$$C_4 = G_3 + P_3C_3$$

These equations are recursive, and the recursion can be removed as indicated below:

$$C_1 = G_0 + P_0C_0$$

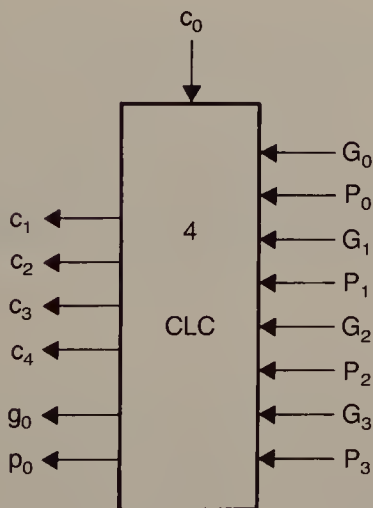
$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

$$\begin{aligned} C_3 &= G_2 + P_2C_2 = G_2 + P_2(G_1 + P_1G_0 + P_1P_0C_0) \\ &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3C_3 = G_3 + P_3(G_2 + P_2G_1 + P_2P_1G_0 + P_1P_1P_0C_0) \\ &= G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0 \end{aligned}$$

This result suggests that C_1 , C_2 , C_3 , and C_4 can be generated directly from C_0 . For this reason, these equations are called *carry look-ahead equations*, and the hardware that implements these equations is called a *4-stage carry look-ahead circuit (4-CLC)*. The block diagram of such circuit is shown in Figure 3.14. Following are some important points about this system:

- A 4-CLC can be implemented as a two-level AND/OR logic circuit (The first level comprises AND gates, whereas the second level comprises OR gates). In



NOTE: $g_0 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$
 $p_0 = P_3P_2P_1P_0$

Figure 3.14 A Four-Stage Carry Look-ahead Circuit

particular, the fan-in requirement for this realization is 5. To generate $P_3 P_2 P_1 P_0 C_0$, a 5-input AND gate is required; to generate C_4 from C_0 a 5-input OR gate is needed.

- If $G_i P_i$ ($0 \leq i \leq 3$) is available, then all C_i 's with $1 \leq i \leq 4$ can be generated in 2 gate delays.
- The outputs g_0 and p_0 are useful to obtain a higher-order look-ahead system.

To construct a 4-bit CLA, assume the existence of the basic adder cell shown in Figure 3.15.

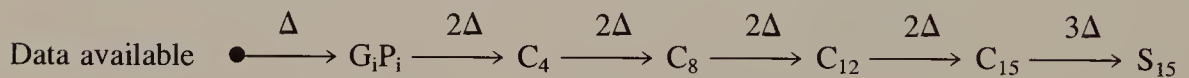
Using this basic cell and 4-bit CLC, the design of a 4-bit CLA can be completed as shown in Figure 3.16.

Using this 4-bit CLA as the building block, a 16-bit adder can be designed, as shown in Figure 3.17.

The worst-case add-time of this adder can be calculated as follows:

	Delay
For $G_i P_i$ generation from $X_i Y_i$ ($0 \leq i \leq 15$)	Δ
To generate C_4 from C_0	2Δ
To generate C_8 from C_4	2Δ
To generate C_{12} from C_8	2Δ
To generate C_{15} from C_{12}	2Δ
To generate S_{15} from C_{15}	3Δ
Total delay	12Δ

A graphical illustration of this calculation can be shown as follows:



From this calculation, it is apparent that the new 16-bit adder is faster than the 16-bit CPA by a factor of 3. In fact, this system can be speeded up further by employing

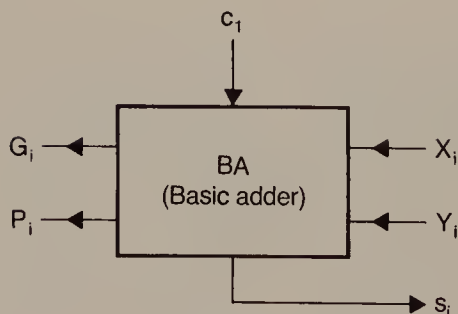


Figure 3.15 Basic CLA Cell

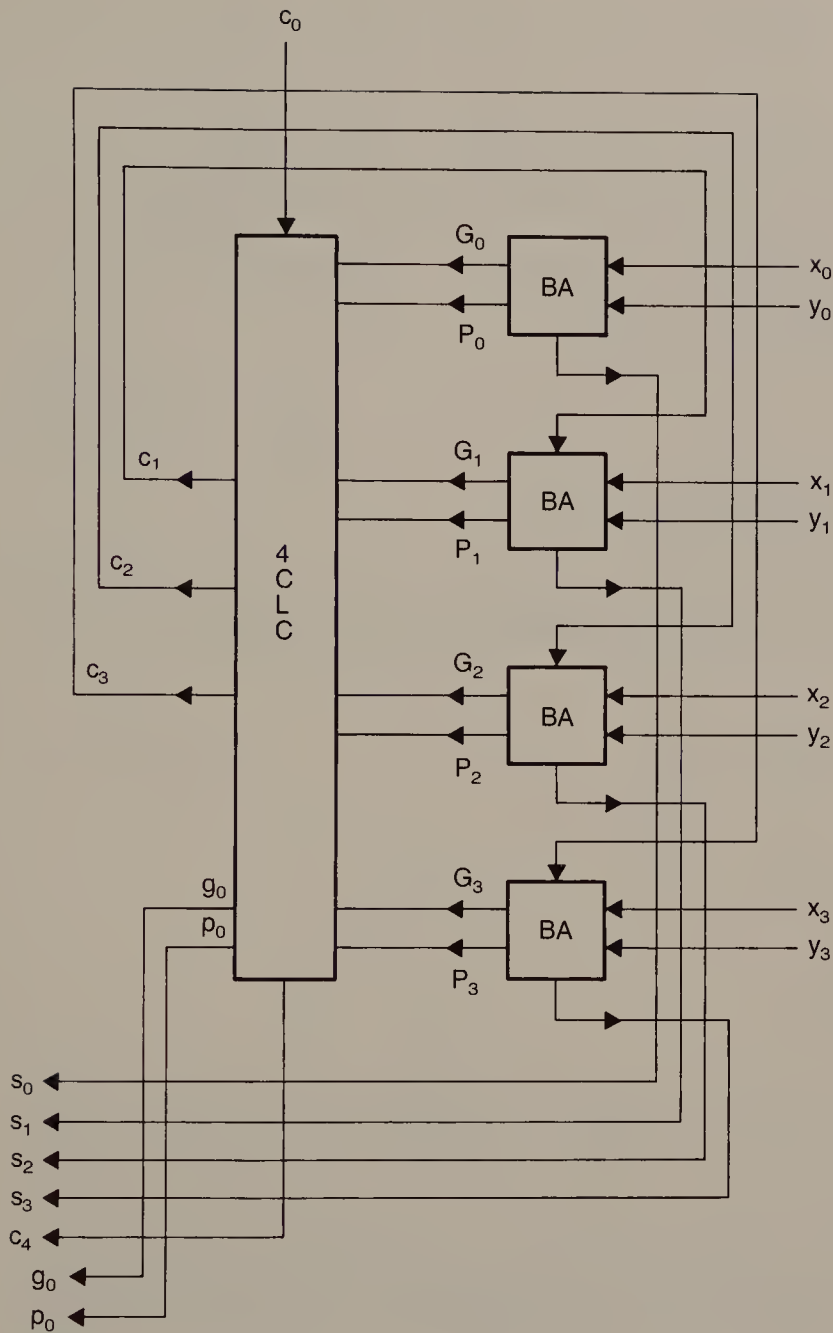


Figure 3.16 A 4-bit CLA

another 4-CLC and eliminating the carry propagation between the 4-bit CLA blocks. For this purpose, the g_i and p_i outputs generated by the 4-bit CLA are used. This design task is left as an exercise to the reader.

It should be emphasized that a two-level realization of a n -CLC requires an $n + 1$ input AND gate and an $n + 1$ input OR gate. Since the maximum fan-in offered by the existing technology is 8, it is not possible to build a two-level CLC when $n + 1 > 8$. For this condition, design of an n -bit adder in terms of 4-bit slice blocks offers an excellent compromise.

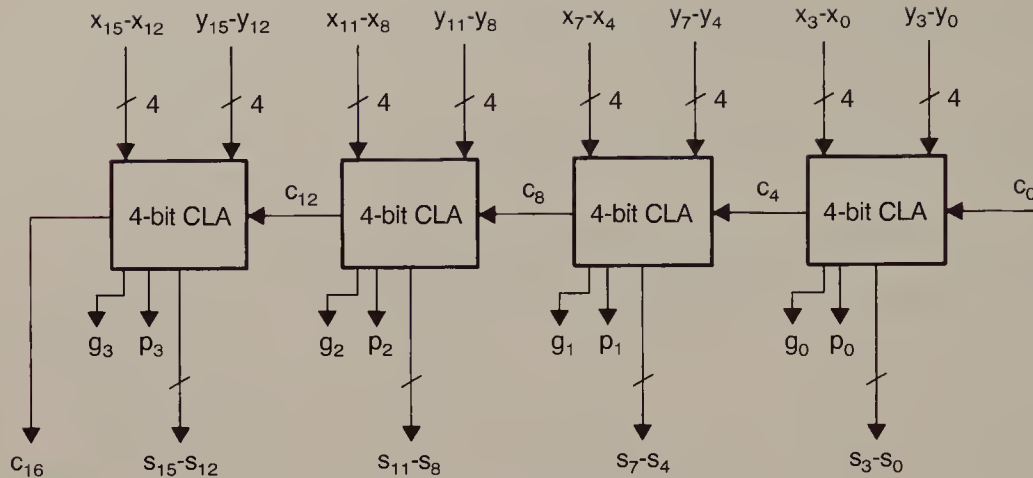


Figure 3.17 Design of 16-bit Adder Using 4-bit CLA's

If there is a need to add more than 3 operands, a technique known as *carry-save addition* is used. To see the effectiveness, consider the following example:

```

44
28
32
79
63 ← Sum vector
12 ← Carry vector
183 ← Final answer

```

In this example, four decimal numbers are added. First, the unit digits are added, and producing a sum of 3 and a carry digit of 2. Similarly, the tens digits are added, producing a sum digit of 6 and a carry digit of 1. Since there is no carry propagation from the unit digit to the tenth digit, these summations can be carried out in parallel to produce a sum vector of 63 and a carry vector of 12. When all operands are exhausted, the sum and the shifted carry vector are added in the conventional manner, which produces the final answer. Note that the carry is propagated only in the last step, which generates the final answer no matter how many operands are added. This concept is also referred to as *addition by deferred carry assimilation*.

The carry-save addition process can be implemented in fully parallel mode or in series-parallel mode. In the first method, all operands are to be added simultaneously. For example, consider the summation:

$$Z = A + B + C + D$$

where A, B, C, and D are 3-bit 2's complement numbers. The block diagram representation of this summation process is shown in Figure 3.18(a).

In this system, the first carry-save adder (CSA) adds A, B, and C and produces a sum vector (S^1) and a carry vector (C^1). The sum vector and the shifted carry vector,

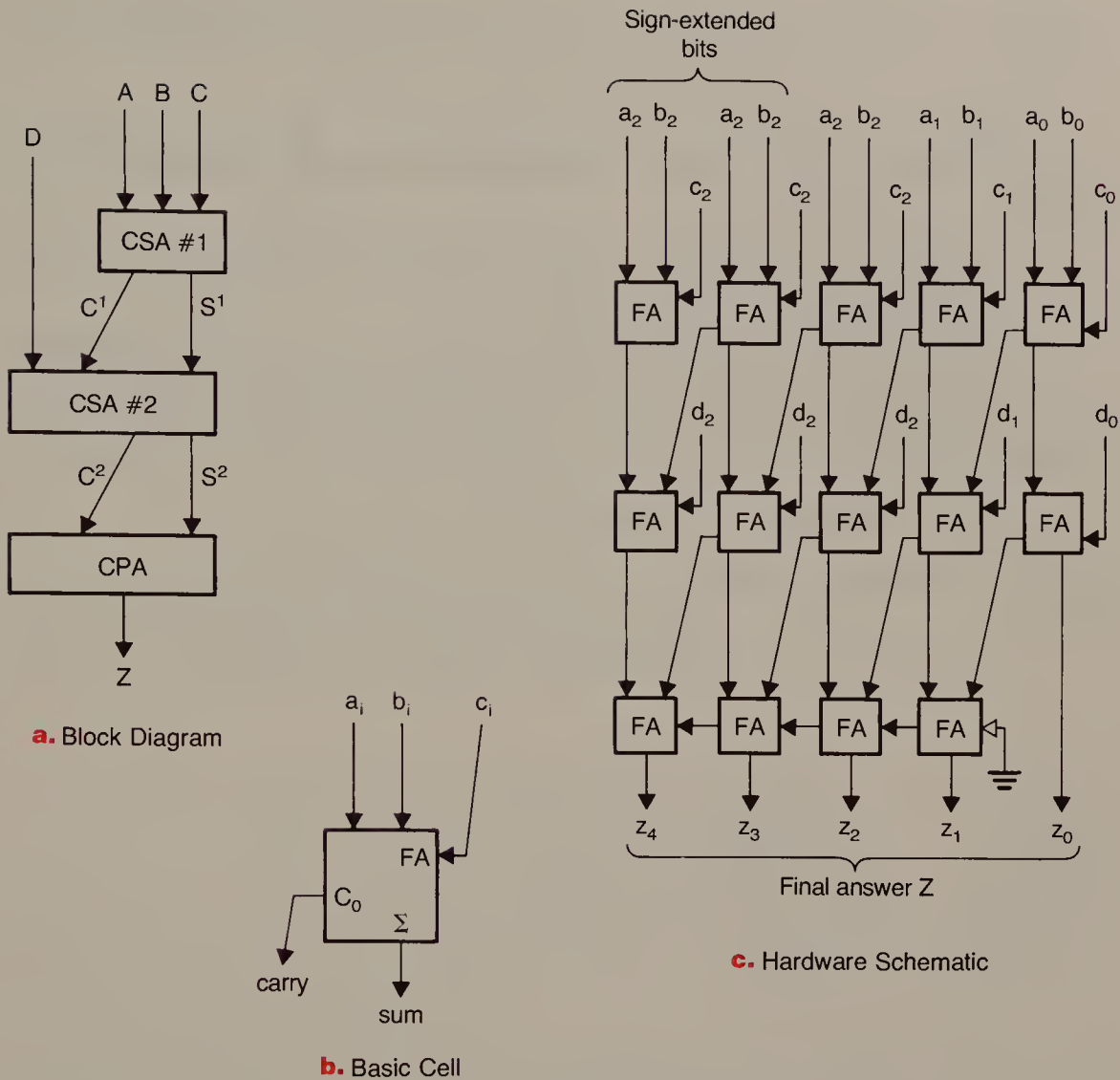


Figure 3.18 5-operand Summation

along with the fourth operand D, are applied as the input to the second CSA. The results produced by the CSA are finally handled by a CPA to generate Z. The carry is propagated only in the last step. So the total time required to add 4 operands is:

$$T(4) = 2 * [\text{CSA add time}] + [\text{CPA time}]$$

In general, n operands can be summed in:

$$T(n) = (n - 2) * [\text{CSA add time}] + [\text{CPA time}]$$

This result can further be improved by using a CLA in the last stage (as opposed to a CPA). Figure 3-18(b) and (c) shows the hardware implementation of the 4-operand summation. When four 3-bit operands are added, the sum, Z, lies in the range of -16 and 12 (with 3 bits we can cover numbers in the range of -4 and 3). Hence, 5 bits are required to represent Z. These extra bits (2 bits in this case) are called *guard bits*, and,

in general an n -operand summation requires $\lceil \log_2 n \rceil$ guard bits. The operator $\lceil \rceil$ is called the ceiling and is formally defined as follows:

$$\lceil X \rceil = Y \quad \text{where } Y \text{ is the least integer such that } Y \geq X$$

To include these guard bits, the given 3-bit operands are sign-extended to obtain a 5-bit data.

The number of CSA stages in a fully parallel multioperand summation can be reduced if the CSAs are organized in the form of a tree. Tree structure corresponding to a 6-operand summation is shown in Figure 3.19.

This structure is a Wallace tree. Basically, a Wallace tree is an interconnection of CSAs that reduces the column of height N to a height of 2. Note that a column of height 2 can be immediately handled by a CPA or CLA. In such a scheme, it can be proven that the total number of CSA levels can be approximated by the following formula:

$$\text{number of CSA levels} = \lceil \log_{1.5}(N/2) \rceil$$

By using a Wallace tree structure, n operands can be summed within $T(n)$ time units, where:

$$T(n) = \lceil \log_{1.5}(n/2) \rceil * (\text{CSA add time}) + (\text{CPA or CLA add time})$$

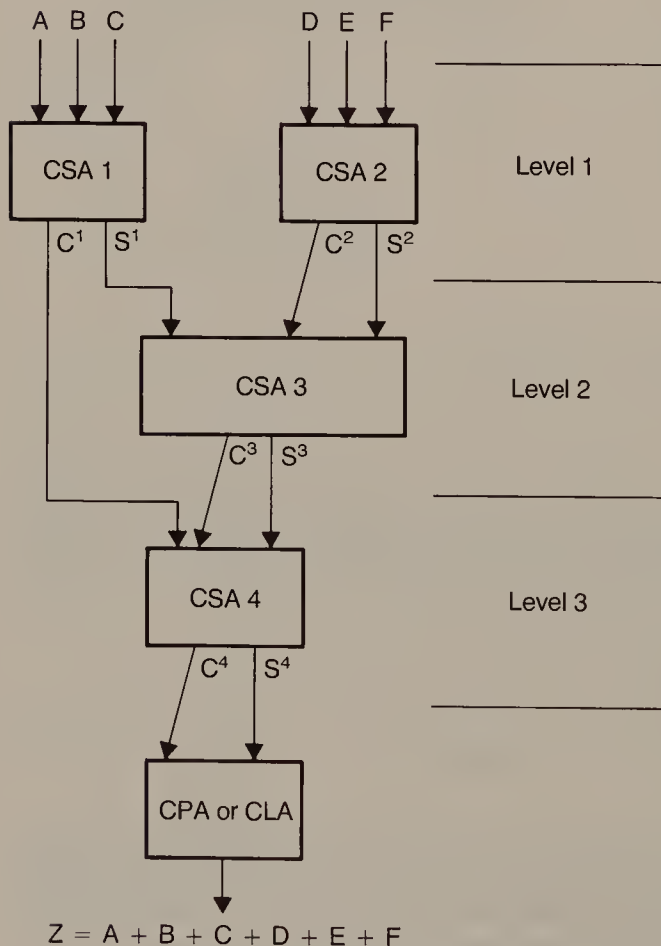


Figure 3.19 A Tree Structure for 6-operand Summation

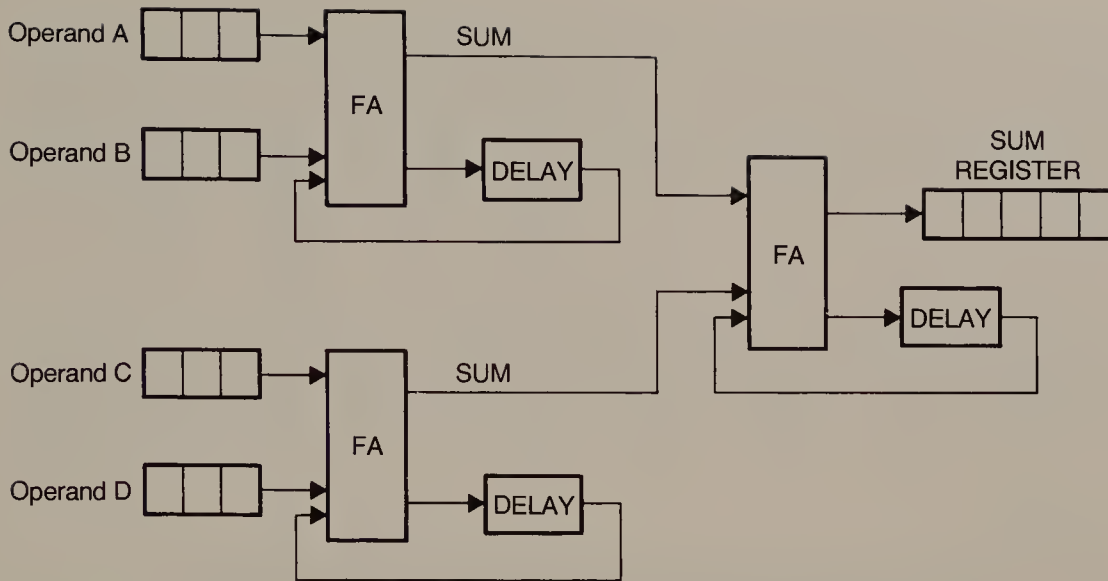


Figure 3.20 Series-parallel Approach to 4-operand Summation (From Norman R. Scott, *Computer Number Systems and Arithmetic*, Prentice-Hall, 1985, p. 66. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

The Wallace tree structure is an essential component in the design of high-speed multipliers. This concept is expanded upon later in this chapter.

Although the fully parallel approach offers great speed, it requires expensive hardware. To reduce the hardware cost, a series-parallel approach is used as a compromise. In this method, the operands are input column by column serially. However, all bits of the same columns are summed in parallel. For example, consider the summation of four 3-bit operands shown in Figure 3.20.

For this hardware, the 3-bit operands are held in shift registers. During the first clock, the first column of all operands (the least-significant bits) are summed by the full adders, and the final sum is shifted into the sum register of the next clock. The delay unit at the output of the FA delays the carry output by one clock period, causing the carry to be added to the next column sum. Although the hardware produces the final answer in a serial fashion, it is still well suited for pipelining. The pipeline concept is a method for obtaining a speedup via overlapped execution of two or more operations. This is discussed in a later chapter. Suppose that there are n operands. If $n = 2k$, for some $k > 1$, the series-parallel approach will require $(n - 1)$ adders. If there are m bits per operand, the entire operation can be completed within a time T , where:

$$T \equiv m * (\text{clock period}) + (\log_2 k) * (\text{full adder time})$$

The task of verifying these results is left as an exercise.

It was mentioned earlier that $X - Y$ is the same as $X + 2$'s complement of $Y = X + y^1 + 1$. The hardware that implements this concept is shown in Figure 3.21.

The select input controls the input carry (C_0) and the left input of each full adder are as follows for this configuration:

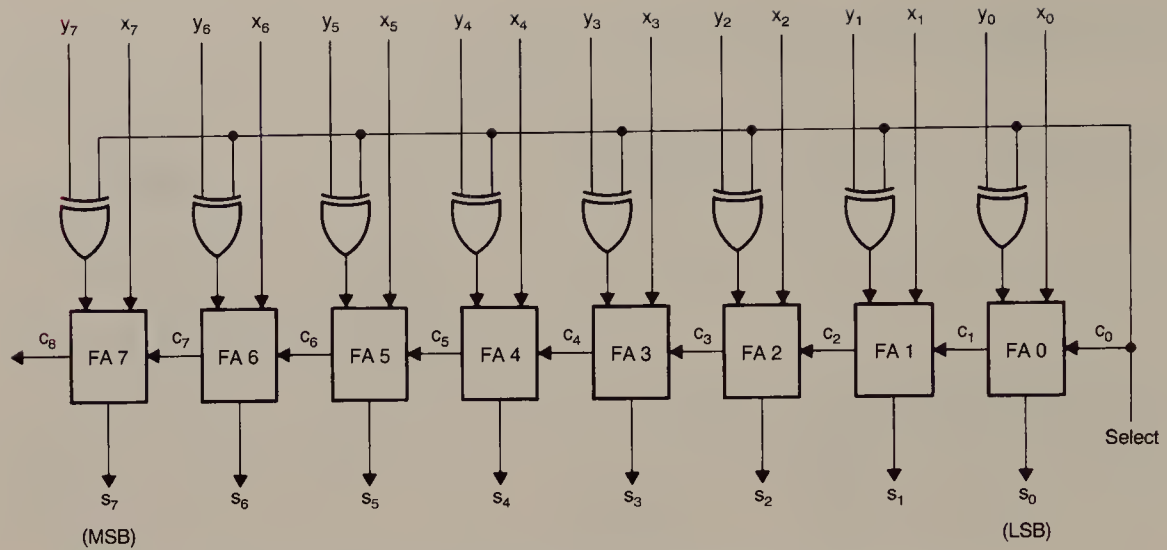


Figure 3.21 Eight-bit Adder/Subtractor (M. Morris Mano, *Digital Logic and Computer Design* © 1979 p. 375. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

- If select = 0, then $C_0 = 0$, and Y_i is routed to the left input of the full adder FA_i (because $0 \oplus Y_i = Y_i$) for $i = 0, 1, \dots, 7$.
- If select = 1, then $C_0 = 1$, and the complement of Y_i is routed to the left input of the full adder, FA_i , (because $1 \oplus Y_i = Y_i'$) for $i = 0, 1, \dots, 7$.

It is clear that when the select input is zero, output S is the sum of X and Y . Yet, when the select input is one, the output S is:

$$\begin{aligned}
 S &= X + Y' + 1 \\
 &= X + 2\text{'s complement of } Y \\
 &\triangleq X - Y
 \end{aligned}$$

In conclusion, the circuit can be configured as an adder or subtractor by controlling the select input.

3.4 ALU DESIGN

Functionally, an ALU can be divided into two segments: the arithmetic unit and the logic unit. The arithmetic unit performs typical arithmetic operations such as addition, subtraction, and increment or decrement by 1. Usually, the operands involved may be signed or unsigned integers. In some cases, however, an arithmetic unit must handle 4-bit binary coded decimal (BCD) numbers and floating-point numbers. Therefore, this unit must include necessary electronics to manipulate these data types. As the name implies, the logic unit contains hardware elements that perform typical operations such as Boolean NOT and OR. In this section, the design of a simple ALU using typical combinational elements such as gates, multiplexers, and a 4-bit parallel adder is discussed. For this approach, first an arithmetic unit and a logic unit are designed separately; then they are combined to obtain an ALU.

For the first step, a two-function arithmetic unit, as shown in Figure 3.22, is designed. The key element of this system is a 4-bit parallel adder. The multiplexers select either Y or Y' for the 3-input of the parallel adder. In particular, if $S_0 = 0$, then $B = Y$; otherwise $B = Y'$. Since the selection input (S_0) also controls the input carry (C_{in}), the following results:

$$\begin{aligned} \text{If } s_0 = 0 \quad & \text{then} \quad F = X \text{ plus } Y \\ & \text{else} \quad F = X \text{ plus } Y' \text{ plus } 1 \\ & \quad = X \text{ minus } Y \end{aligned}$$

The operation of this circuit is identical to the adder/subtractor circuit discussed in the previous section.

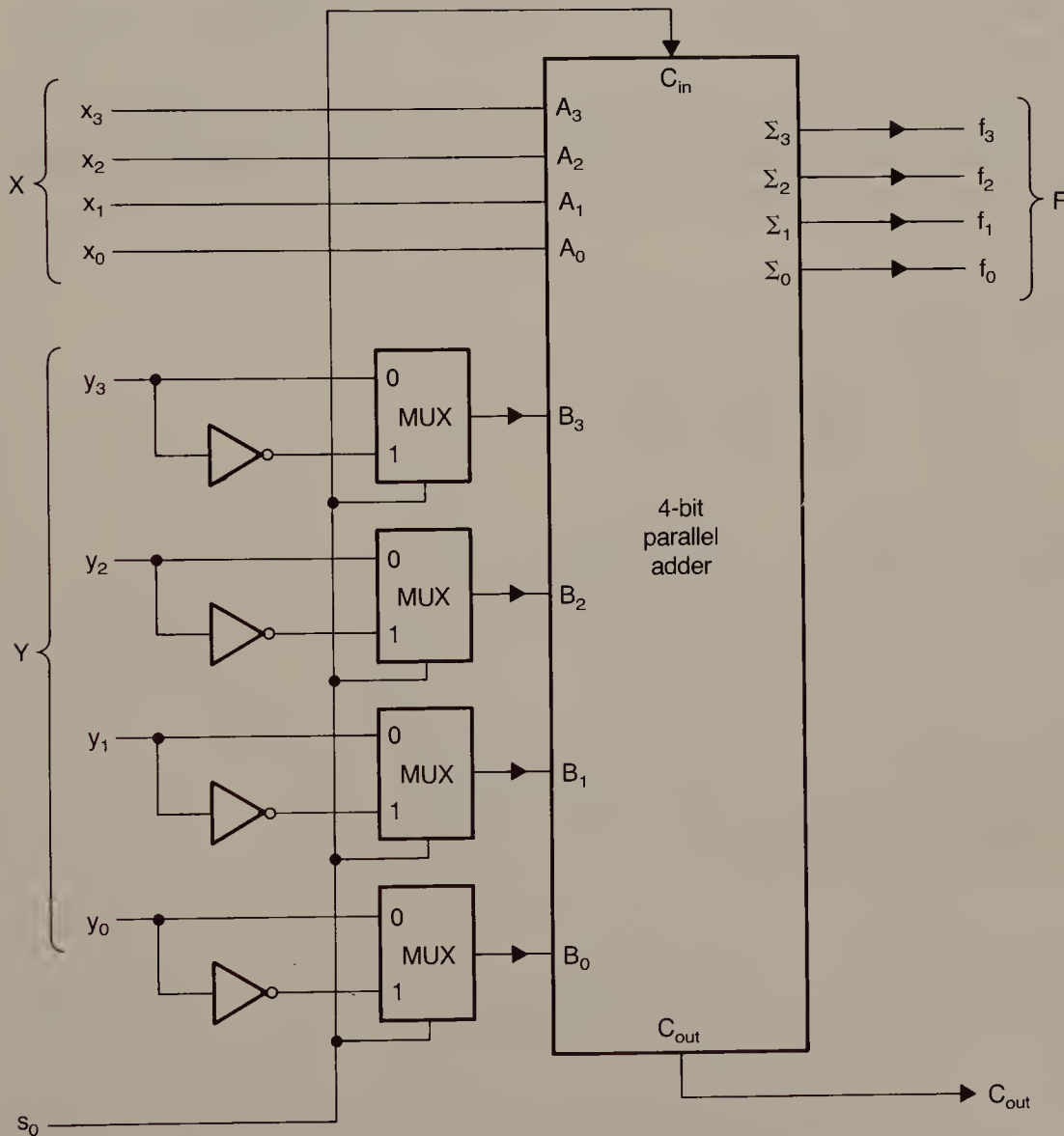


Figure 3.22 Organization of an Arithmetic Unit

For the second step, let us design a two-function logic unit; this is shown in Figure 3.23.

From Figure 3.23, it can be seen that when $s_0 = 0$, the output $G = X \text{ AND } Y$; otherwise the output $G = X \oplus Y$. Note that from these two Boolean operations, other operations such as NOT and OR can be derived by the following Boolean identities:

$$1 \oplus x = x'$$

$$x \text{ OR } y = x \oplus y \oplus xy$$

Therefore, NOT, and OR operations can be obtained by using additional hardware and the circuit of Figure 3.23. The outputs generated by the arithmetic and logic units can be combined by using a set of four multiplexers, as shown in Figure 3.24.

From this organization it can be seen that when the select line $s_1 = 1$, the multiplexers select outputs generated by the logic unit; otherwise, the outputs of arithmetic unit are selected.

More commonly, the select line, s_1 , is referred to as the *mode input* since it selects the desired mode of operation (arithmetic or logic). A complete block diagram schematic of this ALU is shown in Figure 3.25.

The truth table illustrating the operation of this ALU is shown in Figure 3.26.

This table shows that this ALU is capable of performing 2 arithmetic and 2 logic operations on the 4-bit operands X and Y.

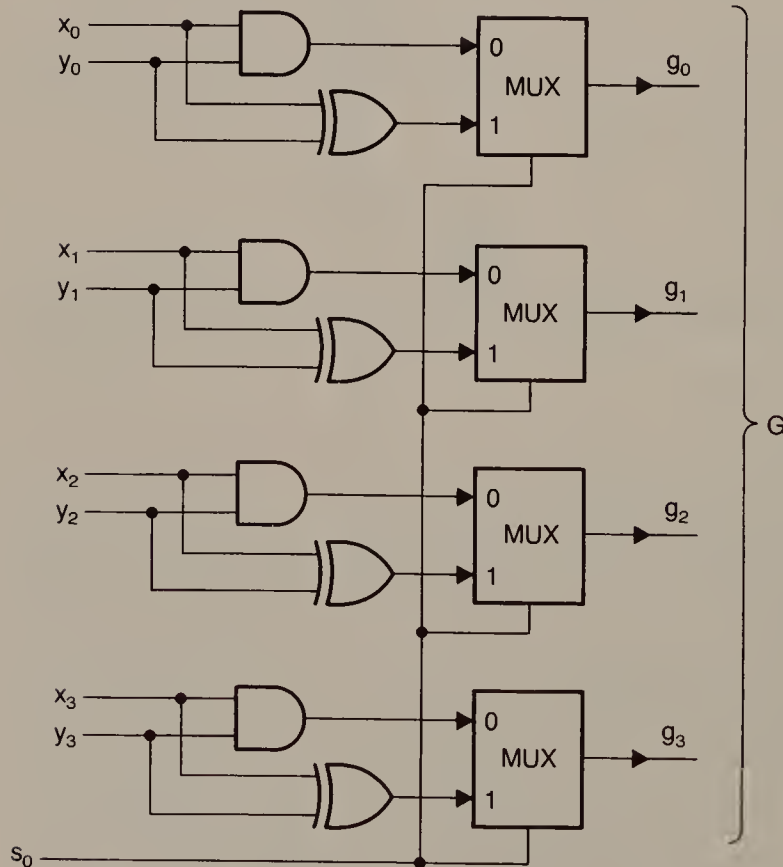


Figure 3.23 Organization of a 4-bit Two-function Logic Unit

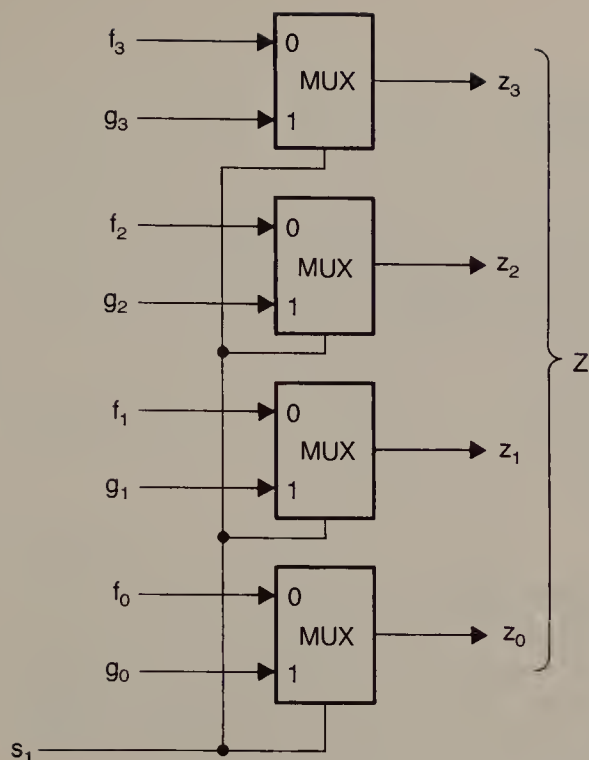


Figure 3.24 Combining the Outputs Generated by the Arithmetic and Logic Units

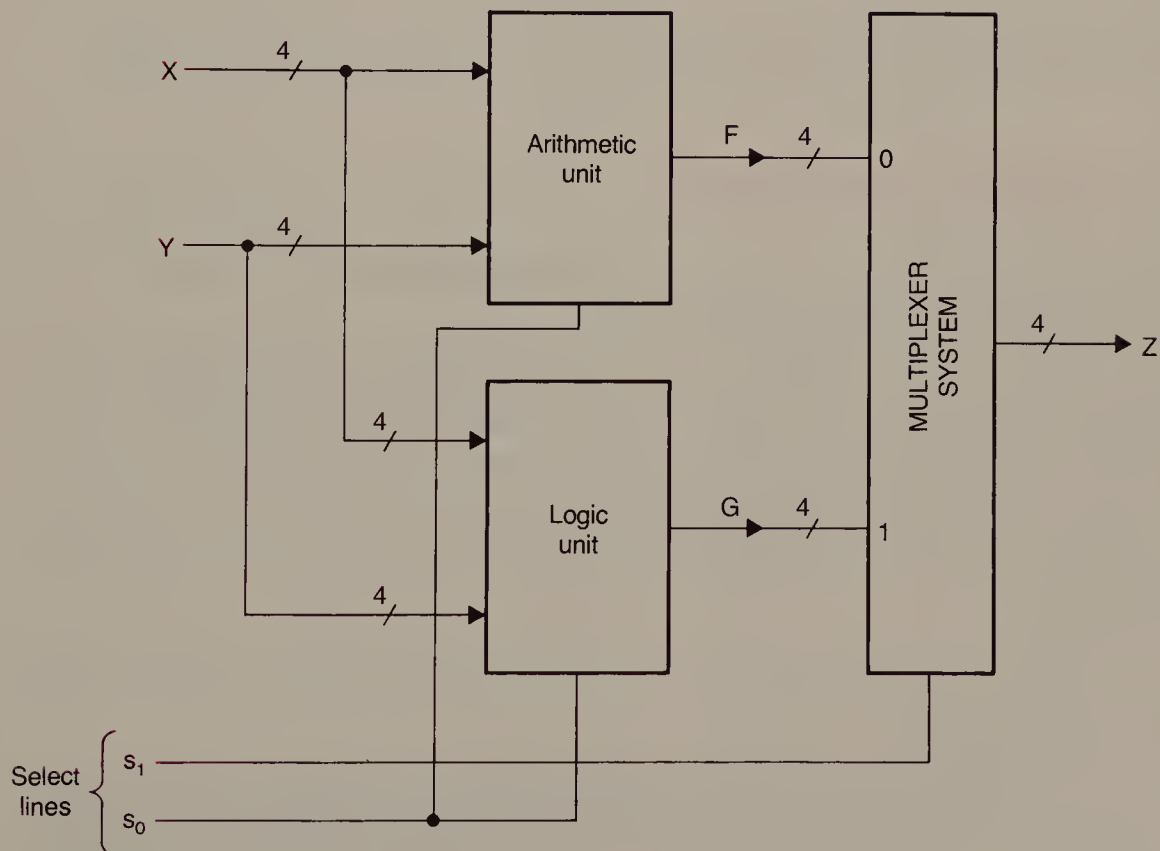


Figure 3.25 Schematic Representation of the Four-functions ALU (M. Morris Mano, *Digital Logic and Computer Design* © 1979 p. 377. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

Select Lines		Output Z	Comment
s_1	s_0		
0	0	X plus Y	Addition
0	1	X plus Y plus 1	2's complement subtraction
1	0	$X \wedge Y$	Boolean AND
1	1	$X \oplus Y$	Exclusive-OR

Figure 3.26 Truth Table Controlling the Operation of the ALU of Figure 3.25

The rapid growth in IC technology permitted the manufacturers to produce an ALU as an MSI block. Such systems implement many operations, and their use as a system component reduces the hardware cost, board space, debugging effort, and failure rate. Usually, each MSI ALU chip is designed as a 4-bit slice. However, a designer can easily interconnect n such chips to get a $4n$ -bit ALU. The most popular 4-bit ALU chips are the 74381 and 74181. The 74381 ALU performs 3 arithmetic, 3 Boolean, and 2 miscellaneous operations on 4-bit operands. The 74181 ALU performs 16 arithmetic and 16 Boolean operations on two 4-bit operands, using either active high or active low data.

A complete description and operational characteristics of these devices may be found in the original-equipment manufacturer's data books listed in the reference.

3.5 BIT-SLICE PROCESSOR

The chip count, which results in high power dissipation and poor reliability, are the main disadvantages of MSI functional blocks.

To eliminate these difficulties, a new approach called the *bit-slice technique* became feasible because of advances in IC technology. A bit-slice processor chip typically includes an ALU and a few registers. Using bit slices, a designer can build a processor of any word length. For example, a 4-bit slice processor includes a register file and an ALU for performing operations on 4-bit data, so that four such chips can be combined to build a 16-bit processor unit. In this section, we discuss the architecture and operational characteristics of a popular bit-slice processor—the Advanced Micro-Device's AM 2901.

The block diagram schematic of the AM 2901, 4-bit processor is shown in Figure 3.27. The key elements are the 16×4 two-port RAM, the high-speed ALU, and the accumulator register Q.

The RAM has two ports: Port A and Port B. Port A is a read-only port, and Port B may be used as a read/write port. Data from Ports A and B can be simultaneously read by specifying the appropriate addresses on the A and B address lines, though new data is always written into the RAM word whose address is specified by the B-address line.

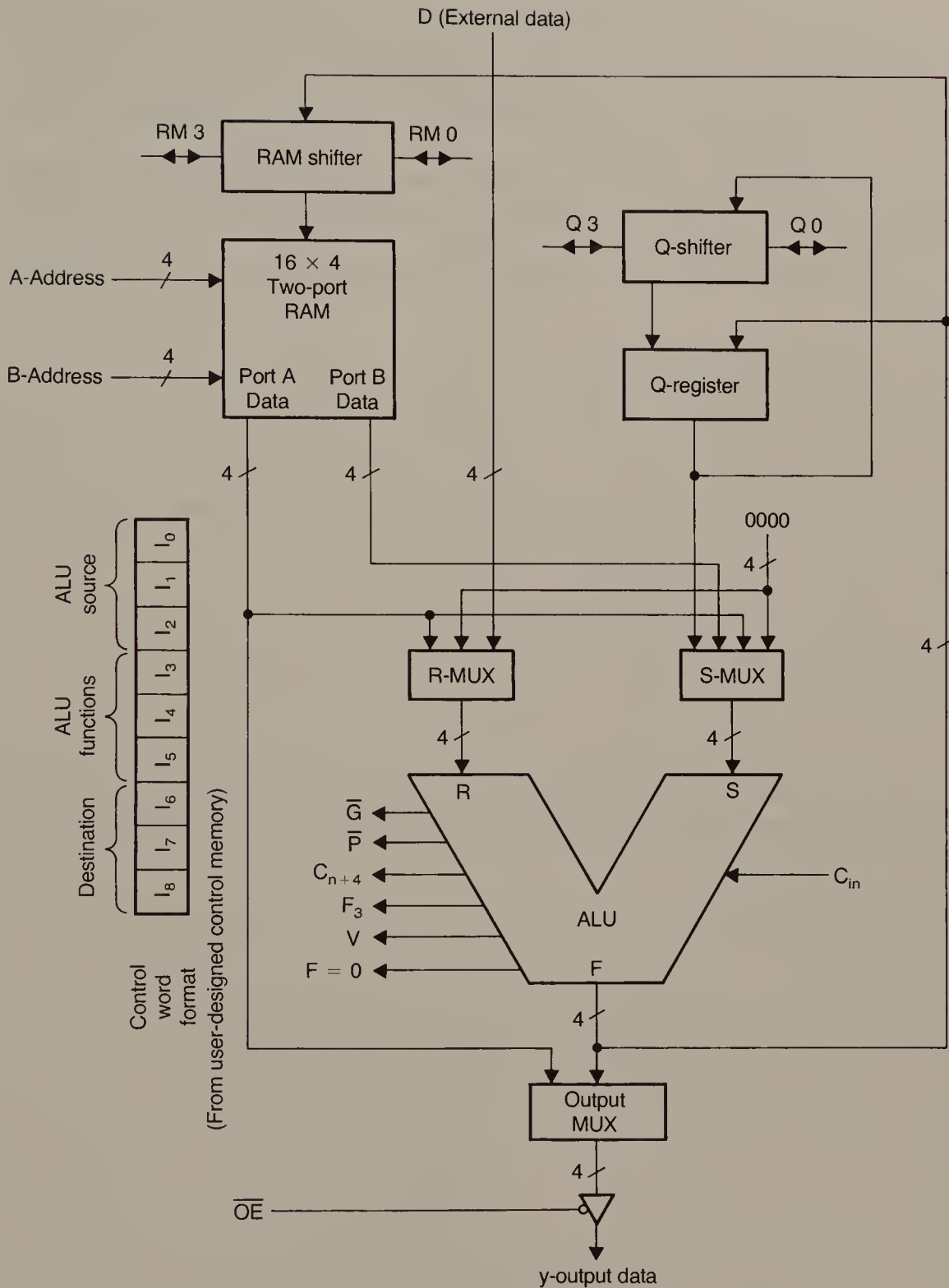


Figure 3.27 Block Diagram Schematic of the AM 2901 Bit-slice Processor (From J. P. Hayes, *Digital System Design and Microprocessors*, McGraw-Hill, 1984, p. 702. Reprinted by permission of McGraw-Hill)

The ALU performs 3 arithmetic and 5 Boolean operations. The entire operation of the bit slice is controlled by the 9-bit control word (I_8 through I_0). These control words are fetched from the control memory designed by the user (see Chapter 5). Three bits, I_5 , I_4 , and I_3 , are used to select the specific ALU operation. The interpretation of this 3-bit field is shown in Figure 3.28.

The ALU has two inputs—R and S. By examining Figure 3.27 it can be seen that the data for R and S inputs are selected by the multiplexers R-MUX and S-MUX, respectively.

The R-MUX selects one of the following three sources for the R input of the ALU:

- Output of the RAM-port A
- A zero value, which is 0000
- Four-bit external data D

The S-MUX selects one of the following four sources for the S input of the ALU:

- Output of the accumulator Q
- Output of the RAM Port B
- Output of the RAM Port A
- A zero value

This discussion suggests that there will be 12 distinct input pairs for the ALU. The 2901 designers have chosen only 8 of them, however. Each pair is selected using the bits I_2 , I_1 , and I_0 of the control word. The interpretation of this 3-bit field (source field) is summarized in Figure 3.29.

ALU Function Field			Operation Performed	Remarks
I_5	I_4	I_3		
0	0	0	R plus S plus C_{in}	Addition with a carry
0	0	1	S minus R minus C_{in}'	Subtraction with a borrow
0	1	0	R minus S minus C_{in}'	Subtraction with a borrow
0	1	1	$R + S$	Boolean OR
1	0	0	$R \cdot S$	Boolean AND
1	0	1	$R' \cdot S$	NOT (R) AND S
1	1	0	$R \oplus S$	Exclusive-OR
1	1	1	$(R \oplus S)'$	Exclusive-NOR

Figure 3.28 Interpretation of the ALU Function Field

ALU Source Field			Source Input Pair	
I_2	I_1	I_0	ALU input R	ALU input S
0	0	0	RAM port A	Q register
0	0	1	RAM port A	RAM port B
0	1	0	0000	Q register
0	1	1	0000	RAM port B
1	0	0	0000	RAM port A
1	0	1	External input D	RAM port A
1	1	0	External input D	Q register
1	1	1	External input D	0000

Figure 3.29 Interpretation of the ALU Source Field of the Control Word

In this system, there are three destination points, as follows:

- External output bus Y: This output, derived using output MUX, selects either ALU output F or RAM Port A. From Figure 4.27 it is shown that output y is a tristated output. This bus is in the high-impedance state when input $\overline{OE}$ is held high.
- Q register: This register obtains its input from either the ALU output or the Q-shifter. This is a combinational shifter that allows one to compute $Q/2$ or $2Q$ and has two bidirectional lines (Q3 and Q0) for the purpose of expansion.
- Two-port RAM: The output generated by the ALU is routed to the two-port RAM via the RAM shifter. This shifter is identical to the Q-shifter and it allows F, $F/2$, or $2F$ to be saved into the word address specified on the B-address line.

The destination points are selected using the bits I_8 , I_7 , and I_6 of the control word. A complete interpretation of this 3-bit field is summarized in Figure 3.30.

The AM 2901 chip generates the following status outputs:

C_{n+4}	:	End around carry
$\overline{G}$	:	Carry generate function
$\overline{P}$	:	Carry propagate function
F_3	:	Most-significant bit of the ALU (sign of the results)
Z	:	Zero status output (This output will be 1 if $F = 0$.)
V	:	Overflow status output

These outputs may be used to design the condition code register. The following examples demonstrate the versatility of the AM 2901 device.

Destination Field			Values Appear (or stored) in Different Destinations		
I ₈	I ₇	I ₆	Y	Two-port RAM	Q Register
0	0	0	F	None	F
0	0	1	F	None	None
0	1	0	RAM port-A	F	None
0	1	1	F	F	None
1	0	0	F	F/2	Q/2
1	0	1	F	F/2	None
1	1	0	F	2F	2Q
1	1	1	F	2F	None

Figure 3.30 Interpretation of the Destination-control Field of the AM 2901 Control Word

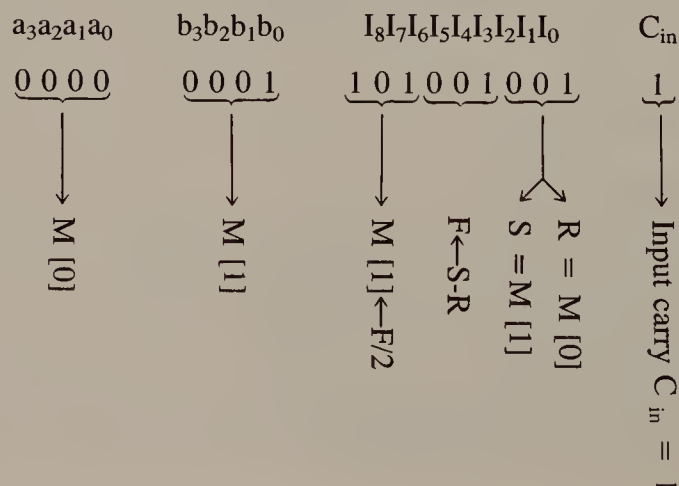
In this discussion, it is assumed that the complete control word is organized as follows:

A-ADDRESS	B-ADDRESS	DESTINATION FIELD	ALU FUNCTION	SOURCE FIELD	CARRY INPUT
a ₃ a ₂ a ₁ a ₀	b ₃ b ₂ b ₁ b ₀	I ₈ I ₇ I ₆	I ₅ I ₄ I ₃	I ₂ I ₁ I ₀	C _{in}

Using this format, the control words corresponding to some typical microoperations are written. For the first example, consider the following microoperation. (an operation completed during one clock cycle is called a *microoperation*):

$$M[1] \leftarrow (M[1] \text{ minus } M[0]) / 2;$$

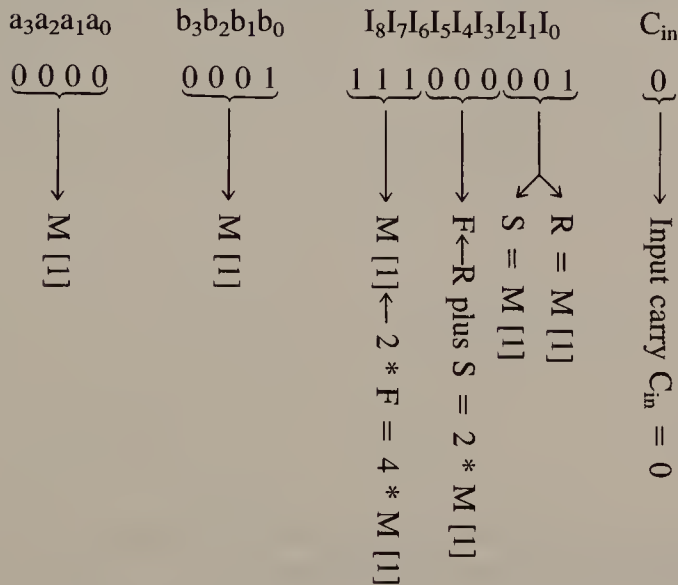
The symbol $M[i]$ denotes the RAM word whose address is i . The control word corresponding to this operation is shown next:



For a second example, here is another microoperation:

$$M[1] \leftarrow 4 * M[1]$$

The control word for this operation is shown next:



In both the examples, the ALU output F also appears at the external output bus Y , though this is discarded.

In general, one can obtain a 4K-bit processor by simply cascading K AM 2901s. In Figure 3.31 three AM 2901s are cascaded to form a 12-bit processing unit.

Some technical points relevant to this interconnection are summarized next:

- For all devices, except for the most significant, the bidirectional shift lines, Q_3 and RM_3 , are connected, respectively, to the bidirectional shift lines Q_0 and RM_0 of the more-significant devices.
- Since the $F = 0$ output is an open-collector output, the overall zero status output is obtained by wire ANDing the $F = 0$ output of each device.
- The final sign-status output is derived by using the F_3 output of the most significant device.
- The end-around carry status of the 12-bit system is the same as the C_{n+4} output of the most-significant device.

Finally, the cost of such a simple hardware configuration is the reduction in the operating speed, since a ripple-carry scheme is used to improve speed. The AM 2902, a 4-bit look-ahead-carry generator chip, may be used to increase the operating speed.

A complete discussion of this technique is beyond the scope of this chapter.

The AM 2901C and AM 2901C-1 are variations of the AM 2901 bit-slice RALU discussed in this section. The AM 2901C is a plug-in replacement for earlier versions of the AM 2901 but is considerably faster than its predecessors. The AM 2901C is a speed selected version of the AM 2901C, offering a 30% speed improvement on critical paths.

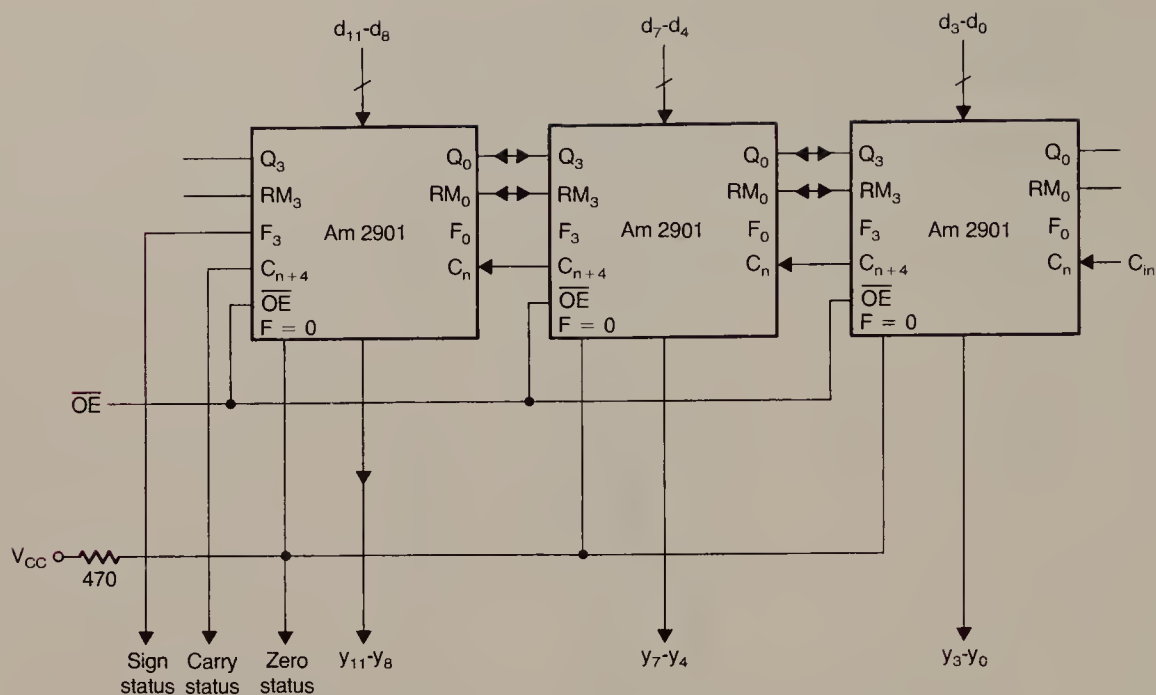


Figure 3.31 Cascaded Interconnection of Three AM 2901s (Copyright © Advanced Micro Devices, Inc., 1982. Reprinted by permission of copyright owner)

Another upgrade for the Am 2901 is the AM 2903 bit-slide device. The AM 2903 is a 4-bit expandable bipolar bit-slide ALU, which provides all functions performed by the AM 2901 and a number of enhancements useful in arithmetic-oriented processors, including a larger instruction set with nine special functions that use internal controls to facilitate operations such as 2's complement multiply, sign-extend, and normalization.

3.6 MULTIPLICATION OF UNSIGNED AND SIGNED INTEGERS

In a typical 8-bit processor such as the Intel 8085, the processor hardware does not include any provision to multiply two numbers, due to a limitation on the component density. Hence, the processor's instruction set does not include a multiplication instruction. In this situation, however, one can write a program that multiplies two numbers. Although this solution seems viable, the operational speed is unsatisfactory.

For application environments such as real-time digital filtering, in which the processor is expected to perform 32 to 64 eight-bit multiplication operations within 100 μ sec (sampling frequency = 10 KHZ), speed is an important factor.

New device technology such as high-density metal oxide semiconductors (HMOS) allows manufacturers to pack nearly a quarter of a million devices per chip. Consequently, state-of-the-art 32-bit microprocessors such as the MC68020 and I 80386 designed using this technology, have a larger instruction set than their predecessors, in-

cluding multiplication and division. In this section, multiplier design principles are discussed.

Two unsigned integers can be multiplied the same way as two decimal numbers are by hand. Consider the multiplication of two unsigned integers, where the multiplier $Q = 15$ and the multiplicand is $M = 14$, as illustrated:

$M \longrightarrow 1110 \longrightarrow$ Multiplicand (14_{10})

$Q \longrightarrow 1111 \longrightarrow$ Multiplier (15_{10})

```

      1110 ←
      1110 ←
    1110 ← Partial products
    1110 ←
  
```

$P \longrightarrow 11010010 \longleftarrow$ Final product

In the paper and pencil algorithm, shifted versions of multiplicands are added. This procedure can be implemented by using combinational circuit elements such as AND gates and FULL adders.

Generally, a 4-bit unsigned multiplier Q and a 4-bit unsigned multiplicand M can be written as

$M: m_3m_2m_1m_0$

$Q: q_3q_2q_1q_0$

The process of generating the partial products and the final product can also be generalized as shown in Figure 3.32.

Each cross-product term (m_iq_j) in this figure can be generated using an AND gate. This requires 16 AND gates to generate all cross-product terms that are summed by full adder arrays, as shown in Figure 3.33(b).

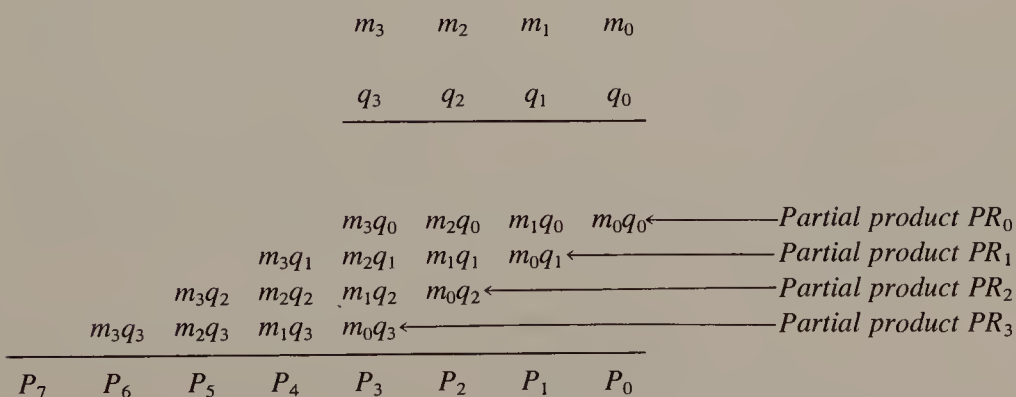


Figure 3.32 Generalized Version of the Multiplication of Two 4-bit Numbers Using the Paper and Pencil Algorithm

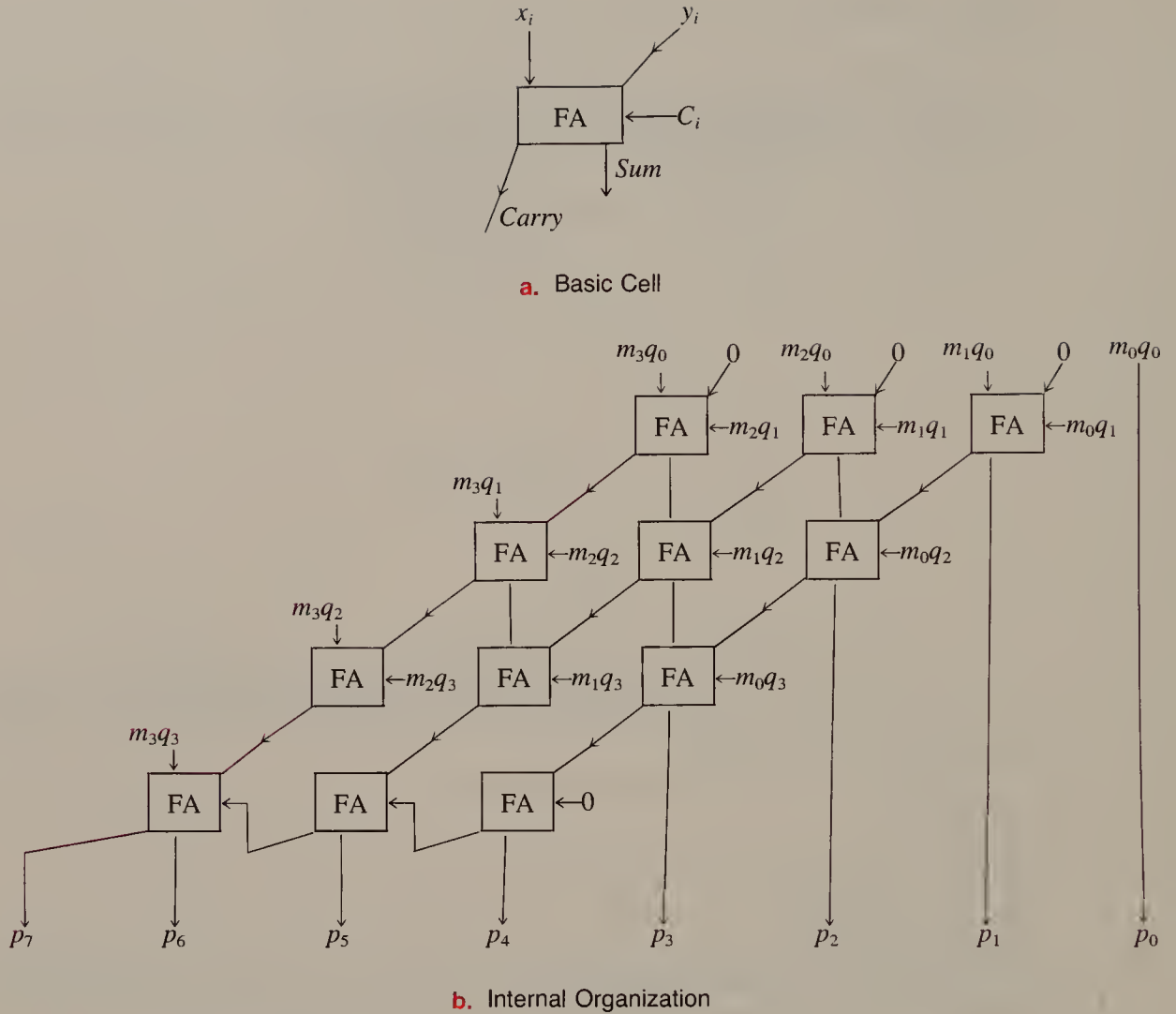


Figure 3.33 4×4 Array Multiplier

Consider the generation of p_2 in Figure 3.33(b). From Figure 3.32, p_2 is the sum of m_2q_0 , m_1q_1 and m_0q_2 . The sum of these three elements is obtained by using two full adders. (See column for p_2 in Figure 3.33(b)). The top full-adder in this column generates the sum $m_2q_0 + m_1q_1$. This sum is then added to m_0q_2 by the bottom full-adder along with any carry from the previous full-adder for p_1 .

The time required to complete the multiplication can be estimated by considering the longest carry propagation path comprising of the rightmost diagonal (which includes the full-adder for p_1 and the bottom full-adders for p_2 and p_5), and the last row (which includes the full-adder for p_6 and the bottom full-adders for p_4 and p_5). The time taken to multiply two n -bit numbers can be expressed as follows:

$$T(n) = \Delta_{\text{AND gate}} + (n - 1)\Delta_{\text{carry propagation}} + (n - 1)\Delta_{\text{carry propagation}}$$

In this equation, all cross-product terms m_iq_i can be generated simultaneously by an array of AND gates. Therefore, only one AND gate delay is included in the equation.

Also, the rightmost diagonal and the bottom row contain $(n - 1)$ full-adders each for the $n \times n$ multiplier.

Assuming that $\Delta_{\text{AND gate}} = \Delta_{\text{carry propagation}} = 2 \text{ gate delays} = 2\Delta$, the preceding expression can be simplified as shown:

$$\begin{aligned} T(n) &= 2\Delta + (2n - 2)2\Delta \\ &= (4n - 2)\Delta \end{aligned}$$

The array multiplier that has been considered so far is known as Braun's multiplier. The hardware is often called a *nonadditive multiplier* (NM), since it does not include any additive inputs. An *additive multiplier* (AM) includes an extra input R; it computes products of the form

$$P = M * Q + R$$

This type of multiplier is useful in computing the sum of products of the form $\sum X_i Y_i$.

Both an NM and an AM are available as standard IC blocks. Since these systems require more components, they are available only to handle 4- or 8-bit operands.

Alternatively, the same 4×4 NM discussed earlier can be obtained using a 256×8 ROM as shown in Figure 3.34.

It can be seen that a given MQ pair defines a ROM address, where the corresponding 8-bit product is held. The ROM approach is attractive for small-scale multipliers because:

- The technological advancements allow the manufacturers to produce low-cost ROMs.
- The design effort is minimum.

In case of large multipliers, ROM implementation is unfeasible, since large-size ROMs are required. For example—in order to implement an 8×8 multiplier, a $2^{16} \times 16$ ROM is required. If the required 8×8 product is decomposed into a linear combination of four 4×4 products, an 8×8 multiplier can be implemented using four 256×8 ROMs and a few 4-bit parallel adders. This construction is left as an exercise.

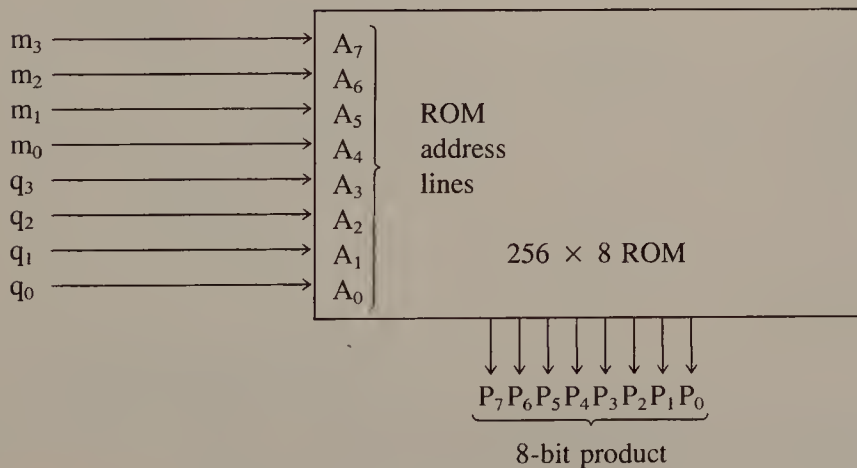
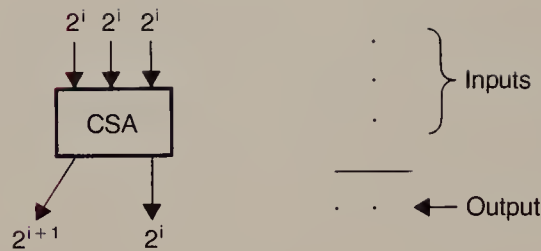


Figure 3.34 ROM-based 4×4 Multiplier



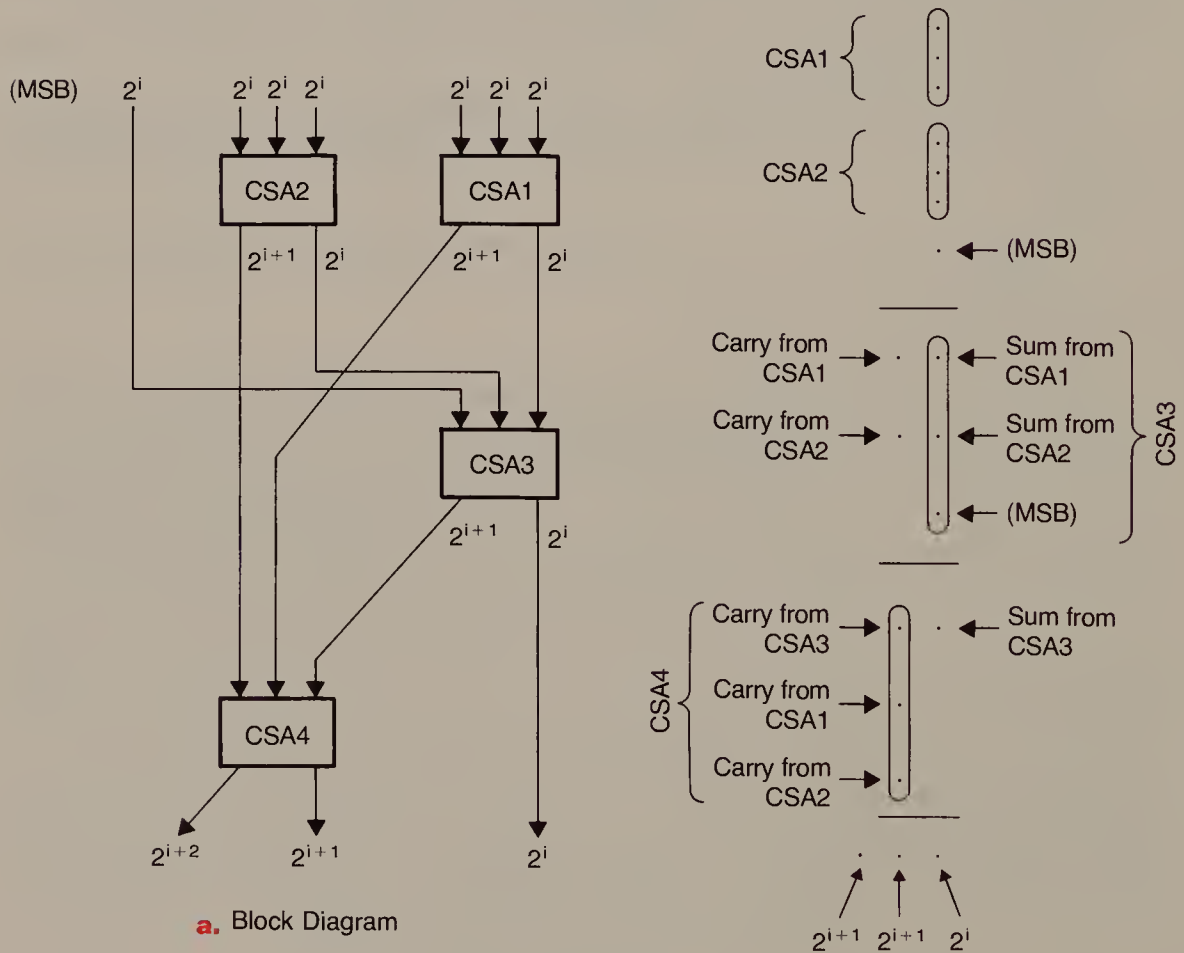
a. Block Diagram

b. Dot Pattern

Figure 3.35 Three-input Wallace Tree

A fast 8×8 multiplier can be designed by using 4×4 NMs and Wallace tree. Recall that a Wallace tree is an interconnection of CSAs that reduces a column height N to less than 2. A 3-input Wallace tree and the associated dot pattern are shown in Figure 3.35.

The 3-input Wallace tree is implemented by using a CSA. The dot pattern shows



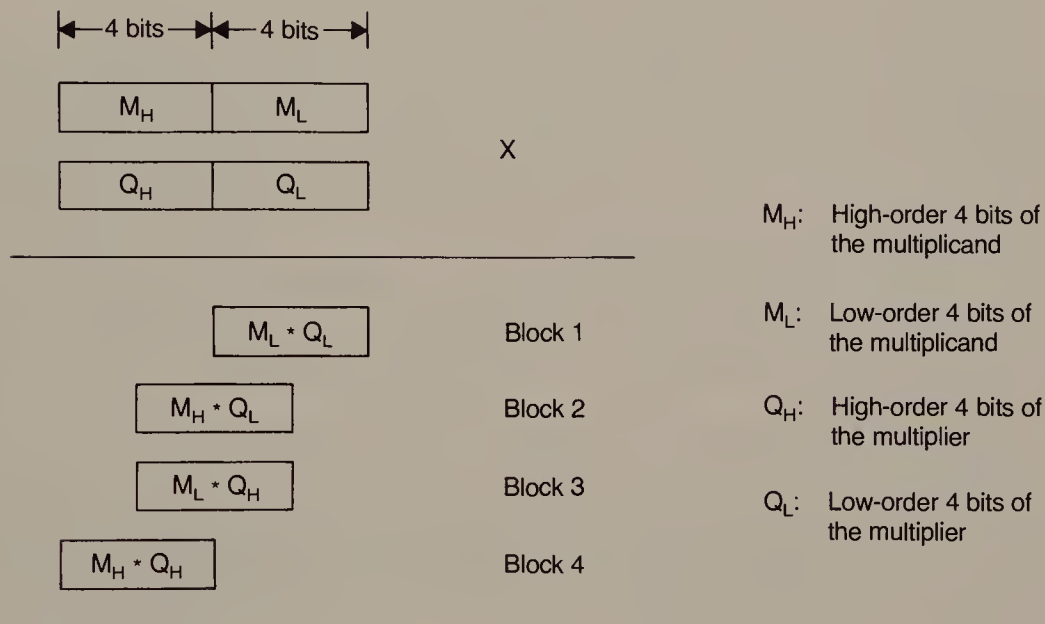
a. Block Diagram

b. Dot Pattern

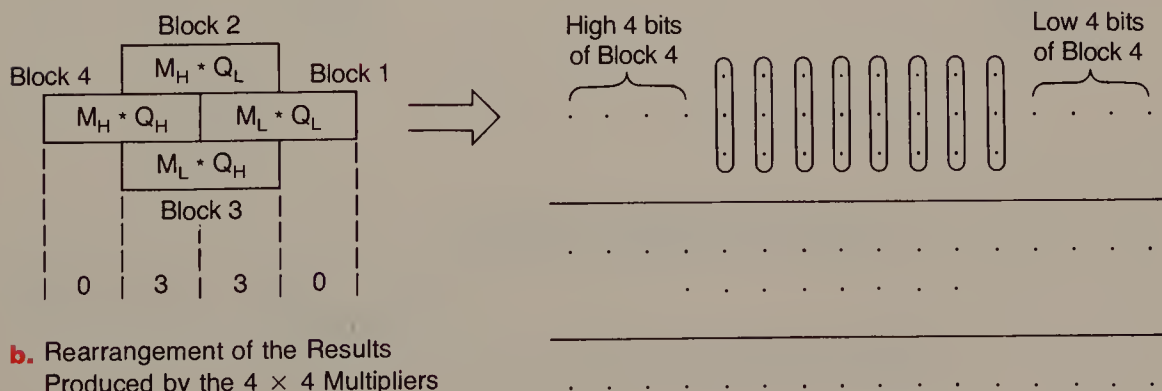
Figure 3.36 Seven-input Wallace Tree (From Norman R. Scott, *Computer Number System and Arithmetic*, Prentice-Hall, 1985, pp. 102–103. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

that this module actually reduces a column of height 3 to a height of 1. It is possible to design a 7-input Wallace tree using the 3-input Wallace tree as a building block. This process is shown in Figure 3.36.

With these Wallace tree adders and the 4×4 NMs, a fully combinational 8×8 or 16×16 multiplier can be constructed that works at great speed. Consider the design of an 8×8 multiplier. In this case, first divide the multiplicand M and multiplier Q into 4-bit groups (M_H , M_L , Q_H , Q_L) as indicated in Figure 3.37(a). Then use four 4×4 NMs to evaluate the products $M_L * Q_L$, $M_H * Q_L$ and $M_L * Q_H$. Each 4×4 multiplication produces an 8-bit product. These results can be rearranged as shown in Figure 3.37(b). The low-order 4 bits of block 1 and the high-order 4 bits of block 4 directly provide the final answer. The low-order 4 bits of block 4, high-order 4 bits of block 1, and all bits of blocks 2 and 3 must be added in the right order to obtain the



a. 8×8 Multiplication Using 4×4 Multipliers



b. Rearrangement of the Results
Produced by the 4×4 Multipliers

c. Dot Pattern

Figure 3.37 8×8 Multiplier Design

final answer. Therefore, blocks 1 and 4 are concatenated as a 16-bit number and placed in the middle of blocks 2 and 3.

The dot pattern corresponding to the rearranged products is shown Figure 3.38. The top row is block 2 (8 bits), the middle row contains the concatenation of blocks 1 and 4 (16 bits), and the bottom row is block 3 (8 bits).

It is apparent that the overlapping portions of these three rows provide a maximum column height of 3, and, therefore, eight 3-input Wallace tree adders are required. For the most-significant (high 4 bits of block 4) and the least-significant (low 4 bits of block 1) 4 columns, no Wallace tree adders are required, since the column height is 1. These observations are indicated by the number sequence 0 3 3 0 (0 means no Wallace tree adders are required, and 3 means a 3-input Wallace tree adder is required) underneath the rearranged pattern of Figure 3.37(b). Finally, a 12-bit CLA is used to obtain the 16-bit result. This process is shown by the dot pattern in Figure 3.38.

This concept can be extended to obtain 16×16 or 32×32 multipliers.

Even though a fully combinational Wallace tree multiplier operates at high speed, it is not practical for designing large multipliers, since it requires more hardware than a Braun multiplier. An alternative is the scheme proposed by L. Dadda—a modification of Wallace’s scheme to achieve some hardware savings at the expense of reduction in the overall operational speed. Interested readers are suggested to consult the original work of L. Dadda [4] for more information.

The Wallace tree was used by the multiplier of the IBM 360/91. In order to reduce hardware, however, IBM designers have employed an iterative scheme through the use of a small Wallace tree multiplier (12×12 multiplier), which is used to perform 56×56 multiplications.

So far, unsigned combinational multipliers have been discussed. In small systems, where high-speed operation is unnecessary, two unsigned numbers can be multiplied, using an iterative approach (sequential multiplier). The key to this approach is that the product is the sum of appropriately shifted multiplicands. To illustrate this, consider the design of a 4×4 unsigned sequential multiplier. The hardware configuration of such a multiplier is shown in Figure 3.39.

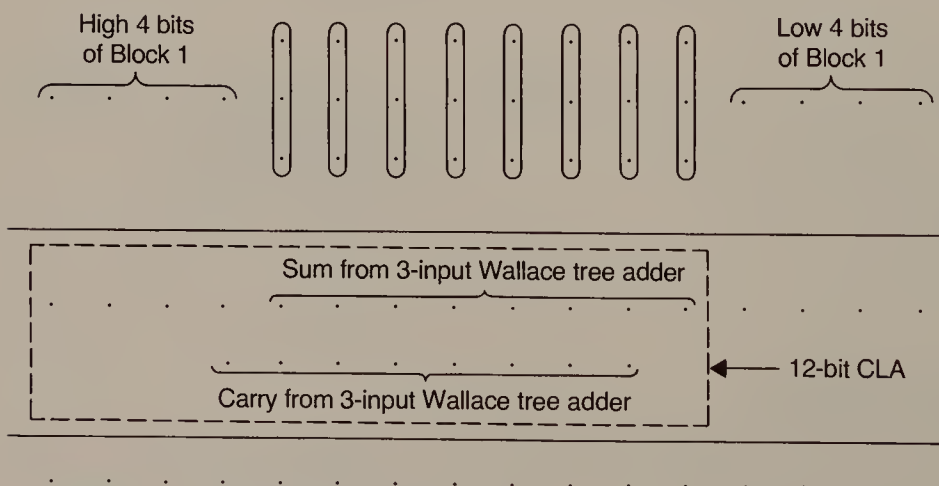
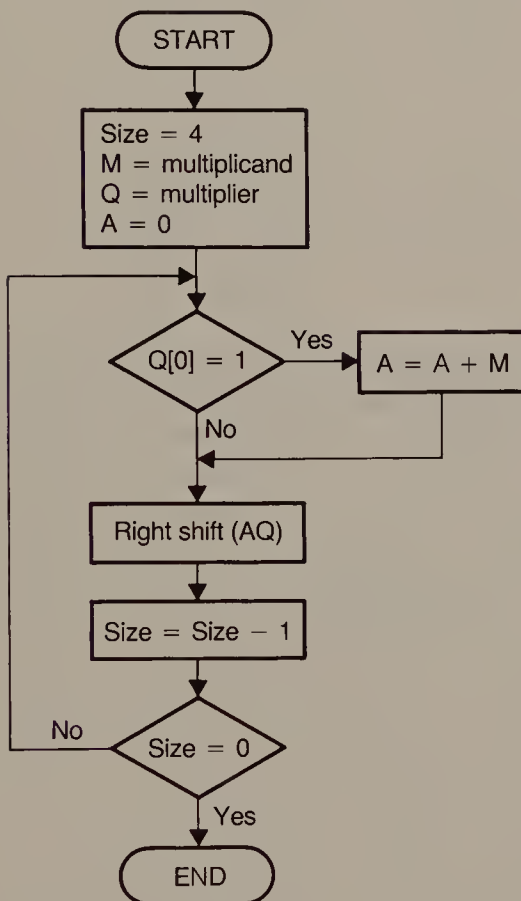


Figure 3.38 8×8 Multiplier Design

A 4×4 unsigned multiplier has three 4-bit registers, A, M, and Q. The A register, called the *accumulator*, is initially cleared to zero. When the algorithm is terminated, the A register holds the high-order 4 bits of the product. The Q register is initialized with the multiplier. When the algorithm is terminated, the Q register holds the low-order 4 bits of the product. The register pair AQ then holds the required 8-bit product. The M register always holds the multiplicand. The 4-bit parallel adder produces the sum of the contents of the A and M registers. The A register is then updated with the result produced by the parallel adder. The F flip-flop holds the end-around carry generated in the addition process. This carry is used as the serial input, while the register pair AQ is shifted one position to the right.

The multiplication algorithm can be described in the accompanying flowchart.



To illustrate this procedure, the following trace is provided, assuming that $M = 5_{10} = 0101_2$ and $Q = 12_{10} = 1100_2$:

	<i>M</i>	<i>F</i>	<i>A</i>	<i>Q</i>	<i>Size</i>
Initial configuration	0101	0	0000	1100	4

Iteration 1					
Shift AQ					
(because					
Q [0] = 0)	0101	0	0000	0110	3
Iteration 2					
Shift AQ					
(because					
Q [0] = 0)	0101	0	0000	0011	2
Iteration 3					
A := A + M					
(because					
Q [0] = 1)	0101	0	0101	0011	
Shift AQ	0101	0	0010	1001	1
Iteration 4					
A := A + M					
(because					
Q [0] = 1)	0101	0	0111	1001	
Shift AQ	0101	0	0011	1100	
					Product = 60 ₁₀

From the preceding trace, it can be seen that this procedure requires n iterations, where n is the number of bits in the multiplier. Observe that the operation $A := A + M$ is performed only when $Q[0] = 1$. Also, for $Q[0] = 0$ or 1 , the content of register

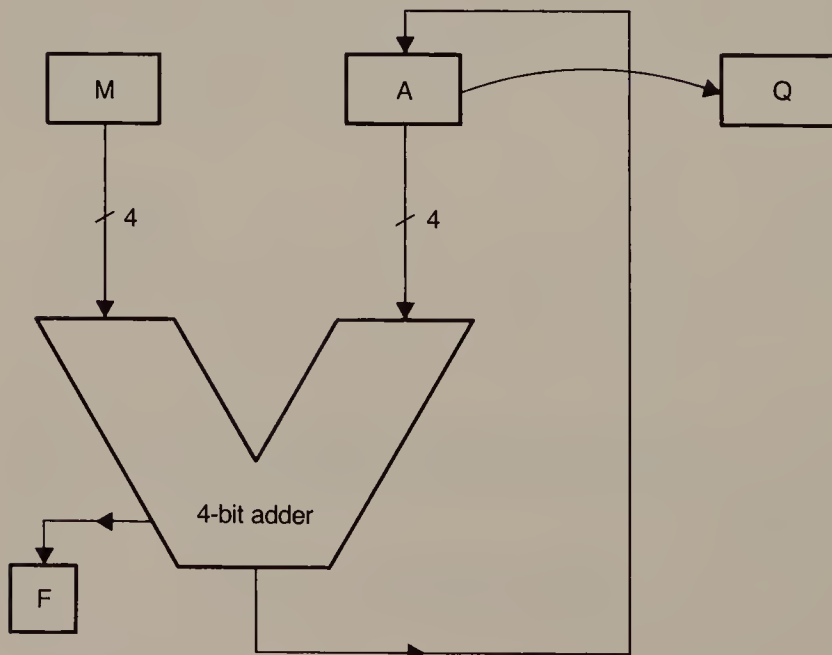


Figure 3.39 4 × 4 Unsigned Sequential Multiplier Hardware

pair AQ is shifted. In the worst case, (when all multiplier bits are 1), n additions plus n right shifts are needed. Therefore, whenever the multiplier contains fewer ones, more speed is achieved.

In the case of signed numbers, there are three possibilities:

1. M and Q are in sign-magnitude form.
2. M and Q are in 1's complement form.
3. M and Q are in 2's complement form.

For the first case, multiply the magnitudes either combinatorially or sequentially. The sign of the product is determined as $M_n \oplus Q_n$, where M_n and Q_n are the most-significant bits (sign bits) of the multiplicand (M) and the multiplier (Q), respectively. For the second case, proceed as follows:

Step 1: If $M_n = 1$, then compute the 1's complement of M.

Step 2: If $Q_n = 1$ then compute the 1's complement of Q.

Step 3: Multiply the $n - 1$ bits of the multiplier and the multiplicand either combinatorially or sequentially.

Step 4: $S_n = M_n \oplus Q_n$

Step 5: If $S_n = 1$, then compute the 1's complement of the result obtained in Step 3.

Whenever the 1's complement of a negative number (sign bit = 1) is taken, the sign is reversed. Hence, with respect to the multiplier, the inputs are always a positive quantity. When the sign of the result is negative, however, ($M_n \oplus Q_n = 1$), the result must be represented in 1's complement form. This is why the 1's complement is computed of the product found by the unsigned multiplier. When M and Q are in 2's complement form, the same procedure is repeated, with the exception that the 2's complement must be determined when $Q_n = 1$, $M_n = 1$, or $Q_n \oplus M_n = 1$. Consider M and Q as 2's complement numbers. Suppose $M = 1100_2$ and $Q = 0111_2$. Since $M_n = 1$, take the 2's complement of $M = 0100_2$; since $Q_n = 0$, do not change Q. Multiply 0111_2 and 0100_2 using the unsigned multiplier (sequential or combinations) discussed before. The product is 00011100_2 . The sign of the product is $S_n = M_n \oplus Q_n = 1 \oplus 0 = 1$. Hence, take 2's complement of the product 00011100_2 to obtain 11100100_2 , which is the final answer: -28_{10} .

The multiplication of signed numbers requires extra processing steps besides the core unsigned multiplication. This is an overhead when operands are denoted in 1's or 2's complement form. The overhead is at a maximum when M and Q are in 2's complement form, since computation of 2's complement requires an increment operation besides the bit-by-bit complement. This overhead can be eliminated by the recoded multiplication approach, in which the multiplier is mapped in accordance with the recoding procedure, a specific mapping rule.

The key to all recording procedures is the string property:

In a binary sequence, a block of consecutive k 1s may be replaced with a block of $k - 1$ consecutive 0s surrounded by the digits 1 and $\bar{1}$.

For example, consider the following sequence:

$$\begin{array}{c} 5 \text{ ones} \\ 0 \ 0 \ \overbrace{11111} \ 0 \end{array}$$

By the string property it may be rewritten as follows;

$$\begin{array}{c} 0 \ 0 \ 11111 \ 0 \\ \Downarrow \\ 0 \ 1 \ 0000\bar{1} \ 0 \end{array}$$

In the rewritten form, a block of four 0s surrounded by the digits 1 and $\bar{1}$ is present. If a weight of -1 is assigned to the digit $\bar{1}$, assuming that the original sequence represents a 2's complement number, both the original and the new patterns represent the same number. This is shown next:

$$\begin{array}{lcl} & -2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0 & \\ \text{Original pattern:} & 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 & = 2^5 + 2^4 + 2^3 + 2^2 + 2^1 = 62_{10} \\ \text{Recoded pattern:} & 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ \bar{1} \ 0 & = 1 \cdot 2^6 + (-1)2^1 = 62_{10} \end{array}$$

It must be emphasized that the string property also works if the sequence is interpreted as an unsigned integer. Since the string property increases the density of zeros in a given multiplier, the number of addition (or subtraction) operations to be performed is reduced when the recoded multiplier is used. This allows for quick multiplication.

Booth's recording procedure, a method that makes use of the string property, inspects two multiplier bits and decides which action to take, as shown in Figure 3.40. In this algorithm the multiplier is scanned from the right, 2 bits at a time. These decide the action to be performed. In the beginning, the index is $j = 0$; q_{j-1} is assumed to be 0. Following are two examples that illustrate the mechanics of Booth's recording procedure:

Given multiplier (implicit value = +7)

$$\begin{array}{ccccccc} 0 & 1 & 1 & 1 & 0 \leftarrow q_{j-1} & \text{(assumed to be zero)} \\ \downarrow & \downarrow & \downarrow & \downarrow & & \\ 1 & 0 & 0 & \bar{1} & & \end{array}$$

Booth's recoded multiplier (implicit value = $8 - 1 = 7$)

Given multiplier

$$\begin{array}{ccccccccccc} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \leftarrow q_{j-1} & \text{(assumed to be 0)} \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & & \\ 0 & \bar{1} & 1 & 0 & 0 & \bar{1} & 0 & 0 & & \end{array}$$

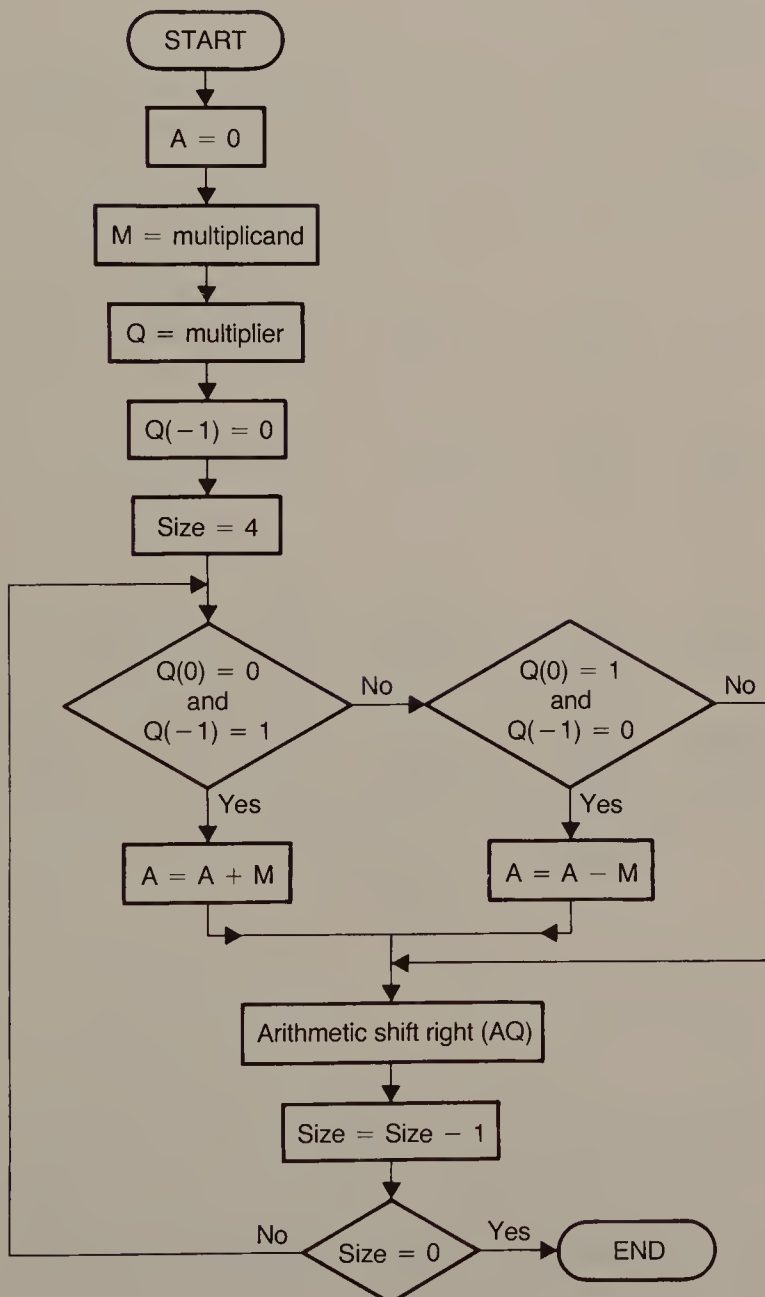
Booth's recoded multiplier (implicit value = $-64 + 32 - 4 = -36$)

(In these examples, it is assumed that the multipliers are represented in 2's complement form).

Booth's procedure can be implemented by modifying the sequential multiplier hardware (see Figure 3.39) as follows:

- Extend the Q register size from 4 to 5 bits so that the extra position will initially hold the fictitious zero.
- Replace the 4-bit parallel adder with a 4-bit adder/subtractor. (See Figure 3.21.)

The accompanying flowchart specified the mechanics of a 4×4 two's complement sequential Booth's multiplier:



Multiplier Bits Inspected $q_j q_{j-1}$	Recorded Digit Underneath Position j	Implied Action in Position j
0 0	0	None
0 1	1	Add M
1 0	$\bar{1}$	Sub M
1 1	0	None

Figure 3.40 Booth's Recording Procedure

To see how this procedure works, the following trace is provided. Assume that $M = -4 = 1100$ and $Q = 7 = 0111$.

	<i>M</i>	<i>A</i>	<i>Q</i>	
Initial configuration	1100	0000	01110	$q_{j-1} = \text{assumed } 0$ Bit pair inspected = 10
Iteration 1				
A: = A - M	1100	0100	01110	
Shift (AQ)	1100	0010	00111	Bit inspected = 11
Iteration 2				
Shift (AQ)	1100	0001	00011	Bit pair inspected = 11
Iteration 3				
Shift (AQ)	1100	0000	10001	Bit pair inspected = 01
Iteration 4				
A: = A + M	1100	1100	10001	
Shift (AQ)	1100	1110	01000	
				↓
Product =				11100100
				= 100100
				= <u>-28</u>

From this trace, notice the following:

- Preprocessing steps are unnecessary, so Booth's algorithm treats signed numbers in a uniform manner.
- Only one addition and one subtraction are needed to carry out this multiplication. (Observe that $M * 7 = M * (8 - 1) = (8M - M)$).

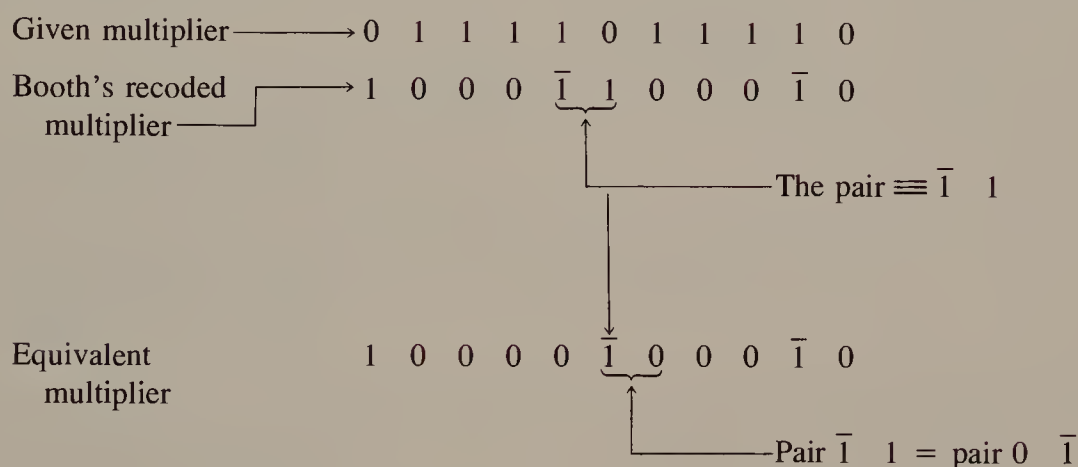
Booth's algorithm is called a *skipping over 1 technique*, since it skips over a block of consecutive ones. This process resembles the differentiation process; therefore,

Booth's recording procedure is referred to as a *differential recoding scheme*. (See N. Scott [15].)

Booth's procedure can be made more efficient if the following situations are considered:

	BOOTH'S PAIR	EQUIVALENT PAIR	REASON
Weight $\longrightarrow$	$2^n \quad 2^{n-1}$	$2^n \quad 2^{n-1}$	
	$\bar{1} \quad 1$	$0 \quad \bar{1}$	$-2^n + 2^{n-1} = -2^{n-1}$
	$1 \quad \bar{1}$	$0 \quad 1$	$2^n - 2^{n-1} = 2^{n-1}$

Booth's algorithm generates the pair $\bar{1} \ 1$ when there is a single zero and a continuous sequence of ones:



This single 0 is called an *isolated 0*. If the equivalence bit pair $\bar{1} \ 1 =$ bit pair $0 \ 1$ is applied, the final equivalent multiplier can be obtained. The final form is preferred, since it contains only three nonzero digits (whereas the Booth's multiplier contains four nonzero digits). Since the application of the equivalence rule increases the density of zeros, the multiplication can be performed quickly.

Likewise, Booth's algorithm generates the pair $1 \ \bar{1}$ when there is a single 1 in a continuous sequence of zeros:

Given multiplier	0 0 0 0 1 0 0 0
Booth's recoded multiplier	0 0 0 1 $\bar{1}$ 0 0 0

This single 1 is called an *isolated 1*. The equivalence pair $1 \ \bar{1} =$ pair $0 \ 1$ is applied to the Booth's recoded multiplier, and the original multiplier is returned. The original multiplier is preferred since it contains only one nonzero digit (whereas the Booth's recoded multiplier has two nonzero digits).

When the following modifications are introduced to the conventional Booth's recoding procedure, fewer nonzero digits result from the recoded multiplier:

- If an isolated zero is present, place a 1 into the corresponding position of the recoded multiplier, and continue the scanning process.
- Do not recode an isolated 1.

In order to implement these modifications, an additional bit m is introduced—the *mode bit*. Initially, the mode bit is reset to 0. It is set to 1 only when the beginning of a block of 1s is detected. Similarly, when the end of a 1s block is recognized, the m bit is reset to zero. When the extra bit is included, the modified Booth's recoding procedure is obtained, as shown in Figure 3.41.

When the modified Booth's recoding procedure is used, the following points must be considered:

m	q_{j+1}	q_j	Recoded Digit underneath position j	Action in Position j	Reason
0	0	0	0	None	The string of 1s has not yet started so continue scanning
0	0	1	1	Add M	This is an isolated 1, and therefore it is not recoded. This means that 1 is plugged into the recoded multiplier as well.
0	1	0	0	None	This may be an isolated 1, and thus nothing is done until the next pair is inspected.
0	1	1	$\bar{1}$	SUB M	The beginning of a string of 1s is recognized; A $\bar{1}$ is plugged in and m is set to 1.
1	0	0	1	Add M	Here, the string of 1s ends. A 1 is plugged in and m is reset to 0.
1	0	1	0	None	In this case, this 0 may be an isolated zero; nothing is done until the next pair is inspected
1	1	0	$\bar{1}$	SUB M	This 0 is an isolated 0. A $\bar{1}$ is plugged into the recoded multiplier.
1	1	1	0	None	The string of 1s continues. They are skipped without doing anything.

Figure 3.41 Modified Booth's Recoding procedure

- This procedure starts with the pair q_1q_0 . Therefore, no fictitious 0 is needed to the right of the least-significant bit of the multiplier.
- If the n -bit multiplier Q is expressed in 2's complement form, then sign-extending may be necessary
- If the n -bit multiplier Q is expressed as an unsigned integer, appending either one fictitious 0 or a pair of fictitious 0s (that is, 00) to the most-significant bit q_{n-1} may be required. In the case of unsigned integers, the most-significant bit is a positive quantity with a weighted SUM = $2^{n+1} \cdot q_{n-1}$. For 2's complement numbers, the most-significant bit indicates the sign of the number, so the weighted sum in this case becomes $(-2^{n-1}) \cdot q_{n-1}$.

Case 1.

Given multipliers	→ 1 0 1 1	Interpretation: It is in 2's complement form	Implicit value = -5
Value of m	→ 1 1 1 1 0		
Sign extension	→ 1 0 1 1		
Modified Booth's recoded multiplier	→ 0 1 0 1	Implicit value = $(-4) + (-1) = -5$	

Case 2.

Given Multiplier	→ 0 1 0 1	Interpretation: It is in 2's complement form	Implicit value = +5
Value of m	→ 0 0 0 0 0		
Sign Extension	→ 0 1 0 1		
Modified Booth's Recoded Multiplier	→ 0 1 0 1	Implicit value = +5	

Case 3.

Given multiplier	→ 1 0 1 1	Interpretation: unsigned integer	Implicit value = 11
Value of m	→ 0 1 1 1 1 0		
Pair of fictitious 0s	→ 1 0 1 1		
Modified Booth's recoded multiplier	→ 1 0 1 0 1	Implicit value = $16 - 4 - 1$ = $16 - 5 = 11$	

Case 4.

Given multiplier	→ 0 1 1 1	Interpretation: unsigned integer	Implicit value = 7
Value of m	→ 0 1 1 1 1 0		
Pair of fictitious zeros	→ $\overbrace{00}^{\text{fictitious zeros}}$ 1 0 1 1		
Modified Booth's recoded multiplier	→ 1 0 0 $\overline{1}$	Implicit value = $8 - 1 = 7$	

A multiplier recoding procedure is said to be *minimal* if it produces a minimum number of nonzero digits. A canonical multiplier recoding system is minimal and guarantees that all nonzero digits are separated by at least one 0. The modified Booth's recoding procedure is therefore a canonical recoding algorithm.

A recoding procedure can be devised that is capable of inspecting 3 bits simultaneously. This procedure, the *overlapping triplet technique*, is employed in the multiplier design of the IBM 360/91 CPU. The principal advantage of this approach is its guarantee of uniform shifting. A discussion of this topic may be found in the works of N. Scott [15] or K. Hwang [11]. Note that the method of overlapping triplets is also employed in the combinational 4×4 additive multipliers designed by manufacturers such as Texas Instruments and Advanced Micro-devices. Interested readers may consult the manufacturer's data book [17] for more information.

3.7 DIVISION OF UNSIGNED INTEGERS

This section describes how to divide two unsigned integers. Given a dividend (D) and a divisor (V), the quotient (Q) and remainder (R) are related as follows:

$$D = QV + R$$

In particular, a Q that will make the product VQ very close to D is needed. This implies that the remainder, R , of a division will always be in the range of $0 \leq R < V$. In many practical situations, only the quotient Q is desired. This means that the remainder, R , that is generated in the division process is ignored. Usually, the quotient, Q , is represented using a finite number of bits, and it may be necessary to approximate the quotient by truncation or rounding. It can then be represented by using a fixed number of allocated bits. When a high degree of accuracy is desired, however, the remainder (R) is further divided by the divisor (V) to produce a more-accurate quotient.

When examining the relation $D \approx Q \times V$, a great deal of similarity between multiplication and division can be recognized. The following are analogous pairs: the product and the dividend, the quotient and the multiplicand, and the multiplier and the divisor. Such a relationship implies that the same hardware unit used for multiplication can be used to perform division, after some modification. In a sequential multiplier, the product is obtained by adding shifted multiplicands. However, in the case of division, the quotient is obtained by repeatedly subtracting the shifted divisor from the dividend.

Subtractive division algorithms are derived from the paper and pencil approach. This process is illustrated in Figure 3.42 using $D = 22$ and $Q = 3$.

In this method, 1 quotient bit per iteration is generated. First, line up the divisor against the dividend and try to subtract it from the dividend. If the result of the subtraction is positive, then enter a 1 for the quotient, and shift the divisor one position to the right. Then, repeat the whole process. However, if the divisor cannot be subtracted from the dividend without yielding a negative result, enter a 0 for the quotient and shift the divisor to the right. Then, try to subtract the same from the dividend. This process continues until all bits of the dividend are covered.

This algorithm can be mechanized with the following modifications:

- Do not move the divisor. Instead shift the dividend to the left. When dividing two 4-bit numbers, use three 4-bit registers (M, A, and Q). The register M is used to hold the divisor. Register A and Q are initialized with the dividend. Initially, the A register is cleared to 0, and the Q register holds the 4-bit dividend. In each iteration, the contents of A and Q are shifted to the left first. The contents of M are then subtracted from the contents of A. If the result of subtraction is positive, a 1 is placed into the vacant position created by the left shift operation; otherwise a 0 is plugged into this position. The process is repeated until all the bits of the dividend originally stored in the Q register are moved into the A register. In this particular case, the entire operation must be repeated four times, since the size of the Q register is 4 bits. The A register is used as the accumulator. Therefore, the result of the subtraction is always saved in the A register. When the algorithm terminates, the A register contains the remainder and the Q register contains the quotient.
- When the result of subtraction yields a negative answer, restore the contents of A by adding the contents of A register with M before beginning the next iteration. This step explains why the algorithm is referred to as a *restoring division algorithm*.

To divide two n -bit numbers using the restoring division algorithm, repeat the steps shown in Figure 3.43 n times.

The hardware that implements the restoring division algorithm is given in Figure 3.44.

$$\begin{array}{r}
 \begin{array}{r} 011 \end{array} \overline{) \begin{array}{r} 10110 \\ 011 \\ \hline 101 \\ 011 \\ \hline 100 \\ 011 \\ \hline 011 \end{array}} \\
 \begin{array}{r} 111 \leftarrow \text{Quotient} \\ \\ \\ \\ \\ \\ 011 \leftarrow \text{Remainder} \end{array}
 \end{array}$$

Figure 3.42 Paper and Pencil Division Technique

Step 1: Shift A and Q registers to the left.

Step 2: Subtract M from A.

Step 3: If the result of the subtraction is positive, set $Q[0] = 1$; otherwise set $Q[0] = 0$.

Figure 3.43 Basic Steps of the Restoring Division Algorithm

	<i>M</i>	<i>A</i>	<i>Q</i>	
Initial				
Configuration	00011	00000	1011	
<i>Iteration 1</i>				
Left shift (AQ)	00011	00001	011?	
$A = A - M$	00011	11110	011?	
Restore A and Reset Q [0] to 0 (since A [4] = 1)	00011	00001	0110	
<i>Iteration 2</i>				
Left shift (AQ)	00011	00010	110?	
$A = A - M$	00011	11111	110?	
Restore A and reset Q [0] to 0 (since A [4] = 1)	00011	00010	1100	
<i>Iteration 3</i>				
Left shift (AQ)	00011	00101	100?	
$A - A - M$	00011	00010	100?	
Set Q [0] to 1 (since A [4] = 0)	00011	00010	1001	
<i>Iteration 4</i>				
Left shift (AQ)	00011	00101	001?	
$A = A - M$	00011	00010	001?	
Set Q [0] to 1 (since A [4] = 0)	00011	00010	0011	Quotient = 3 Remainder = 2

From the preceding trace, notice that the quotient is 3 and the remainder is 2.

The restoring division algorithm discussed so far needs n subtractions and extra additions when dividing two n -bit numbers, due to the restoration process. The exact number of extra additions depends on the data, so it cannot be figured in deterministic manner. Assuming, however, that 50% of the subtraction operations yield a negative result, then $n/2$ extra additions are needed. By properly restructuring the restoring divi-

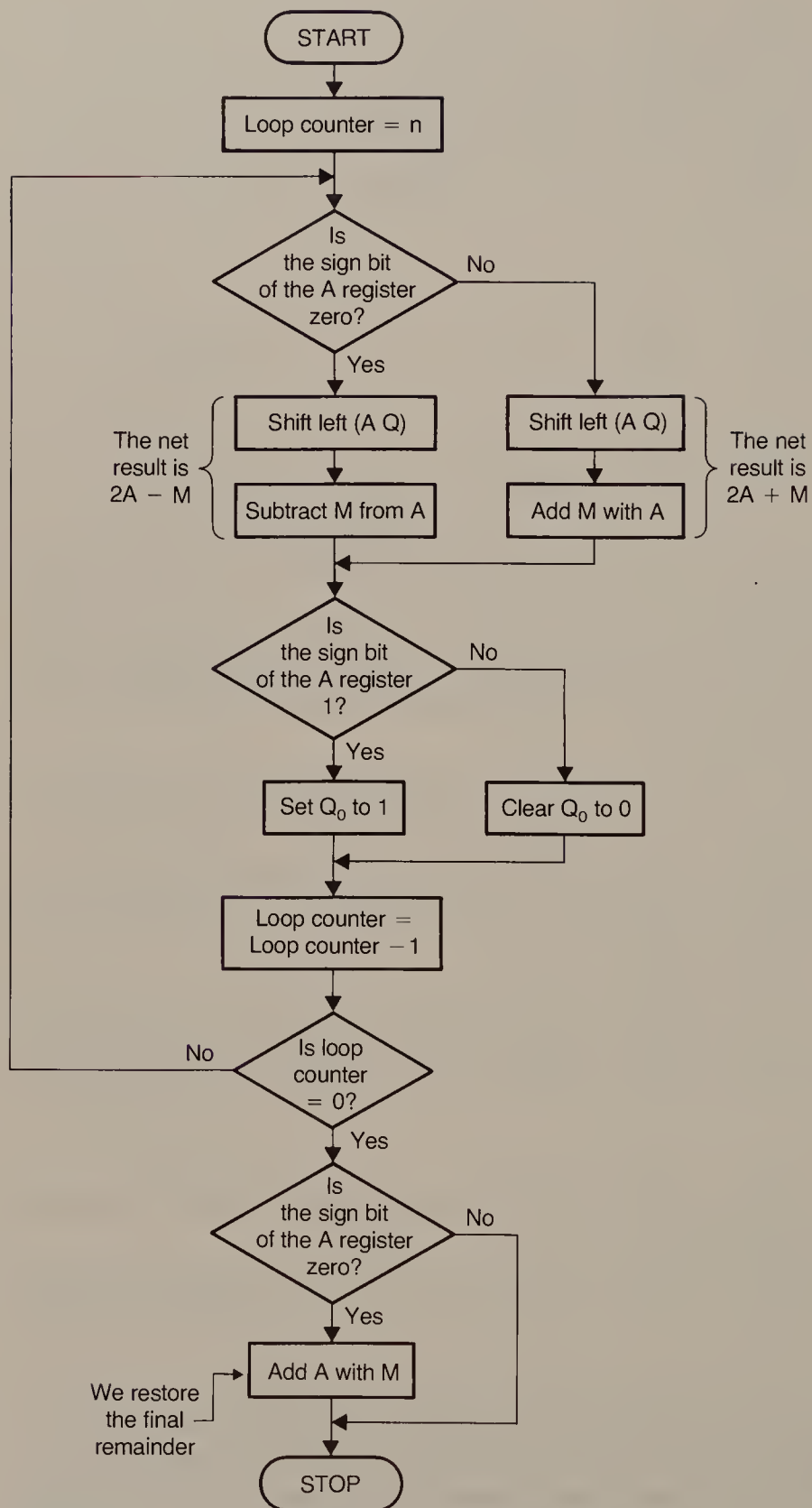


Figure 3.45 Salient Characteristics of the Nonrestoring Division Algorithm

To appreciate this concept, rework the previous division example using the nonrestoring division algorithm. Shown next is a complete trace of this process.

	<i>M</i>	<i>A</i>	<i>Q</i>
Initial configuration	00011	00000	1011
Iteration 1			
(Note that $A[4] = 0$)			
Shift left (AQ)	00011	00001	011-
$A: = A - M$	00011	11110	011-
Clear $Q[0]$			
(since $A[4] = 1$)	00011	11110	0110
Iteration 2			
(Note that $A[4] = 1$)			
Shift left (AQ)	00011	11100	110-
$A: = A + M$	00011	11111	110-
Clear $Q[0]$			
(Since $A[4] = 1$)	00011	11111	1100
Iteration 3			
(Note that $A[4] = 1$)			
Shift left (AQ)	00011	11111	100-
$A: = A + M$	00011	00010	100-
Set $Q[0]$ to 1			
(since $A[4] = 0$)	00011	00010	1001
Iteration 4			
(Note that $A[4] = 0$)			
Shift left (AQ)	00011	00101	001-
$A: = A - M$	00011	00010	001-
Set $Q[0]$ to 1			
(since $A[4] = 0$)	00011	00010	0011

Quotient = 3

Remainder = 2

In this example, there is no need to restore the remainder, since the A register already holds a positive number at the end of the fourth iteration. These algorithms may be extended to handle signed integers if both the dividend and the divisor are mapped to positive values. The sign of the result must be figured later, as in the case of multiplication. A discussion of this procedure may be found in the work of S. Waser and M. J. Flynn [19].

Another approach to carry-out division is through multiplication. Although this method appears unorthodox, it is the method employed by such mainframe computer

manufacturers as Honeywell and Amdahl in their high-speed models. The key to this concept is that D/V is equivalent to D times $1/V$. Therefore, by finding the reciprocal of the divisor V , the quotient can be computed by performing multiplication.

To compute the reciprocal of a number V , use an iterative algorithm, such as Newton-Raphson.

The Newton-Raphson algorithm is normally used to find the root of a equation of the form $f(x) = 0$. Here, the trial root x_i is found, so a better estimate is obtained by using the following recursive formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

The mechanics of this process may be graphically illustrated, as shown in Figure 3.46.

Note that $f'(x_i)$ is the derivative of $f(x)$ when $x = x_i$.

Suppose that $f(x) = x - 1/V$. Then $f(x) = 0$ implies that $x = 1/V$. Therefore, the reciprocal-finding problem is nothing other than solving the equation $f(x) = 0$, where $x = 1/V$. In this situation, the following results are obtained:

$$f(x_i) = \frac{1}{x_i} - V$$

$$f'(x_i) = \frac{-1}{x_i^2}$$

Therefore,

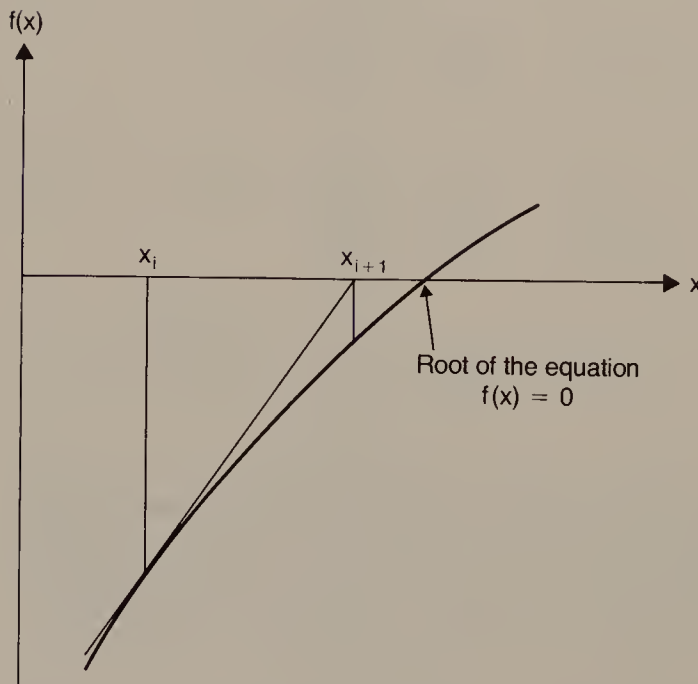


Figure 3.46 Solving the Equation $f(x) = 0$ Using the Newton-Raphson Algorithm

$$\begin{aligned}
x_{i+1} &= x_i - \frac{f(x_i)}{f'(x_i)} \\
&= x_i + \frac{1/x_i - V}{1/x_i^2} \\
&= x_i + (1 - Vx_i)x_i \\
&= 2x_i - Vx_i^2 \\
&= x_i(2 - Vx_i)
\end{aligned}$$

Thus, the general Newton-Raphson recursion formula becomes:

$$x_{i+1} = x_i(2 - Vx_i)$$

The following example illustrates the effectiveness of this approach.



EXAMPLE 3.2

Using Newton-Raphson, find $1/V$ when $V = 0.42$.

Solution

Assume that $x_0 = 1$, which provides the following results:

ITERATION NUMBER	NEW ESTIMATE	ERROR
1	$x_1 = X_0(2 - VX_1) = 1.58000$	0.80095
2	$x_2 = X_1(2 - VX_1) = 2.11151$	0.26944
3	$x_3 = X_2(2 - VX_2) = 2.35046$	0.03049
4	$x_4 = X_3(2 - VX_3) = 2.38056$	0.00039

Therefore, the approximate answer is 2.38056.

Example 3.2 shows that the error decreases quadratically. In fact, it can be proven that $E_{i+1} \leq E_i^2$, where E_i is the error in the i th iteration. The result shows that the Newton-Raphson approach is a quadratic convergence algorithm. Such a proof may be found in the work of S. Waser and M. J. Flynn [19].

In a practical implementation, a ROM is used to find the first estimate with a significance of 8 bits. The second iteration produces a 16-bit significant estimate, and the third iteration produces a 32-bit significant estimate.

In the IBM 360/91 CPU, the division is performed through iterative multiplications. This procedure is based on the binomial expansion of $1/V$. Readers may consult the original paper by M. J. Flynn [5] to find more information. In his research paper, Flynn illustrates the equivalence between binomial expansion and the Newton-Raphson techniques.

3.8 COPROCESSORS*

In typical 8-bit microprocessors such as the Intel 8085 and Z=80, technology places a limit on the chip area. In consequence, these CPUs include no hardware or firmware for performing scientific computations such as floating-point arithmetic, matrix manipulation, and graphic-data processing. Therefore, users of these systems must write these operation programs. Unfortunately, this approach is unacceptable in high-speed applications, since program execution takes a significant amount of time. To eliminate this problem, coprocessors are used.

In this approach, a single chip is built for performing scientific computations at high speed. However, the chip is regarded as a companion to the original or host CPU. Typically, each special operation is encoded as an instruction that can be interpreted only by the companion CPU. When the companion CPU encounters one of these special instructions, it assumes the processing functions independent of the host CPU. The companion CPU that operates in this manner is called the coprocessor. Therefore, this concept not only extends the capabilities of the host CPU but also increases the processing rate of the system. The coprocessor concept is widely used with typical 32-bit microprocessors such as the Motorola 68020 and Intel 80386.

Functionally, the coprocessor provides a logical extension of the programmer's model in the way of instructions, registers, and operand types. This extension is transparent to the programmer.

It is important to make the distinction between standard peripheral hardware and a coprocessor. A coprocessor is a device that has the capability of communicating with the main processor through the protocol defined as the coprocessor interface. As mentioned before, the coprocessor also adds additional instructions, registers, and data types that are not directly supported by the main processor. The coprocessor provides capabilities to the user without appearing to be hardware external to the main processor.

Standard peripheral hardware, on the other hand, is generally accessed through the use of interface registers mapped into the memory space of the main processor. The programmer uses standard processor instructions to access the peripheral interface registers and thus utilize the services provided by the peripheral. It should be pointed out that even though a peripheral can provide capabilities equivalent to a coprocessor for many applications, the programmer must implement the communication protocol between the main processor and the peripheral necessary to use the peripheral hardware. There are three general techniques that may be used to pass commands to a coprocessor. These are: intelligent monitor interface, coprocessors with special signals and instructions, and coprocessors using special signals.

3.8.1 Intelligent Monitor Interface

Figure 3.47 shows a block diagram of the *intelligent monitor interface*. The coprocessor must monitor the instruction stream by obtaining commands directly from the bus at the same time as the main processor. The Intel 8087 floating-point coprocessor is of this type, as it monitors the instruction stream simultaneously with a main processor such as

*Modified from ELECTRONICS, November 17, 1983 issue. Copyright © 1983, McGraw-Hill, Inc. All rights reserved. Reprinted with permission.

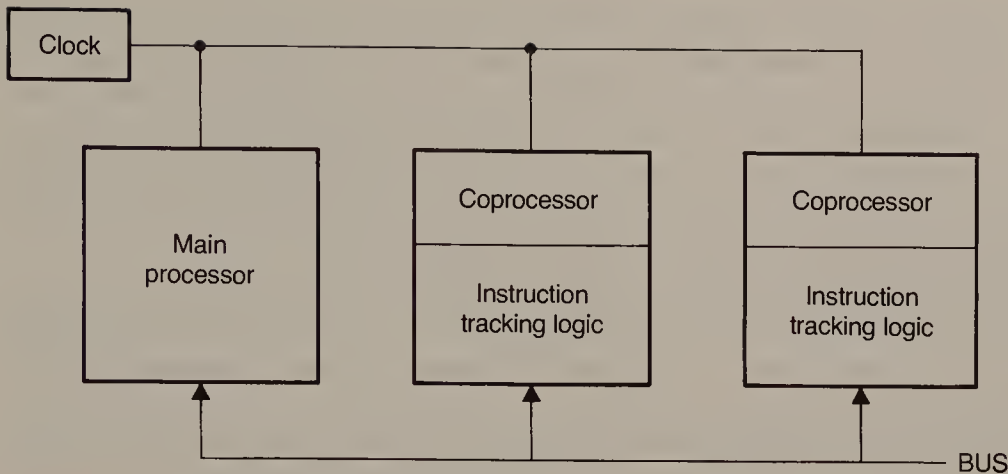


Figure 3.47 Intelligent Monitor Interface

the Intel 8086. Since the 8086 uses an instruction queue, the coprocessor must maintain an identical queue. This has the obvious advantage of requiring no additional bus cycles to pass the content of the instruction word to the coprocessor. There are several disadvantages, however:

- Each coprocessor in the system must duplicate the bus monitoring circuitry and instruction queue, tracking all branches, wait states, operand fetches, and instruction fetches.
- Simultaneous bus-monitoring techniques require that the processor/coprocessor pair operate no faster than the capabilities of the slowest member of the pair, even when no coprocessor instructions are executed.

3.8.2 Interface Using Special Bus Signals and Instructions

The coprocessor may be implicitly addressed by certain instructions that invoke a special protocol; a special bus line tells the coprocessor when to listen to the instruction stream. Figure 3.48 shows a block diagram of this type of coprocessor interface.

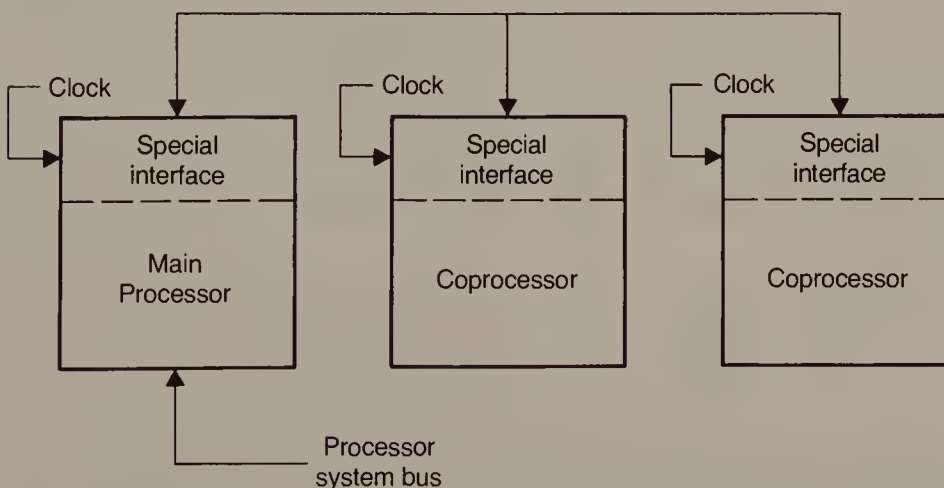


Figure 3.48 Coprocessor Using Special Signals and Instructions

An important attribute of this technique is that the main processor must have a prior knowledge of the coprocessor if the two are to work together effectively. For example, the main processor must know all data sizes of all coprocessors that are to be attached. An advantage of this method is that the coprocessor may run at different speeds since data transfer occurs asynchronously via handshaking. One of the main disadvantages is the extra CPU overhead. That is, the main processor has to recognize a coprocessor instruction and direct it to the appropriate coprocessor.

The National NS16081 is an example of this type. A special signal, an address translation/slave processor control, is used as a bidirectional handshake line between the main processor and coprocessor. In the National implementation, the coprocessor does not monitor the original instruction fetch by the processor, and the circuitry required to monitor and store the instruction queue is not required. However, the coprocessor identity is specific to a coprocessor type such that only one coprocessor of a given type is permitted in a system. As implemented by National, only sequential (nonconcurrent) operations are permitted. The main processor must always read status at the end of each command.

3.8.3 Coprocessor Interface Using Special Instructions

The coprocessor may be explicitly addressed by certain instructions, which initiate a special sequence of microinstructions in the main processor to effect command and operand transfer. Figure 3.49 is a block diagram of a system configured using this technique.

In this approach, when the main processor executes a coprocessor instruction, it decodes the instruction and writes a command in the command register (one of the interface registers) specifying the operation required by the coprocessor. In response, the coprocessor writes data back in a register called the *response register* (one of the interface registers). The main processor can read this data, and it tells the main processor certain information such as whether additional information is required by the coprocessor to carry out the operation. If such data is required, the main processor provides

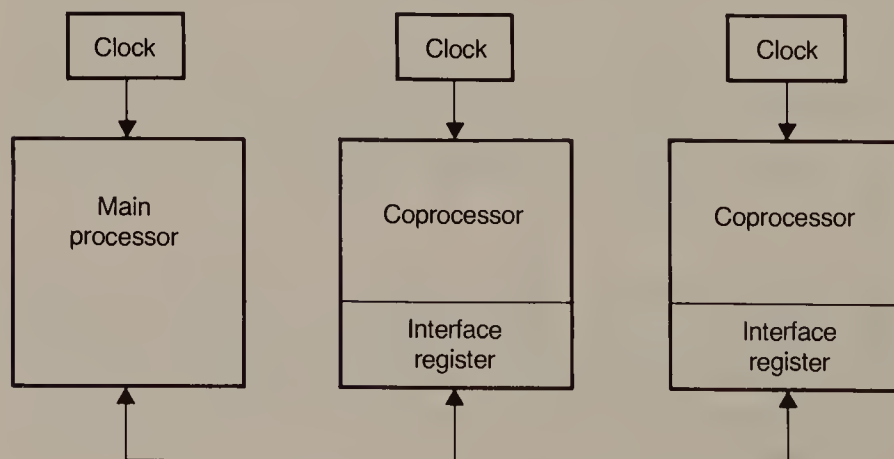


Figure 3.49 Coprocessor Interface Using Special Instructions

this; otherwise, the coprocessor carries out the operation concurrently with the main and provides the result.

Some of the advantages of this method are as follows:

- No special signals are required.
- No inherent limit to the number of coprocessors.

One of the main disadvantages of this method is that once the main processor detects a coprocessor instruction, the main has to use bus bandwidth and timing to transmit the command to the appropriate coprocessor. The Motorola 68881 (floating-point coprocessor) is of this type.

The MC68881 floating-point coprocessor is intended primarily for use as a coprocessor to the MC 68020 32-bit microprocessor for providing a logical extension to the 68020 arithmetic capabilities. The major features of the MC68881 are these:

- Eight general-purpose floating-point data registers, each supporting a full 80-bit extended precision real-data format (a 64-bit mantissa plus a sign bit and a 15-bit signed exponent).
- A 67-bit ALU to allow very fast calculations.
- A 67-bit barrel shifter for high-speed shifting operations.
- Forty-six instructions, including 35 arithmetic operations.
- Full set of trigonometric and logarithmic functions.
- Twenty-two constants available in the on-chip ROM including π , e , and powers of 10.

The MC68881 is internally divided into three processing elements: the bus interface unit (BIU), the execution control unit (ECU), and the microcode control unit (MCU). The BIU contains the coprocessor interface register and the 32-bit control, status, and instruction address registers. The eight 80-bit floating-point data registers are located in the ECU. In addition, the ECU contains the 67-bit ALU for both mantissa and exponent calculations, the 67-bit barrel shifter and the ROM constant. The MCU contains the clock generator, a two-level microcoded sequencer that controls the ECU, the microcode ROM, and self-test circuitry.

When the 68020 detects a typical 68881 instruction, the 68020 writes the instruction to the memory-mapped command register in the 68881 and then reads the response register in the 68881. In this response, the BIU encodes requests for any additional action required of the 68020 on behalf of the 68881. Once the 68020 fulfills the 68881 request, the 68020 is free to fetch and execute subsequent instructions.

QUESTIONS AND PROBLEMS

- 3.1** Explain the significance of CPU registers.
- 3.2** Consider the following statement: The number of CPU registers provides some measure of the power of a processor. Comment on the validity of this statement.

3.3 Design a 4-bit general purpose register as follows:

s_1s_0 FUNCTION

00 Load external data

01 Rotate left ($A_3 \leftarrow A_0$; $A_i \leftarrow A_{i-1}$ for $i = 1, 2, 3$)

10 Rotate right ($A_0 \leftarrow A_3$; $A_i \leftarrow A_{i+1}$ for $i = 0, 1, 2$)

11 Increment

3.4 Design a full 18×16 barrel shifter.

3.5 Explain the importance of the flag register in a computer.

3.6 Prove the validity of the results shown in Figure 3.10.

3.7 Show that the carry output of a full adder can be expressed as $c_{i+1} = y_i y_i + (x_i \oplus y_i) c_i$. What is the advantage of this representation?

3.8 Using a 4-bit CLA as a building block, design a 16-bit adder whose worst-case add-time is 10Δ .

3.9 Using a 4-bit CLA as the building block, design the fastest 64-bit adder. Estimate the worst-case add-time of your design.

3.10 Design a combinational circuit to compute the function $f(x) = (3/8)*x$, where x is a 4-bit 2's complement number.

3.11 Suppose there are n operands where $n = 2^k$ for some $k > 1$. Show that the series-parallel multioperand summation approach requires $n - 1$ adders. Also, show that when there are m -bits per operand, one add-time T will be

$$T = (m * \text{clock period}) + (\log_2 k) * (\text{full adder time})$$

3.12 Design a combinational circuit to compute the absolute value of the 4-bit, 2's complement number x . Use 4-bit adders and exclusive-Or gates.

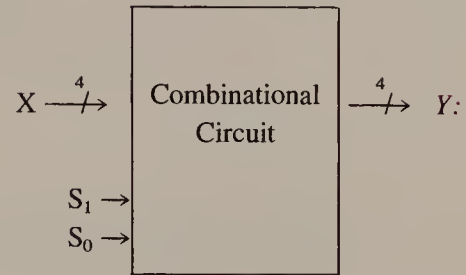
3.13 Design and implement a combinational circuit that will work as follows:

S_1	S_0	F
0	0	A plus B
0	1	Shift left (A)
1	0	A plus B plus 1
1	1	Shift left (A) + 1

Note that A and B are 4-bit operands

3.14 a. Design a combinational circuit that will satisfy the following specification.

S_1	S_0	Y
0	0	0
0	1	0
1	0	X
1	1	X



Using the results of part (a), design a 4-bit, 8-function arithmetic unit that
b. will function as described next:

S_2	S_1	S_0	F
0	0	0	A
0	0	1	A plus B
0	1	0	A plus B'
0	1	1	A minus 1
1	0	0	A plus 1
1	0	1	A plus B plus 1
1	1	0	A plus B' plus 1
1	1	1	A

3.15 Design a 4-bit, 8-function arithmetic unit that will meet the following specifications:

S_2	S_1	S_0	F
0	0	0	2A
0	0	1	A plus B'
0	1	0	A plus B
0	1	1	A minus 1
1	0	0	2A plus 1
1	0	1	A plus B' plus 1
1	1	0	A plus B plus 1
1	1	1	A

- 3.16 a.** Using a 4-bit parallel adder with inputs A, B, and C_{in} outputs F and C_{out} , and one selection bit S_0 , design an arithmetic circuit as follows:

S_0 FUNCTION TO BE PERFORMED

0 A plus B

1 B plus 1

- b.** Using another selection bit S_1 , modify the circuit of (a) to include the arithmetic and logic functions as follows:

S_1 S_0 FUNCTION TO BE PERFORMED

0 0 F = A plus B

0 1 F = B

1 0 F = shift left (logical) A

1 1 F = A'

- 3.17** Design a 4-bit logic unit that will function as follows:

S_1	S_0	F
0	0	$A \vee B$
0	1	$A \wedge B$
1	0	A'
1	1	$A \oplus B$

- 3.18** Point out four important features of the Am 2901 bit-slice processor.

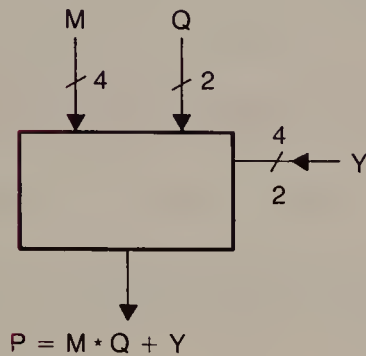
- 3.19** In the AM 2901, why are the following features provided?

- a.** Zero as an ALU input
- b.** Two-port RAM
- c.** Q and RAM shifters
- d.** $F = 0$ output
- e.** Q register

- 3.20** Write a suitable AM 2901 code sequence to perform each one of the following operations:

- a.** $M[0] \leftarrow (M[0] \text{ plus } M[1])/2$
- b.** $M[5] \leftarrow 0$
- c.** $M[2] \leftarrow -M[3]$
- d.** $M[1] \leftarrow \text{average of } M[1] \text{ through } M[4]$

- 3.21** Design and implement a 6×6 Braun's multiplier.
- 3.22** Design an unsigned 8×4 non-additive multiplier using additive-multiplier-module whose block diagrams representation is as follows:



Assume that M , Q , and Y are unsigned integers.

- 3.23** Using four 256×8 ROMs and 4-bit parallel adders, design a 8×8 unsigned, nonadditive multiplier. Draw a logic diagram of your implementation.
- 3.24** Comment on the hardware requirements for a fully combinational 32×32 Wallace tree multiplier.
- 3.25** Using add-shift method, perform the following 5×5 multiplication $10101 * 11001$ ($21 * 25$)
- 3.26** Perform the following two's complement multiplication $110011 * 110110$ ($-13 * -10$) using a) Booth's Algorithm and b) Modified Booth's Algorithm.
- 3.27** Perform the division $111 \overline{)101011}$ using each algorithm.
- a. Restoring division algorithm
 - b. Nonrestoring division algorithm
- 3.28** Explain how to extend the division procedures described in this chapter to handle signed numbers.
- 3.29** Using the Newton-Raphson method, find $1/V$, where $V = 0.60$.
- 3.30** How is the Newton-Raphson iteration method extended to find $\sqrt{N}$? Formulate $f(x)$ and derive the necessary recursive relation. Using this relation, find $\sqrt{5}$. Assume that $X_0 = 1$.
- 3.31** Explain the differences between a coprocessor and a peripheral device.
- 3.32** What are the various types of coprocessor interfaces? Discuss them briefly.

REFERENCES

1. Advanced Micro Devices, *Bipolar Microprocessor Logic and Interface Databook*. 1982.
2. Advanced Micro Devices. *AM 9500 Peripheral Products Interface Guide*. 1980.
3. Anderson, S. F., J. G. Earle, R. E. Goldschmit, and D. M. Powers. "The IBM System/360 Model 91: floating-point execution unit." *IBM Journal of Research and Development* 11, no. 1 (January 1967) 34–53.
4. Dadda, L. "Some Schemes for Parallel Multipliers." *Alta Frequenza* 34, no. 5 (May 1965) 349–356.
5. Flynn, M. J. "On Division by Functional Iteration." *IEEE Transactions on Computers* C-19, no. 8 (August 1970) 702–706.
6. Groves, Stan. "The Making of a Coprocessor." *Electronics* (April 29, 1983).
7. Hallin, T. G., and M. J. Flynn. "Pipelining of Arithmetic Functions." *IEEE Transactions on Computers* C-21, no. 8 (August 1972) 880–886.
8. Hamacher, V. C., Z. G. Vranesic and S. G. Zaky. *Computer Organization*. New York: McGraw-Hill, 1984.
9. Hayes, J. P., *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
10. Hayes, J. P., *Digital System Design and Microprocessors*. New York: McGraw-Hill, 1984.
11. Hwang, K. *Computer Arithmetic: Principles and Practice*. New York: John Wiley, 1979.
12. Mano, M. M., *Computer System Architecture*. Englewood Cliffs, N.J.: Prentice-Hall, 1982.
13. Palmer, J., R. Nave, C. Wymore, R. Koehler, and C. McMinn. "Making Mainframe Mathematics Accessible to Microprocessors." *Electronics* (May 1980).
14. Roth, C. *Fundamentals of Logic Design*. St. Paul: West Publishing, 1981.
15. Scott, N. *Computer Number Systems and Arithmetic*. Englewood Cliffs, N.J.: Prentice-Hall, 1985.
16. Singh, S., and R. Waxman. "Multioperand Addition and Multiplication." *IEEE Transactions on Computers* C-22, no. 2 (February 1973) 113–120.
17. Texas Instruments, "TTL Data Book for Design Engineers", 1972.
18. Wallace, C. S., "A Suggestion for A Fast Multiplier." *IEEE Transactions on Electronic Computers* EC-13, no. 1 (February 1964) 14–17.
19. Waser, S., and M. J. Flynn. *Introduction to Arithmetic for Digital Systems Designers*. New York: Holt, Reinhart & Winston, 1982.